Human review packet: FreundSchapire1997Hedge

Generated from byte-pinned source excerpts, the expanded PaperInterface specifications, and hash-bound raw-source-to-Spec screening records.

Paper: FreundSchapire1997Hedge
Paper id: FreundSchapire1997Hedge
Source version: Journal of Computer and System Sciences 55(1):119-139 (1997); byte-pinned official PDF transcript
Source record: audit/paper_statement_map.json
Rows: 14
Generated: 2026-09-06
Source URL: official source

How to use this packet

For each source claim, compare the exact source excerpts with the expanded PaperInterface specification. Mark up this PDF or fill the blank reviewer box in the TeX source; return the annotated file for reconciliation into the paper-local human-review log.

Open the interactive dashboard

The interactive dashboard is an optional alternative to using this PDF; it presents the same review surface rather than an additional review requirement. After forking and cloning (or pulling) AppliedModelingLib, run the following from the repository root. The cache command is needed only the first time in a new clone.

```
cd <your-repository-checkout>
lake exe cache get
python3 scripts/review_dashboard.py --paper FreundSchapire1997Hedge --serve
```

Open http://127.0.0.1:8765/ in a browser and keep that terminal running while reviewing. The dashboard records saved annotations in this paper's local reviewer-owned review record.

Contents

- lemma4_learning_rate_boundSpec
- theorem5_decision_prediction_generalSpec
- theorem6_adaBoost_finiteSpec
- theorem8_weighted_threshold_vcSpec
- theorem9_soft_adaBoost_errorSpec
- equation21_adaBoost_binaryKLSpec
- equation22_adaBoost_uniform_edgeSpec
- equation23_adaBoost_quadratic_iterationsSpec
- theorem10_adaBoostM1_errorSpec
- theorem11_adaBoostM2_errorSpec
- theorem12_adaBoostR_errorSpec

Governing Lean declarations

These exact graph-bound declaration bodies are retained by the displayed specifications or reviewed prerequisites. They are supporting context and do not add source-result rows or semantic judgments.

AppliedModelingLib.FiniteProbabilitySimplex

Declaration location: reusable library

Lean declaration body

type:
{α : Type u_1} → [Fintype α] → (α → ℝ) → Prop

value:
fun {α : Type u_1} [Fintype α] (mass : α → ℝ) => (∀ (i : α), 0 ≤ mass i) ∧ ∑ i : α, mass i = 1

AppliedModelingLib.Learning.Online.AdaBoostM2Instance

Declaration location: reusable library

Lean declaration body

type:
(Training : Type u_1) → (Label : Type u_2) → (Training → Label) → Type (max 0 u_1 u_2)

value:
fun (Training : Type u_1) (Label : Type u_2) (label : Training → Label) =>
  { pair : Training × Label // pair.2 ≠ label pair.1 }

AppliedModelingLib.Learning.Online.FiniteAllocationPolicy

Declaration location: reusable library

Lean declaration body

type:
(Expert : Type u_1) → [Fintype Expert] → Type u_1

value:
fun (Expert : Type u_1) [Fintype Expert] => List (Expert → ℝ) → Expert → ℝ

AppliedModelingLib.Learning.Online.FiniteAllocationPolicyAdmissible

Declaration location: reusable library

Lean declaration body

type:
{Expert : Type u_1} → [inst : Fintype Expert] → AppliedModelingLib.Learning.Online.FiniteAllocationPolicy Expert → Prop

value:
fun {Expert : Type u_1} [Fintype Expert] (policy : AppliedModelingLib.Learning.Online.FiniteAllocationPolicy Expert) =>
  ∀ (history : List (Expert → ℝ)), AppliedModelingLib.FiniteProbabilitySimplex (policy history)

Direct retained dependencies

- AppliedModelingLib.FiniteProbabilitySimplex
- AppliedModelingLib.Learning.Online.FiniteAllocationPolicy

AppliedModelingLib.Learning.Online.adaBoostDiscount

Declaration location: reusable library

Lean declaration body

type:
ℝ → ℝ

value:
fun (error : ℝ) => error / (1 - error)

AppliedModelingLib.Learning.Online.adaBoostLoss

Declaration location: reusable library

Lean declaration body

type:
{Training : Type u_1} → (Training → ℝ) → (Training → ℝ) → Training → ℝ

value:
fun {Training : Type u_1} (label hypothesis : Training → ℝ) (a : Training) => 1 - |hypothesis a - label a|

AppliedModelingLib.Learning.Online.adaBoostM1BinaryHypothesis

Declaration location: reusable library

Lean declaration body

type:
{Training : Type u_1} →
  {Label : Type u_2} → [DecidableEq Label] → (Training → Label) → (Training → Label) → Training → ℝ

value:
fun {Training : Type u_1} {Label : Type u_2} [DecidableEq Label] (label hypothesis : Training → Label) (a : Training) =>
  if hypothesis a = label a then 0 else 1

AppliedModelingLib.Learning.Online.adaBoostM1BinaryHypotheses

Declaration location: reusable library

Lean declaration body

type:
{Training : Type u_1} →
  {Label : Type u_2} → [DecidableEq Label] → (Training → Label) → List (Training → Label) → List (Training → ℝ)

value:
fun {Training : Type u_1} {Label : Type u_2} [DecidableEq Label] (label : Training → Label)
    (hypotheses : List (Training → Label)) =>
  List.map (AppliedModelingLib.Learning.Online.adaBoostM1BinaryHypothesis label) hypotheses

Direct retained dependencies

- AppliedModelingLib.Learning.Online.adaBoostM1BinaryHypothesis

AppliedModelingLib.Learning.Online.adaBoostM2BinaryHypothesis

Declaration location: reusable library

Lean declaration body

type:
{Training : Type u_1} →
  {Label : Type u_2} →
    [DecidableEq Label] →
      (label : Training → Label) →
        (Training → Label → ℝ) → AppliedModelingLib.Learning.Online.AdaBoostM2Instance Training Label label → ℝ

value:
fun {Training : Type u_1} {Label : Type u_2} [DecidableEq Label] (label : Training → Label)
    (hypothesis : Training → Label → ℝ)
    (a : AppliedModelingLib.Learning.Online.AdaBoostM2Instance Training Label label) =>
  1 / 2 * (1 - hypothesis (↑a).1 (label (↑a).1) + hypothesis (↑a).1 (↑a).2)

Direct retained dependencies

- AppliedModelingLib.Learning.Online.AdaBoostM2Instance

AppliedModelingLib.Learning.Online.adaBoostM2BinaryHypotheses

Declaration location: reusable library

Lean declaration body

type:
{Training : Type u_1} →
  {Label : Type u_2} →
    [DecidableEq Label] →
      (label : Training → Label) →
        List (Training → Label → ℝ) →
          List (AppliedModelingLib.Learning.Online.AdaBoostM2Instance Training Label label → ℝ)

value:
fun {Training : Type u_1} {Label : Type u_2} [DecidableEq Label] (label : Training → Label)
    (hypotheses : List (Training → Label → ℝ)) =>
  List.map (AppliedModelingLib.Learning.Online.adaBoostM2BinaryHypothesis label) hypotheses

Direct retained dependencies

- AppliedModelingLib.Learning.Online.AdaBoostM2Instance
- AppliedModelingLib.Learning.Online.adaBoostM2BinaryHypothesis

AppliedModelingLib.Learning.Online.adaBoostRIntervalError

Declaration location: reusable library

Lean declaration body

type:
ℝ → ℝ → ℝ → ℝ

value:
fun (truth prediction y : ℝ) => if y ∈ Set.uIcc truth prediction then 1 else 0

AppliedModelingLib.Learning.Online.adaBoostRMeanSquaredError

Declaration location: reusable library

Lean declaration body

type:
{Training : Type u_1} → [Fintype Training] → (Training → ℝ) → (Training → ℝ) → (Training → ℝ) → ℝ

value:
fun {Training : Type u_1} [Fintype Training] (exampleWeight label prediction : Training → ℝ) =>
  ∑ sample : Training, exampleWeight sample * (prediction sample - label sample) ^ 2

AppliedModelingLib.Learning.Online.adaBoostRMedianCandidates

Declaration location: reusable library

Lean declaration body

type:
{Training : Type u_1} → List (Training → ℝ) → Training → Finset ℝ

value:
fun {Training : Type u_1} (hypotheses : List (Training → ℝ)) (sample : Training) =>
  insert 0 (insert 1 (List.map (fun (hypothesis : Training → ℝ) => hypothesis sample) hypotheses).toFinset)

AppliedModelingLib.Learning.Online.adaBoostRUnitMeasure

Declaration location: reusable library

Lean declaration body

type:
MeasureTheory.Measure ℝ

value:
MeasureTheory.volume.restrict (Set.Icc 0 1)

AppliedModelingLib.Learning.Online.adaBoostRDensityTotal

Declaration location: reusable library

Lean declaration body

type:
{Training : Type u_1} → [Fintype Training] → (Training → ℝ → ℝ) → ℝ

value:
fun {Training : Type u_1} [Fintype Training] (weight : Training → ℝ → ℝ) =>
  ∑ sample : Training, ∫ (y : ℝ), weight sample y ∂AppliedModelingLib.Learning.Online.adaBoostRUnitMeasure

Direct retained dependencies

- AppliedModelingLib.Learning.Online.adaBoostRUnitMeasure

AppliedModelingLib.Learning.Online.adaBoostRNormalizer

Declaration location: reusable library

Lean declaration body

type:
{Training : Type u_1} → [Fintype Training] → (Training → ℝ) → (Training → ℝ) → ℝ

value:
fun {Training : Type u_1} [Fintype Training] (exampleWeight label : Training → ℝ) =>
  ∑ sample : Training,
    exampleWeight sample * ∫ (y : ℝ), |y - label sample| ∂AppliedModelingLib.Learning.Online.adaBoostRUnitMeasure

Direct retained dependencies

- AppliedModelingLib.Learning.Online.adaBoostRUnitMeasure

AppliedModelingLib.Learning.Online.adaBoostRBaseDensity

Declaration location: reusable library

Lean declaration body

type:
{Training : Type u_1} → [Fintype Training] → (Training → ℝ) → (Training → ℝ) → Training → ℝ → ℝ

value:
fun {Training : Type u_1} [Fintype Training] (exampleWeight label : Training → ℝ) (a : Training) (a_1 : ℝ) =>
  exampleWeight a * |a_1 - label a| / AppliedModelingLib.Learning.Online.adaBoostRNormalizer exampleWeight label

Direct retained dependencies

- AppliedModelingLib.Learning.Online.adaBoostRNormalizer

AppliedModelingLib.Learning.Online.adaBoostRBaseState

Declaration location: reusable library

Lean declaration body

type:
{Training : Type u_1} →
  [inst : Fintype Training] →
    (exampleWeight label : Training → ℝ) →
      (∀ (sample : Training), 0 ≤ exampleWeight sample) →
        ∑ sample : Training, exampleWeight sample = 1 →
          (∀ (sample : Training), 0 ≤ label sample ∧ label sample ≤ 1) →
            AppliedModelingLib.Learning.Online.AdaBoostRDensityState Training

value:
fun {Training : Type u_1} [Fintype Training] (exampleWeight label : Training → ℝ)
    (hexample_nonneg : ∀ (sample : Training), 0 ≤ exampleWeight sample)
    (hexample_total : ∑ sample : Training, exampleWeight sample = 1)
    (hlabel : ∀ (sample : Training), 0 ≤ label sample ∧ label sample ≤ 1) =>
  { weight := AppliedModelingLib.Learning.Online.adaBoostRBaseDensity exampleWeight label,
    integrable := AppliedModelingLib.Learning.Online.adaBoostRBaseState._proof_1 exampleWeight label,
    nonneg :=
      AppliedModelingLib.Learning.Online.adaBoostRBaseState._proof_2 exampleWeight label hexample_nonneg hexample_total
        hlabel,
    total_pos :=
      AppliedModelingLib.Learning.Online.adaBoostRBaseState._proof_3 exampleWeight label hexample_nonneg hexample_total
        hlabel }

Direct retained dependencies

- AppliedModelingLib.Learning.Online.AdaBoostRDensityState
- AppliedModelingLib.Learning.Online.adaBoostRBaseDensity
- AppliedModelingLib.Learning.Online.adaBoostRDensityTotal
- AppliedModelingLib.Learning.Online.adaBoostRUnitMeasure

AppliedModelingLib.Learning.Online.adaBoostRReducedError

Declaration location: reusable library

Lean declaration body

```
type:
{Training : Type u_1} →
  [inst : Fintype Training] →
    AppliedModelingLib.Learning.Online.AdaBoostRDensityState Training → (Training → ℝ) → (Training → ℝ) → ℝ

value:
fun {Training : Type u_1} [Fintype Training] (state : AppliedModelingLib.Learning.Online.AdaBoostRDensityState Training)
    (label hypothesis : Training → ℝ) =>
  (∑ sample : Training,
      ∫ (y : ℝ),
        AppliedModelingLib.Learning.Online.adaBoostRIntervalError (label sample) (hypothesis sample) y *
          state.weight sample y ∂AppliedModelingLib.Learning.Online.adaBoostRUnitMeasure) /
    AppliedModelingLib.Learning.Online.adaBoostRDensityTotal state.weight
```

Direct retained dependencies

- AppliedModelingLib.Learning.Online.AdaBoostRDensityState
- AppliedModelingLib.Learning.Online.adaBoostRDensityTotal
- AppliedModelingLib.Learning.Online.adaBoostRIntervalError
- AppliedModelingLib.Learning.Online.adaBoostRUnitMeasure

AppliedModelingLib.Learning.Online.adaBoostRStepWeight

Declaration location: reusable library

Lean declaration body

```
type:
{Training : Type u_1} →
  [inst : Fintype Training] →
    AppliedModelingLib.Learning.Online.AdaBoostRDensityState Training →
      (Training → ℝ) → (Training → ℝ) → ℝ → Training → ℝ → ℝ

value:
fun {Training : Type u_1} [Fintype Training] (state : AppliedModelingLib.Learning.Online.AdaBoostRDensityState Training)
    (label hypothesis : Training → ℝ) (error : ℝ) (a : Training) (a_1 : ℝ) =>
  if a_1 ∈ Set.ulcc (label a) (hypothesis a) then state.weight a a_1
  else AppliedModelingLib.Learning.Online.adaBoostDiscount error * state.weight a a_1
```

Direct retained dependencies

- AppliedModelingLib.Learning.Online.AdaBoostRDensityState
- AppliedModelingLib.Learning.Online.adaBoostDiscount

AppliedModelingLib.Learning.Online.adaBoostRStep

Declaration location: reusable library

Lean declaration body

```
type:
{Training : Type u_1} →
  [inst : Fintype Training] →
    (state : AppliedModelingLib.Learning.Online.AdaBoostRDensityState Training) →
      (label hypothesis : Training → ℝ) →
        0 < AppliedModelingLib.Learning.Online.adaBoostRReducedError state label hypothesis ∧
            AppliedModelingLib.Learning.Online.adaBoostRReducedError state label hypothesis ≤ 1 / 2 →
          AppliedModelingLib.Learning.Online.AdaBoostRDensityState Training

value:
fun {Training : Type u_1} [Fintype Training] (state : AppliedModelingLib.Learning.Online.AdaBoostRDensityState Training)
    (label hypothesis : Training → ℝ)
    (herror :
      0 < AppliedModelingLib.Learning.Online.adaBoostRReducedError state label hypothesis ∧
        AppliedModelingLib.Learning.Online.adaBoostRReducedError state label hypothesis ≤ 1 / 2) =>
  {
    weight :=
      AppliedModelingLib.Learning.Online.adaBoostRStepWeight state label hypothesis
        (AppliedModelingLib.Learning.Online.adaBoostRReducedError state label hypothesis),
    integrable := AppliedModelingLib.Learning.Online.adaBoostRStep._proof_2 state label hypothesis,
    nonneg := AppliedModelingLib.Learning.Online.adaBoostRStep._proof_3 state label hypothesis herror,
    total_pos := AppliedModelingLib.Learning.Online.adaBoostRStep._proof_4 state label hypothesis herror }
```

Direct retained dependencies

- AppliedModelingLib.Learning.Online.AdaBoostRDensityState
- AppliedModelingLib.Learning.Online.adaBoostRDensityTotal
- AppliedModelingLib.Learning.Online.adaBoostRReducedError
- AppliedModelingLib.Learning.Online.adaBoostRStepWeight
- AppliedModelingLib.Learning.Online.adaBoostRUnitMeasure

AppliedModelingLib.Learning.Online.adaBoostRTheorem12Factor

Declaration location: reusable library

Lean declaration body

```
type:
{Training : Type u_1} →
  [inst : Fintype Training] →
    (state : AppliedModelingLib.Learning.Online.AdaBoostRDensityState Training) →
      (label : Training → ℝ) →
        (hypotheses : List (Training → ℝ)) →
          AppliedModelingLib.Learning.Online.AdaBoostRAdmissible state label hypotheses → ℝ

value:
fun {Training : Type u_1} [Fintype Training] (state : AppliedModelingLib.Learning.Online.AdaBoostRDensityState Training)
    (label : Training → ℝ) (hypotheses : List (Training → ℝ))
    (hvalid : AppliedModelingLib.Learning.Online.AdaBoostRAdmissible state label hypotheses) =>
  (List.map (fun (error : ℝ) => 2 * √(error * (1 - error)))
      (AppliedModelingLib.Learning.Online.adaBoostRErrors state label hypotheses hvalid)).prod
```

Direct retained dependencies

- AppliedModelingLib.Learning.Online.AdaBoostRAdmissible
- AppliedModelingLib.Learning.Online.AdaBoostRDensityState
- AppliedModelingLib.Learning.Online.adaBoostRErrors

AppliedModelingLib.Learning.Online.adaBoostVoteWeight

Declaration location: reusable library

Lean declaration body

```
type:
ℝ → ℝ

value:
fun (error : ℝ) => Real.log (1 / AppliedModelingLib.Learning.Online.adaBoostDiscount error)
```

Direct retained dependencies

- AppliedModelingLib.Learning.Online.adaBoostDiscount

AppliedModelingLib.Learning.Online.adaBoostRGoodMedianCandidates

Declaration location: reusable library

Lean declaration body

```
type:
{Training : Type u_1} →
  [inst : Fintype Training] →
    (state : AppliedModelingLib.Learning.Online.AdaBoostRDensityState Training) →
      (label : Training → ℝ) →
        (hypotheses : List (Training → ℝ)) →
          AppliedModelingLib.Learning.Online.AdaBoostRAdmissible state label hypotheses → Training → Finset ℝ

value:
fun {Training : Type u_1} [Fintype Training] (state : AppliedModelingLib.Learning.Online.AdaBoostRDensityState Training)
    (label : Training → ℝ) (hypotheses : List (Training → ℝ))
    (hvalid : AppliedModelingLib.Learning.Online.AdaBoostRAdmissible state label hypotheses) (sample : Training) =>
  {y ∈ AppliedModelingLib.Learning.Online.adaBoostRMedianCandidates hypotheses sample |
    AppliedModelingLib.Learning.Online.adaBoostRVoteWeightSum state label hypotheses hvalid / 2 ≤
      AppliedModelingLib.Learning.Online.adaBoostRVote state label hypotheses hvalid sample y}
```

Direct retained dependencies

- AppliedModelingLib.Learning.Online.AdaBoostRAdmissible
- AppliedModelingLib.Learning.Online.AdaBoostRDensityState
- AppliedModelingLib.Learning.Online.adaBoostRMedianCandidates
- AppliedModelingLib.Learning.Online.adaBoostRVote
- AppliedModelingLib.Learning.Online.adaBoostRVoteWeightSum

AppliedModelingLib.Learning.Online.adaBoostRFinalHypothesis

Declaration location: reusable library

Lean declaration body

```
type:
{Training : Type u_1} →
  [inst : Fintype Training] →
    (state : AppliedModelingLib.Learning.Online.AdaBoostRDensityState Training) →
      (label : Training → ℝ) →
        (hypotheses : List (Training → ℝ)) →
          (∀ hypothesis ∈ hypotheses, ∀ (sample : Training), 0 ≤ hypothesis sample ∧ hypothesis sample ≤ 1) →
            AppliedModelingLib.Learning.Online.AdaBoostRAdmissible state label hypotheses → Training → ℝ

value:
fun {Training : Type u_1} [Fintype Training] (state : AppliedModelingLib.Learning.Online.AdaBoostRDensityState Training)
    (label : Training → ℝ) (hypotheses : List (Training → ℝ))
    (hhypotheses : ∀ hypothesis ∈ hypotheses, ∀ (sample : Training), 0 ≤ hypothesis sample ∧ hypothesis sample ≤ 1)
    (hvalid : AppliedModelingLib.Learning.Online.AdaBoostRAdmissible state label hypotheses) (a : Training) =>
  (AppliedModelingLib.Learning.Online.adaBoostRGoodMedianCandidates state label hypotheses hvalid a).min'
    (AppliedModelingLib.Learning.Online.adaBoostRGoodMedianCandidates_nonempty state label hypotheses hhypotheses hvalid
      a)
```

Direct retained dependencies

- AppliedModelingLib.Learning.Online.AdaBoostRAdmissible
- AppliedModelingLib.Learning.Online.AdaBoostRDensityState
- AppliedModelingLib.Learning.Online.adaBoostRGoodMedianCandidates

AppliedModelingLib.Learning.Online.allocationExpertCumulativeLoss

Declaration location: reusable library

Lean declaration body

```
type:
{Expert : Type u_1} → List (Expert → ℝ) → Expert → ℝ

value:
fun {Expert : Type u_1} (losses : List (Expert → ℝ)) (expert : Expert) =>
  (List.map (fun (loss : Expert → ℝ) => loss expert) losses).sum
```

AppliedModelingLib.Learning.Online.allocationMixtureLoss

Declaration location: reusable library

Lean declaration body

```
type:
{Expert : Type u_1} → [Fintype Expert] → (Expert → ℝ) → (Expert → ℝ) → ℝ

value:
fun {Expert : Type u_1} [Fintype Expert] (allocation loss : Expert → ℝ) =>
  ∑ expert : Expert, allocation expert * loss expert
```

AppliedModelingLib.Learning.Online.allocationPolicyCumulativeLoss

Declaration location: reusable library

Lean declaration body

```
type:
{Expert : Type u_1} →
  [inst : Fintype Expert] → AppliedModelingLib.Learning.Online.FiniteAllocationPolicy Expert → List (Expert → ℝ) → ℝ

value:
fun {Expert : Type u_1} [Fintype Expert] (policy : AppliedModelingLib.Learning.Online.FiniteAllocationPolicy Expert)
    (losses : List (Expert → ℝ)) =>
  AppliedModelingLib.Learning.Online.allocationPolicyCumulativeLossFrom policy [] losses
```

Direct retained dependencies

- AppliedModelingLib.Learning.Online.FiniteAllocationPolicy
- AppliedModelingLib.Learning.Online.allocationPolicyCumulativeLossFrom

AppliedModelingLib.Learning.Online.hedgeDiscountFromBounds

Declaration location: reusable library

Lean declaration body

```
type:
ℝ → ℝ → ℝ

value:
fun (lossBound regretBound : ℝ) => 1 / (1 + √(2 * regretBound / lossBound))
```

AppliedModelingLib.Learning.Online.hedgeDiscountFromBoundsClosed

Declaration location: reusable library

Lean declaration body

```
type:
ℝ → ℝ → ℝ

value:
fun (lossBound regretBound : ℝ) =>
  if lossBound = 0 then 0 else AppliedModelingLib.Learning.Online.hedgeDiscountFromBounds lossBound regretBound
```

Direct retained dependencies

- AppliedModelingLib.Learning.Online.hedgeDiscountFromBounds

AppliedModelingLib.Learning.Online.hedgeWeightStateStep

Declaration location: reusable library

Lean declaration body

```
type:
{Action : Type u_1} →
  [inst : Fintype Action] →
    [Nonempty Action] →
      AppliedModelingLib.Learning.Online.HedgeWeightState Action →
        (Action → ℝ) → (discount : ℝ) → 0 < discount → AppliedModelingLib.Learning.Online.HedgeWeightState Action

value:
fun {Action : Type u_1} [Fintype Action] [Nonempty Action]
    (state : AppliedModelingLib.Learning.Online.HedgeWeightState Action) (loss : Action → ℝ) (discount : ℝ)
    (hdiscount : 0 < discount) =>
  { weight := fun (action : Action) => state.weight action * discount ^ loss action,
    nonneg := AppliedModelingLib.Learning.Online.hedgeWeightStateStep._proof_1 state loss discount hdiscount,
    total_pos := AppliedModelingLib.Learning.Online.hedgeWeightStateStep._proof_2 state loss discount hdiscount }
```

Direct retained dependencies

- AppliedModelingLib.Learning.Online.HedgeWeightState

AppliedModelingLib.Learning.Online.hedgeWeightStateSubsetComparatorExtendedBound

Declaration location: reusable library

Lean declaration body

```
type:
ℝ → ℝ → ℝ → EReal

value:
fun (discount initialSelectedMass maxSelectedLoss : ℝ) =>
  if initialSelectedMass = 0 then ⊤
  else ↑((-Real.log initialSelectedMass - maxSelectedLoss * Real.log discount) / (1 - discount))
```

AppliedModelingLib.Probability.binaryKLDivergence

Declaration location: reusable library

Lean declaration body

```
type:
ℝ → ℝ → ℝ

value:
fun (p q : ℝ) => p * Real.log (p / q) + (1 - p) * Real.log ((1 - p) / (1 - q))
```

AppliedModelingLib.Statistics.BinaryClassifier

Declaration location: reusable library

Lean declaration body

```
type:
Type u_1 → Type u_1

value:
fun (X : Type u_1) => X → Bool
```

AppliedModelingLib.Statistics.affineThresholdScore

Declaration location: reusable library

Lean declaration body

```
type:
{dimension : ℕ} → (Fin dimension → ℝ) → ℝ → (Fin dimension → ℝ) → ℝ

value:
fun {dimension : ℕ} (weights : Fin dimension → ℝ) (offset : ℝ) (point : Fin dimension → ℝ) =>
  ∑ coordinate : Fin dimension, weights coordinate * point coordinate - offset
```

AppliedModelingLib.Statistics.binaryFeatureVector

Declaration location: reusable library

Lean declaration body

type:
{X : Type u_1} → (width : ℕ) → (Fin width → AppliedModelingLib.Statistics.BinaryClassifier X) → X → Fin width → ℝ

value:
fun {X : Type u_1} (width : ℕ) (hypotheses : Fin width → AppliedModelingLib.Statistics.BinaryClassifier X) (feature : X)
    (a : Fin width) =>
  if hypotheses a feature = true then 1 else 0

Direct retained dependencies

- AppliedModelingLib.Statistics.BinaryClassifier

AppliedModelingLib.Statistics.binaryThresholdScore

Declaration location: reusable library

Lean declaration body

type:
{X : Type u_1} →
  (width : ℕ) → (Fin width → ℝ) → ℝ → (Fin width → AppliedModelingLib.Statistics.BinaryClassifier X) → X → ℝ

value:
fun {X : Type u_1} (width : ℕ) (weights : Fin width → ℝ) (offset : ℝ)
    (hypotheses : Fin width → AppliedModelingLib.Statistics.BinaryClassifier X) (feature : X) =>
  AppliedModelingLib.Statistics.affineThresholdScore weights offset
    (AppliedModelingLib.Statistics.binaryFeatureVector width hypotheses feature)

Direct retained dependencies

- AppliedModelingLib.Statistics.BinaryClassifier
- AppliedModelingLib.Statistics.affineThresholdScore
- AppliedModelingLib.Statistics.binaryFeatureVector

AppliedModelingLib.Statistics.binaryThresholdCombination

Declaration location: reusable library

Lean declaration body

type:
{X : Type u_1} →
  (width : ℕ) → (Fin width → ℝ) → ℝ → (Fin width → AppliedModelingLib.Statistics.BinaryClassifier X) → X → Bool

value:
fun {X : Type u_1} (width : ℕ) (weights : Fin width → ℝ) (offset : ℝ)
    (hypotheses : Fin width → AppliedModelingLib.Statistics.BinaryClassifier X) (a : X) =>
  decide (0 ≤ AppliedModelingLib.Statistics.binaryThresholdScore width weights offset hypotheses a)

Direct retained dependencies

- AppliedModelingLib.Statistics.BinaryClassifier
- AppliedModelingLib.Statistics.binaryThresholdScore

AppliedModelingLib.Statistics.binaryThresholdCombinationClass

Declaration location: reusable library

Lean declaration body

type:
{X : Type u_1} →
  Set (AppliedModelingLib.Statistics.BinaryClassifier X) → ℕ → AppliedModelingLib.Statistics.BinaryClassifier X → Prop

value:
fun {X : Type u_1} (concepts : Set (AppliedModelingLib.Statistics.BinaryClassifier X)) (width : ℕ)
    (a : AppliedModelingLib.Statistics.BinaryClassifier X) =>
  {combined : AppliedModelingLib.Statistics.BinaryClassifier X |
    ∃ (hypotheses : Fin width → AppliedModelingLib.Statistics.BinaryClassifier X),
      (∀ (index : Fin width), hypotheses index ∈ concepts) ∧
        ∃ (weights : Fin width → ℝ) (offset : ℝ),
          combined = AppliedModelingLib.Statistics.binaryThresholdCombination width weights offset hypotheses}
  a

Direct retained dependencies

- AppliedModelingLib.Statistics.BinaryClassifier
- AppliedModelingLib.Statistics.binaryThresholdCombination

AppliedModelingLib.Statistics.binaryTrace

Declaration location: reusable library

Lean declaration body

type:
```
{X : Type u_1} → Set (AppliedModelingLib.Statistics.BinaryClassifier X) → (sample : Finset X) → Finset (Finset ↥sample)
```

value:
```
fun {X : Type u_1} (concepts : Set (AppliedModelingLib.Statistics.BinaryClassifier X)) (sample : Finset X) =>
  {labels : Finset ↥sample | ∃ classifier ∈ concepts, ∀ (point : ↥sample), point ∈ labels ↔ classifier ↑point = true}
```

Direct retained dependencies

- AppliedModelingLib.Statistics.BinaryClassifier

AppliedModelingLib.Statistics.binaryTraceVCDimension

Declaration location: reusable library

Lean declaration body

type:
```
{X : Type u_1} → Set (AppliedModelingLib.Statistics.BinaryClassifier X) → Finset X → ℕ
```

value:
```
fun {X : Type u_1} (concepts : Set (AppliedModelingLib.Statistics.BinaryClassifier X)) (sample : Finset X) =>
  (AppliedModelingLib.Statistics.binaryTrace concepts sample).vcDim
```

Direct retained dependencies

- AppliedModelingLib.Statistics.BinaryClassifier
- AppliedModelingLib.Statistics.binaryTrace

AppliedModelingLib.Statistics.VCDimensionAtMost

Declaration location: reusable library

Lean declaration body

type:
```
{X : Type u_1} → Set (AppliedModelingLib.Statistics.BinaryClassifier X) → ℕ → Prop
```

value:
```
fun {X : Type u_1} (concepts : Set (AppliedModelingLib.Statistics.BinaryClassifier X)) (d : ℕ) =>
  ∀ (sample : Finset X), AppliedModelingLib.Statistics.binaryTraceVCDimension concepts sample ≤ d
```

Direct retained dependencies

- AppliedModelingLib.Statistics.BinaryClassifier
- AppliedModelingLib.Statistics.binaryTraceVCDimension

AppliedModelingLib.Statistics.sourceThresholdCombinationVCBound

Declaration location: reusable library

Lean declaration body

type:
```
ℕ → ℕ → ℝ
```

value:
```
fun (width d : ℕ) => 2 * ↑(d + 1) * ↑(width + 1) * Real.logb 2 (Real.exp 1 * ↑(width + 1))
```

AppliedModelingLib.finiteMin

Declaration location: reusable library

Lean declaration body

type:
{α : Type u_1} → [Fintype α] → [Nonempty α] → (α → ℝ) → ℝ

value:
fun {α : Type u_1} [Fintype α] [Nonempty α] (f : α → ℝ) => Finset.univ.inf' Finset.univ_nonempty f

AppliedModelingLib.Learning.Online.FiniteAllocationPolicyBound

Declaration location: reusable library

Lean declaration body

type:
{Expert : Type u_1} →
  [inst : Fintype Expert] →
    [Nonempty Expert] → AppliedModelingLib.Learning.Online.FiniteAllocationPolicy Expert → ℝ → ℝ → Prop

value:
fun {Expert : Type u_1} [Fintype Expert] [Nonempty Expert]
    (policy : AppliedModelingLib.Learning.Online.FiniteAllocationPolicy Expert) (c a : ℝ) =>
  ∀ (losses : List (Expert → ℝ)),
    (∀ loss ∈ losses, ∀ (expert : Expert), 0 ≤ loss expert ∧ loss expert ≤ 1) →
      AppliedModelingLib.Learning.Online.allocationPolicyCumulativeLoss policy losses ≤
        c * AppliedModelingLib.finiteMin (AppliedModelingLib.Learning.Online.allocationExpertCumulativeLoss losses) +
          a * Real.log ↑(Fintype.card Expert)

Direct retained dependencies

- AppliedModelingLib.Learning.Online.FiniteAllocationPolicy
- AppliedModelingLib.Learning.Online.allocationExpertCumulativeLoss
- AppliedModelingLib.Learning.Online.allocationPolicyCumulativeLoss
- AppliedModelingLib.finiteMin

AppliedModelingLib.Learning.Online.globalFiniteAllocationPolicyBounded

Declaration location: reusable library

Lean declaration body

type:
ℝ → ℝ → Prop

value:
fun (c a : ℝ) =>
  ∀ (Expert : Type) [inst : Fintype Expert] [inst_1 : Nonempty Expert],
    ∃ (policy : AppliedModelingLib.Learning.Online.FiniteAllocationPolicy Expert),
      AppliedModelingLib.Learning.Online.FiniteAllocationPolicyAdmissible policy ∧
        AppliedModelingLib.Learning.Online.FiniteAllocationPolicyBound policy c a

Direct retained dependencies

- AppliedModelingLib.Learning.Online.FiniteAllocationPolicy
- AppliedModelingLib.Learning.Online.FiniteAllocationPolicyAdmissible
- AppliedModelingLib.Learning.Online.FiniteAllocationPolicyBound

AppliedModelingLib.finiteWeightedPMF

Declaration location: reusable library

Lean declaration body

type:
{α : Type u_1} → [inst : Fintype α] → (weight : α → ℝ) → (∀ (a : α), 0 ≤ weight a) → 0 < ∑ a : α, weight a → PMF α

value:
fun {α : Type u_1} [Fintype α] (weight : α → ℝ) (hweight_nonneg : ∀ (a : α), 0 ≤ weight a)
    (htotal_pos : 0 < ∑ a : α, weight a) =>
  PMF.ofFintype (fun (a : α) => ENNReal.ofReal (weight a / ∑ b : α, weight b))
    (AppliedModelingLib.finiteWeightedPMF._proof_1 weight hweight_nonneg htotal_pos)

AppliedModelingLib.Learning.Online.HedgeWeightState.policy

Declaration location: reusable library

Lean declaration body

type:
{Action : Type u_1} → [inst : Fintype Action] → AppliedModelingLib.Learning.Online.HedgeWeightState Action → PMF Action

value:
fun {Action : Type u_1} [Fintype Action] (state : AppliedModelingLib.Learning.Online.HedgeWeightState Action) =>
  AppliedModelingLib.finiteWeightedPMF state.weight state.nonneg state.total_pos

Direct retained dependencies

- AppliedModelingLib.Learning.Online.HedgeWeightState
- AppliedModelingLib.finiteWeightedPMF

AppliedModelingLib.pmfExp

Declaration location: reusable library

Lean declaration body

type:
{α : Type u_1} → [Fintype α] → PMF α → (α → ℝ) → ℝ

value:
fun {α : Type u_1} [Fintype α] (μ : PMF α) (f : α → ℝ) => ∑ a : α, (μ a).toReal * f a

AppliedModelingLib.Learning.Online.GeneralDecisionHedgeRound.inducedLoss

Declaration location: reusable library

Lean declaration body

type:
{Expert : Type u_1} →
  {Decision : Type u_2} →
    {Outcome : Type u_3} →
      [inst : Fintype Expert] →
        [inst_1 : DecidableEq Expert] →
          [inst_2 : MeasurableSpace Decision] →
            {game : AppliedModelingLib.Learning.Online.GeneralDecisionHedgeGame Expert Decision Outcome} →
              AppliedModelingLib.Learning.Online.GeneralDecisionHedgeRound game → Expert → ℝ

value:
fun {Expert : Type u_1} {Decision : Type u_2} {Outcome : Type u_3} [Fintype Expert] [DecidableEq Expert]
    [MeasurableSpace Decision]
    {game : AppliedModelingLib.Learning.Online.GeneralDecisionHedgeGame Expert Decision Outcome}
    (round : AppliedModelingLib.Learning.Online.GeneralDecisionHedgeRound game) (a : Expert) =>
  game.expectedLoss (round.expertDecision a) round.outcome

Direct retained dependencies

- AppliedModelingLib.Learning.Online.GeneralDecisionHedgeGame
- AppliedModelingLib.Learning.Online.GeneralDecisionHedgeRound

AppliedModelingLib.Learning.Online.GeneralDecisionHedgeRound.mixedDecision

Declaration location: reusable library

Lean declaration body

type:
{Expert : Type u_1} →
  {Decision : Type u_2} →
    {Outcome : Type u_3} →
      [inst : Fintype Expert] →
        [inst_1 : DecidableEq Expert] →
          [inst_2 : MeasurableSpace Decision] →
            {game : AppliedModelingLib.Learning.Online.GeneralDecisionHedgeGame Expert Decision Outcome} →
              AppliedModelingLib.Learning.Online.GeneralDecisionHedgeRound game → PMF Expert → game.Distribution

value:
fun {Expert : Type u_1} {Decision : Type u_2} {Outcome : Type u_3} [Fintype Expert] [DecidableEq Expert]
    [MeasurableSpace Decision]
    {game : AppliedModelingLib.Learning.Online.GeneralDecisionHedgeGame Expert Decision Outcome}
    (round : AppliedModelingLib.Learning.Online.GeneralDecisionHedgeRound game) (allocation : PMF Expert) =>
  game.mixture allocation round.expertDecision

Direct retained dependencies

- AppliedModelingLib.Learning.Online.GeneralDecisionHedgeGame
- AppliedModelingLib.Learning.Online.GeneralDecisionHedgeRound

AppliedModelingLib.Learning.Online.HedgeWeightState.expectation

Declaration location: reusable library

Lean declaration body

```
type:
{Action : Type u_1} →
  [inst : Fintype Action] → AppliedModelingLib.Learning.Online.HedgeWeightState Action → (Action → ℝ) → ℝ

value:
fun {Action : Type u_1} [Fintype Action] (state : AppliedModelingLib.Learning.Online.HedgeWeightState Action)
    (quantity : Action → ℝ) =>
  AppliedModelingLib.pmfExp state.policy quantity
```

Direct retained dependencies

- AppliedModelingLib.Learning.Online.HedgeWeightState
- AppliedModelingLib.Learning.Online.HedgeWeightState.policy
- AppliedModelingLib.pmfExp

AppliedModelingLib.Learning.Online.adaBoostError

Declaration location: reusable library

Lean declaration body

```
type:
{Training : Type u_1} →
  [inst : Fintype Training] →
    AppliedModelingLib.Learning.Online.HedgeWeightState Training → (Training → ℝ) → (Training → ℝ) → ℝ

value:
fun {Training : Type u_1} [Fintype Training] (state : AppliedModelingLib.Learning.Online.HedgeWeightState Training)
    (label hypothesis : Training → ℝ) =>
  state.expectation fun (sample : Training) => |hypothesis sample - label sample|
```

Direct retained dependencies

- AppliedModelingLib.Learning.Online.HedgeWeightState
- AppliedModelingLib.Learning.Online.HedgeWeightState.expectation

AppliedModelingLib.Learning.Online.adaBoostStep

Declaration location: reusable library

Lean declaration body

```
type:
{Training : Type u_1} →
  [inst : Fintype Training] →
    [Nonempty Training] →
      (state : AppliedModelingLib.Learning.Online.HedgeWeightState Training) →
        (label hypothesis : Training → ℝ) →
          0 < AppliedModelingLib.Learning.Online.adaBoostError state label hypothesis ∧
              AppliedModelingLib.Learning.Online.adaBoostError state label hypothesis < 1 →
            AppliedModelingLib.Learning.Online.HedgeWeightState Training

value:
fun {Training : Type u_1} [Fintype Training] [Nonempty Training]
    (state : AppliedModelingLib.Learning.Online.HedgeWeightState Training) (label hypothesis : Training → ℝ)
    (herror :
      0 < AppliedModelingLib.Learning.Online.adaBoostError state label hypothesis ∧
        AppliedModelingLib.Learning.Online.adaBoostError state label hypothesis < 1) =>
  AppliedModelingLib.Learning.Online.hedgeWeightStateStep state
    (AppliedModelingLib.Learning.Online.adaBoostLoss label hypothesis)
    (AppliedModelingLib.Learning.Online.adaBoostDiscount
      (AppliedModelingLib.Learning.Online.adaBoostError state label hypothesis))
    (AppliedModelingLib.Learning.Online.adaBoostStep._proof_1 state label hypothesis herror)
```

Direct retained dependencies

- AppliedModelingLib.Learning.Online.HedgeWeightState
- AppliedModelingLib.Learning.Online.adaBoostDiscount
- AppliedModelingLib.Learning.Online.adaBoostError
- AppliedModelingLib.Learning.Online.adaBoostLoss
- AppliedModelingLib.Learning.Online.hedgeWeightStateStep

AppliedModelingLib.Learning.Online.AdaBoostM1Admissible

Declaration location: reusable library

Lean declaration body

```
type:
{Training : Type u_1} →
  {Label : Type u_2} →
    [inst : Fintype Training] →
      [Nonempty Training] →
        [DecidableEq Label] →
          AppliedModelingLib.Learning.Online.HedgeWeightState Training →
            (Training → Label) → List (Training → Label) → Prop
```

```
value:
fun {Training : Type u_1} {Label : Type u_2} [Fintype Training] [Nonempty Training] [DecidableEq Label]
    (state : AppliedModelingLib.Learning.Online.HedgeWeightState Training) (label : Training → Label)
    (hypotheses : List (Training → Label)) =>
  AppliedModelingLib.Learning.Online.AdaBoostAdmissible state (fun (x : Training) => 0)
    (AppliedModelingLib.Learning.Online.adaBoostM1BinaryHypotheses label hypotheses)
```

Direct retained dependencies

- AppliedModelingLib.Learning.Online.AdaBoostAdmissible
- AppliedModelingLib.Learning.Online.HedgeWeightState
- AppliedModelingLib.Learning.Online.adaBoostM1BinaryHypotheses

AppliedModelingLib.Learning.Online.AdaBoostM2Admissible

Declaration location: reusable library

Lean declaration body

```
type:
{Training : Type u_1} →
  {Label : Type u_2} →
    [inst : Fintype Training] →
      [inst_1 : Fintype Label] →
        [inst_2 : DecidableEq Label] →
          (label : Training → Label) →
            [Nonempty (AppliedModelingLib.Learning.Online.AdaBoostM2Instance Training Label label)] →
              AppliedModelingLib.Learning.Online.HedgeWeightState
                (AppliedModelingLib.Learning.Online.AdaBoostM2Instance Training Label label) →
                List (Training → Label → ℝ) → Prop
```

```
value:
fun {Training : Type u_1} {Label : Type u_2} [Fintype Training] [Fintype Label] [DecidableEq Label]
    (label : Training → Label) [Nonempty (AppliedModelingLib.Learning.Online.AdaBoostM2Instance Training Label label)]
    (state :
      AppliedModelingLib.Learning.Online.HedgeWeightState
        (AppliedModelingLib.Learning.Online.AdaBoostM2Instance Training Label label))
    (hypotheses : List (Training → Label → ℝ)) =>
  AppliedModelingLib.Learning.Online.AdaBoostAdmissible state
    (fun (x : AppliedModelingLib.Learning.Online.AdaBoostM2Instance Training Label label) => 0)
    (AppliedModelingLib.Learning.Online.adaBoostM2BinaryHypotheses label hypotheses)
```

Direct retained dependencies

- AppliedModelingLib.Learning.Online.AdaBoostAdmissible
- AppliedModelingLib.Learning.Online.AdaBoostM2Instance
- AppliedModelingLib.Learning.Online.HedgeWeightState
- AppliedModelingLib.Learning.Online.adaBoostM2BinaryHypotheses

AppliedModelingLib.Learning.Online.AdaBoostSoftThreshold

Declaration location: reusable library

Lean declaration body

```
type:
{Training : Type u_1} →
  [inst : Fintype Training] →
    [inst_1 : Nonempty Training] →
      (state : AppliedModelingLib.Learning.Online.HedgeWeightState Training) →
        (label : Training → ℝ) →
          (hypotheses : List (Training → ℝ)) →
            AppliedModelingLib.Learning.Online.AdaBoostAdmissible state label hypotheses → (ℝ → ℝ) → Prop

value:
fun {Training : Type u_1} [Fintype Training] [Nonempty Training]
    (state : AppliedModelingLib.Learning.Online.HedgeWeightState Training) (label : Training → ℝ)
    (hypotheses : List (Training → ℝ))
    (hvalid : AppliedModelingLib.Learning.Online.AdaBoostAdmissible state label hypotheses) (threshold : ℝ → ℝ) =>
  (∀ (r : ℝ), 0 ≤ r → r ≤ 1 → 0 ≤ threshold r ∧ threshold r ≤ 1) ∧
    (∀ (r : ℝ), 0 ≤ r → r ≤ 1 → threshold (1 - r) = 1 - threshold r) ∧
      ∀ (r : ℝ),
        0 ≤ r →
          r ≤ 1 →
            threshold r ≤
              1 / 2 *
                AppliedModelingLib.Learning.Online.adaBoostDiscountPowerFactor state label hypotheses hvalid (1 / 2 - r)
```

Direct retained dependencies

- AppliedModelingLib.Learning.Online.AdaBoostAdmissible
- AppliedModelingLib.Learning.Online.HedgeWeightState
- AppliedModelingLib.Learning.Online.adaBoostDiscountPowerFactor

AppliedModelingLib.Learning.Online.adaBoostM1Errors

Declaration location: reusable library

Lean declaration body

```
type:
{Training : Type u_1} →
  {Label : Type u_2} →
    [inst : Fintype Training] →
      [inst_1 : Nonempty Training] →
        [inst_2 : DecidableEq Label] →
          (state : AppliedModelingLib.Learning.Online.HedgeWeightState Training) →
            (label : Training → Label) →
              (hypotheses : List (Training → Label)) →
                AppliedModelingLib.Learning.Online.AdaBoostM1Admissible state label hypotheses → List ℝ

value:
fun {Training : Type u_1} {Label : Type u_2} [Fintype Training] [Nonempty Training] [DecidableEq Label]
    (state : AppliedModelingLib.Learning.Online.HedgeWeightState Training) (label : Training → Label)
    (hypotheses : List (Training → Label))
    (hvalid : AppliedModelingLib.Learning.Online.AdaBoostM1Admissible state label hypotheses) =>
  AppliedModelingLib.Learning.Online.adaBoostErrors state (fun (x : Training) => 0)
    (AppliedModelingLib.Learning.Online.adaBoostM1BinaryHypotheses label hypotheses) hvalid
```

Direct retained dependencies

- AppliedModelingLib.Learning.Online.AdaBoostM1Admissible
- AppliedModelingLib.Learning.Online.HedgeWeightState
- AppliedModelingLib.Learning.Online.adaBoostErrors
- AppliedModelingLib.Learning.Online.adaBoostM1BinaryHypotheses

AppliedModelingLib.Learning.Online.adaBoostM1FinalHypothesis

Declaration location: reusable library

Lean declaration body

```
type:
{Training : Type u_1} →
  {Label : Type u_2} →
    [inst : Fintype Training] →
      [inst_1 : Nonempty Training] →
        [Fintype Label] →
          [Nonempty Label] →
            [inst_4 : DecidableEq Label] →
              (state : AppliedModelingLib.Learning.Online.HedgeWeightState Training) →
                (label : Training → Label) →
                  (hypotheses : List (Training → Label)) →
                    AppliedModelingLib.Learning.Online.AdaBoostM1Admissible state label hypotheses → Training → Label

value:
fun {Training : Type u_1} {Label : Type u_2} [Fintype Training] [Nonempty Training] [Fintype Label] [Nonempty Label]
    [DecidableEq Label] (state : AppliedModelingLib.Learning.Online.HedgeWeightState Training)
    (label : Training → Label) (hypotheses : List (Training → Label))
    (hvalid : AppliedModelingLib.Learning.Online.AdaBoostM1Admissible state label hypotheses) (a : Training) =>
  Classical.choose
    (AppliedModelingLib.Learning.Online.adaBoostM1FinalHypothesis._proof_1 state label hypotheses hvalid a)
```

Direct retained dependencies

- AppliedModelingLib.Learning.Online.AdaBoostM1Admissible
- AppliedModelingLib.Learning.Online.HedgeWeightState
- AppliedModelingLib.Learning.Online.adaBoostM1LabelScore

AppliedModelingLib.Learning.Online.adaBoostM1TrainingError

Declaration location: reusable library

Lean declaration body

```
type:
{Training : Type u_1} →
  {Label : Type u_2} →
    [inst : Fintype Training] →
      [inst_1 : Nonempty Training] →
        [Fintype Label] →
          [Nonempty Label] →
            [inst_4 : DecidableEq Label] →
              (state : AppliedModelingLib.Learning.Online.HedgeWeightState Training) →
                (label : Training → Label) →
                  (hypotheses : List (Training → Label)) →
                    AppliedModelingLib.Learning.Online.AdaBoostM1Admissible state label hypotheses → ℝ

value:
fun {Training : Type u_1} {Label : Type u_2} [Fintype Training] [Nonempty Training] [Fintype Label] [Nonempty Label]
    [DecidableEq Label] (state : AppliedModelingLib.Learning.Online.HedgeWeightState Training)
    (label : Training → Label) (hypotheses : List (Training → Label))
    (hvalid : AppliedModelingLib.Learning.Online.AdaBoostM1Admissible state label hypotheses) =>
  ∑ sample : Training,
    state.weight sample *
      if
          AppliedModelingLib.Learning.Online.adaBoostM1FinalHypothesis state label hypotheses hvalid sample =
            label sample then
        0
      else 1
```

Direct retained dependencies

- AppliedModelingLib.Learning.Online.AdaBoostM1Admissible
- AppliedModelingLib.Learning.Online.HedgeWeightState
- AppliedModelingLib.Learning.Online.adaBoostM1FinalHypothesis

AppliedModelingLib.Learning.Online.adaBoostM2FinalHypothesis

Declaration location: reusable library

Lean declaration body

```
type:
{Training : Type u_1} →
  {Label : Type u_2} →
    [inst : Fintype Training] →
      [inst_1 : Fintype Label] →
        [Nonempty Label] →
          [inst_3 : DecidableEq Label] →
```

```
      (label : Training → Label) →
        [inst_4 : Nonempty (AppliedModelingLib.Learning.Online.AdaBoostM2Instance Training Label label)] →
          (state :
              AppliedModelingLib.Learning.Online.HedgeWeightState
                (AppliedModelingLib.Learning.Online.AdaBoostM2Instance Training Label label)) →
            (hypotheses : List (Training → Label → ℝ)) →
              AppliedModelingLib.Learning.Online.AdaBoostM2Admissible label state hypotheses → Training → Label

value:
fun {Training : Type u_1} {Label : Type u_2} [Fintype Training] [Fintype Label] [Nonempty Label] [DecidableEq Label]
    (label : Training → Label) [Nonempty (AppliedModelingLib.Learning.Online.AdaBoostM2Instance Training Label label)]
    (state :
      AppliedModelingLib.Learning.Online.HedgeWeightState
        (AppliedModelingLib.Learning.Online.AdaBoostM2Instance Training Label label))
    (hypotheses : List (Training → Label → ℝ))
    (hvalid : AppliedModelingLib.Learning.Online.AdaBoostM2Admissible label state hypotheses) (a : Training) =>
  Classical.choose
    (AppliedModelingLib.Learning.Online.adaBoostM2FinalHypothesis._proof_1 label state hypotheses hvalid a)
```

Direct retained dependencies

- AppliedModelingLib.Learning.Online.AdaBoostM2Admissible
- AppliedModelingLib.Learning.Online.AdaBoostM2Instance
- AppliedModelingLib.Learning.Online.HedgeWeightState
- AppliedModelingLib.Learning.Online.adaBoostM2LabelScore

AppliedModelingLib.Learning.Online.adaBoostM2TrainingError

Declaration location: reusable library

Lean declaration body

```
type:
{Training : Type u_1} →
  {Label : Type u_2} →
    [inst : Fintype Training] →
      [inst_1 : Fintype Label] →
        [Nonempty Label] →
          [inst_3 : DecidableEq Label] →
            (label : Training → Label) →
              [inst_4 : Nonempty (AppliedModelingLib.Learning.Online.AdaBoostM2Instance Training Label label)] →
                (Training → ℝ) →
                  (state :
                      AppliedModelingLib.Learning.Online.HedgeWeightState
                        (AppliedModelingLib.Learning.Online.AdaBoostM2Instance Training Label label)) →
                    (hypotheses : List (Training → Label → ℝ)) →
                      AppliedModelingLib.Learning.Online.AdaBoostM2Admissible label state hypotheses → ℝ

value:
fun {Training : Type u_1} {Label : Type u_2} [Fintype Training] [Fintype Label] [Nonempty Label] [DecidableEq Label]
    (label : Training → Label) [Nonempty (AppliedModelingLib.Learning.Online.AdaBoostM2Instance Training Label label)]
    (exampleWeight : Training → ℝ)
    (state :
      AppliedModelingLib.Learning.Online.HedgeWeightState
        (AppliedModelingLib.Learning.Online.AdaBoostM2Instance Training Label label))
    (hypotheses : List (Training → Label → ℝ))
    (hvalid : AppliedModelingLib.Learning.Online.AdaBoostM2Admissible label state hypotheses) =>
  ∑ sample : Training,
    exampleWeight sample *
      if
          AppliedModelingLib.Learning.Online.adaBoostM2FinalHypothesis label state hypotheses hvalid sample =
            label sample then
        0
      else 1
```

Direct retained dependencies

- AppliedModelingLib.Learning.Online.AdaBoostM2Admissible
- AppliedModelingLib.Learning.Online.AdaBoostM2Instance
- AppliedModelingLib.Learning.Online.HedgeWeightState
- AppliedModelingLib.Learning.Online.adaBoostM2FinalHypothesis

AppliedModelingLib.Learning.Online.adaBoostM1Theorem10Factor

Declaration location: reusable library

Lean declaration body

```
type:
{Training : Type u_1} →
  {Label : Type u_2} →
    [inst : Fintype Training] →
      [inst_1 : Nonempty Training] →
        [inst_2 : DecidableEq Label] →
          (state : AppliedModelingLib.Learning.Online.HedgeWeightState Training) →
            (label : Training → Label) →
              (hypotheses : List (Training → Label)) →
                AppliedModelingLib.Learning.Online.AdaBoostM1Admissible state label hypotheses → ℝ

value:
fun {Training : Type u_1} {Label : Type u_2} [Fintype Training] [Nonempty Training] [DecidableEq Label]
    (state : AppliedModelingLib.Learning.Online.HedgeWeightState Training) (label : Training → Label)
    (hypotheses : List (Training → Label))
    (hvalid : AppliedModelingLib.Learning.Online.AdaBoostM1Admissible state label hypotheses) =>
  AppliedModelingLib.Learning.Online.adaBoostTheorem6Factor state (fun (x : Training) => 0)
    (AppliedModelingLib.Learning.Online.adaBoostM1BinaryHypotheses label hypotheses) hvalid
```

Direct retained dependencies

- AppliedModelingLib.Learning.Online.AdaBoostM1Admissible
- AppliedModelingLib.Learning.Online.HedgeWeightState
- AppliedModelingLib.Learning.Online.adaBoostM1BinaryHypotheses
- AppliedModelingLib.Learning.Online.adaBoostTheorem6Factor

AppliedModelingLib.Learning.Online.adaBoostM2BinaryTheorem6Factor

Declaration location: reusable library

Lean declaration body

```
type:
{Training : Type u_1} →
  {Label : Type u_2} →
    [inst : Fintype Training] →
      [inst_1 : Fintype Label] →
        [inst_2 : DecidableEq Label] →
          (label : Training → Label) →
            [inst_3 : Nonempty (AppliedModelingLib.Learning.Online.AdaBoostM2Instance Training Label label)] →
              (state :
                  AppliedModelingLib.Learning.Online.HedgeWeightState
                    (AppliedModelingLib.Learning.Online.AdaBoostM2Instance Training Label label)) →
                (hypotheses : List (Training → Label → ℝ)) →
                  AppliedModelingLib.Learning.Online.AdaBoostM2Admissible label state hypotheses → ℝ

value:
fun {Training : Type u_1} {Label : Type u_2} [Fintype Training] [Fintype Label] [DecidableEq Label]
    (label : Training → Label) [Nonempty (AppliedModelingLib.Learning.Online.AdaBoostM2Instance Training Label label)]
    (state :
      AppliedModelingLib.Learning.Online.HedgeWeightState
        (AppliedModelingLib.Learning.Online.AdaBoostM2Instance Training Label label))
    (hypotheses : List (Training → Label → ℝ))
    (hvalid : AppliedModelingLib.Learning.Online.AdaBoostM2Admissible label state hypotheses) =>
  AppliedModelingLib.Learning.Online.adaBoostTheorem6Factor state
    (fun (x : AppliedModelingLib.Learning.Online.AdaBoostM2Instance Training Label label) => 0)
    (AppliedModelingLib.Learning.Online.adaBoostM2BinaryHypotheses label hypotheses) hvalid
```

Direct retained dependencies

- AppliedModelingLib.Learning.Online.AdaBoostM2Admissible
- AppliedModelingLib.Learning.Online.AdaBoostM2Instance
- AppliedModelingLib.Learning.Online.HedgeWeightState
- AppliedModelingLib.Learning.Online.adaBoostM2BinaryHypotheses
- AppliedModelingLib.Learning.Online.adaBoostTheorem6Factor

AppliedModelingLib.Learning.Online.adaBoostM2Theorem11Factor

Declaration location: reusable library

Lean declaration body

```
type:
{Training : Type u_1} →
  {Label : Type u_2} →
    [inst : Fintype Training] →
      [inst_1 : Fintype Label] →
        [inst_2 : DecidableEq Label] →
          (label : Training → Label) →
            [inst_3 : Nonempty (AppliedModelingLib.Learning.Online.AdaBoostM2Instance Training Label label)] →
              (state :
                  AppliedModelingLib.Learning.Online.HedgeWeightState
                    (AppliedModelingLib.Learning.Online.AdaBoostM2Instance Training Label label)) →
                (hypotheses : List (Training → Label → ℝ)) →
                  AppliedModelingLib.Learning.Online.AdaBoostM2Admissible label state hypotheses → ℝ

value:
fun {Training : Type u_1} {Label : Type u_2} [Fintype Training] [Fintype Label] [DecidableEq Label]
    (label : Training → Label) [Nonempty (AppliedModelingLib.Learning.Online.AdaBoostM2Instance Training Label label)]
    (state :
      AppliedModelingLib.Learning.Online.HedgeWeightState
        (AppliedModelingLib.Learning.Online.AdaBoostM2Instance Training Label label))
    (hypotheses : List (Training → Label → ℝ))
    (hvalid : AppliedModelingLib.Learning.Online.AdaBoostM2Admissible label state hypotheses) =>
  ↑(Fintype.card Label - 1) *
    AppliedModelingLib.Learning.Online.adaBoostM2BinaryTheorem6Factor label state hypotheses hvalid
```

Direct retained dependencies

- AppliedModelingLib.Learning.Online.AdaBoostM2Admissible
- AppliedModelingLib.Learning.Online.AdaBoostM2Instance
- AppliedModelingLib.Learning.Online.HedgeWeightState
- AppliedModelingLib.Learning.Online.adaBoostM2BinaryTheorem6Factor

AppliedModelingLib.Learning.Online.adaBoostTheorem9Factor

Declaration location: reusable library

Lean declaration body

```
type:
{Training : Type u_1} →
  [inst : Fintype Training] →
    [inst_1 : Nonempty Training] →
      (state : AppliedModelingLib.Learning.Online.HedgeWeightState Training) →
        (label : Training → ℝ) →
          (hypotheses : List (Training → ℝ)) →
            AppliedModelingLib.Learning.Online.AdaBoostAdmissible state label hypotheses → ℝ

value:
fun {Training : Type u_1} [Fintype Training] [Nonempty Training]
    (state : AppliedModelingLib.Learning.Online.HedgeWeightState Training) (label : Training → ℝ)
    (hypotheses : List (Training → ℝ))
    (hvalid : AppliedModelingLib.Learning.Online.AdaBoostAdmissible state label hypotheses) =>
  2 ^ (hypotheses.length - 1) *
    (List.map (fun (error : ℝ) => √(error * (1 - error)))
      (AppliedModelingLib.Learning.Online.adaBoostErrors state label hypotheses hvalid)).prod
```

Direct retained dependencies

- AppliedModelingLib.Learning.Online.AdaBoostAdmissible
- AppliedModelingLib.Learning.Online.HedgeWeightState
- AppliedModelingLib.Learning.Online.adaBoostErrors

AppliedModelingLib.Learning.Online.adaBoostFinalHypothesis

Declaration location: reusable library

Lean declaration body

```
type:
{Training : Type u_1} →
  [inst : Fintype Training] →
    [inst_1 : Nonempty Training] →
      (state : AppliedModelingLib.Learning.Online.HedgeWeightState Training) →
        (label : Training → ℝ) →
          (hypotheses : List (Training → ℝ)) →
            AppliedModelingLib.Learning.Online.AdaBoostAdmissible state label hypotheses → Training → ℝ
```

```
value:
fun {Training : Type u_1} [Fintype Training] [Nonempty Training]
    (state : AppliedModelingLib.Learning.Online.HedgeWeightState Training) (label : Training → ℝ)
    (hypotheses : List (Training → ℝ))
    (hvalid : AppliedModelingLib.Learning.Online.AdaBoostAdmissible state label hypotheses) (a : Training) =>
  if
      AppliedModelingLib.Learning.Online.adaBoostVoteWeightSum state label hypotheses hvalid / 2 ≤
        AppliedModelingLib.Learning.Online.adaBoostVote state label hypotheses hvalid a then
    1
  else 0
```

Direct retained dependencies

- AppliedModelingLib.Learning.Online.AdaBoostAdmissible
- AppliedModelingLib.Learning.Online.HedgeWeightState
- AppliedModelingLib.Learning.Online.adaBoostVote
- AppliedModelingLib.Learning.Online.adaBoostVoteWeightSum

AppliedModelingLib.Learning.Online.adaBoostNormalizedVote

Declaration location: reusable library

Lean declaration body

```
type:
{Training : Type u_1} →
  [inst : Fintype Training] →
    [inst_1 : Nonempty Training] →
      (state : AppliedModelingLib.Learning.Online.HedgeWeightState Training) →
        (label : Training → ℝ) →
          (hypotheses : List (Training → ℝ)) →
            AppliedModelingLib.Learning.Online.AdaBoostAdmissible state label hypotheses → Training → ℝ

value:
fun {Training : Type u_1} [Fintype Training] [Nonempty Training]
    (state : AppliedModelingLib.Learning.Online.HedgeWeightState Training) (label : Training → ℝ)
    (hypotheses : List (Training → ℝ))
    (hvalid : AppliedModelingLib.Learning.Online.AdaBoostAdmissible state label hypotheses) (a : Training) =>
  AppliedModelingLib.Learning.Online.adaBoostVote state label hypotheses hvalid a /
    AppliedModelingLib.Learning.Online.adaBoostVoteWeightSum state label hypotheses hvalid
```

Direct retained dependencies

- AppliedModelingLib.Learning.Online.AdaBoostAdmissible
- AppliedModelingLib.Learning.Online.HedgeWeightState
- AppliedModelingLib.Learning.Online.adaBoostVote
- AppliedModelingLib.Learning.Online.adaBoostVoteWeightSum

AppliedModelingLib.Learning.Online.adaBoostSoftFinalHypothesis

Declaration location: reusable library

Lean declaration body

```
type:
{Training : Type u_1} →
  [inst : Fintype Training] →
    [inst_1 : Nonempty Training] →
      (state : AppliedModelingLib.Learning.Online.HedgeWeightState Training) →
        (label : Training → ℝ) →
          (hypotheses : List (Training → ℝ)) →
            AppliedModelingLib.Learning.Online.AdaBoostAdmissible state label hypotheses → (ℝ → ℝ) → Training → ℝ

value:
fun {Training : Type u_1} [Fintype Training] [Nonempty Training]
    (state : AppliedModelingLib.Learning.Online.HedgeWeightState Training) (label : Training → ℝ)
    (hypotheses : List (Training → ℝ))
    (hvalid : AppliedModelingLib.Learning.Online.AdaBoostAdmissible state label hypotheses) (threshold : ℝ → ℝ)
    (a : Training) =>
  threshold (AppliedModelingLib.Learning.Online.adaBoostNormalizedVote state label hypotheses hvalid a)
```

Direct retained dependencies

- AppliedModelingLib.Learning.Online.AdaBoostAdmissible
- AppliedModelingLib.Learning.Online.HedgeWeightState
- AppliedModelingLib.Learning.Online.adaBoostNormalizedVote

AppliedModelingLib.Learning.Online.adaBoostSoftTrainingError

Declaration location: reusable library

Lean declaration body

```
type:
{Training : Type u_1} →
  [inst : Fintype Training] →
    [inst_1 : Nonempty Training] →
      (state : AppliedModelingLib.Learning.Online.HedgeWeightState Training) →
        (label : Training → ℝ) →
          (hypotheses : List (Training → ℝ)) →
            AppliedModelingLib.Learning.Online.AdaBoostAdmissible state label hypotheses → (ℝ → ℝ) → ℝ

value:
fun {Training : Type u_1} [Fintype Training] [Nonempty Training]
    (state : AppliedModelingLib.Learning.Online.HedgeWeightState Training) (label : Training → ℝ)
    (hypotheses : List (Training → ℝ))
    (hvalid : AppliedModelingLib.Learning.Online.AdaBoostAdmissible state label hypotheses) (threshold : ℝ → ℝ) =>
  ∑ sample : Training,
    state.weight sample *
      |AppliedModelingLib.Learning.Online.adaBoostSoftFinalHypothesis state label hypotheses hvalid threshold sample -
          label sample|
```

Direct retained dependencies

- AppliedModelingLib.Learning.Online.AdaBoostAdmissible
- AppliedModelingLib.Learning.Online.HedgeWeightState
- AppliedModelingLib.Learning.Online.adaBoostSoftFinalHypothesis

AppliedModelingLib.Learning.Online.adaBoostTrainingError

Declaration location: reusable library

Lean declaration body

```
type:
{Training : Type u_1} →
  [inst : Fintype Training] →
    [inst_1 : Nonempty Training] →
      (state : AppliedModelingLib.Learning.Online.HedgeWeightState Training) →
        (label : Training → ℝ) →
          (hypotheses : List (Training → ℝ)) →
            AppliedModelingLib.Learning.Online.AdaBoostAdmissible state label hypotheses → ℝ

value:
fun {Training : Type u_1} [Fintype Training] [Nonempty Training]
    (state : AppliedModelingLib.Learning.Online.HedgeWeightState Training) (label : Training → ℝ)
    (hypotheses : List (Training → ℝ))
    (hvalid : AppliedModelingLib.Learning.Online.AdaBoostAdmissible state label hypotheses) =>
  ∑ sample : Training,
    state.weight sample *
      if
          AppliedModelingLib.Learning.Online.adaBoostFinalHypothesis state label hypotheses hvalid sample =
            label sample then
        0
      else 1
```

Direct retained dependencies

- AppliedModelingLib.Learning.Online.AdaBoostAdmissible
- AppliedModelingLib.Learning.Online.HedgeWeightState
- AppliedModelingLib.Learning.Online.adaBoostFinalHypothesis

Material library prerequisites

AppliedModelingLib.Learning.Online.HedgeWeightState

Source locator: cited publication, lines 286-304

Verbatim paper-source connection

Lemma 1. For any sequence of loss vectors
$\boldsymbol\ell^1,\ldots,\boldsymbol\ell^T$,
$$
\ln\!\left(\sum_{i=1}^N w_i^{T+1}\right)
\leq -(1-\beta)L_{\operatorname{Hedge}(\beta)}.
$$

[Next verbatim source excerpt]

We formalize our on-line allocation model as follows. The
allocation agent A has N options or strategies to choose
from; we number these using the integers 1, ..., N. At each
time step t=1, 2, ..., T, the allocator A decides on a distribu-
tion pt
over the strategies; that is pt
i [U+001E control character]0 is the amount
allocated to strategy i, and [U+001D control character]N
i=1 pt
i =1. Each strategy i then
suffers some loss lt
i which is determined by the (possibly
adversarial) ``environment.'' The loss suffered by A is then
[U+001D control character]n
i=1 pt
i lt
i =pt
} lt
, i.e., the average loss of the strategies with
respect to A's chosen allocation rule. We call this loss func-
tion the mixture loss.
In this paper, we always assume that the loss suffered by
any strategy is bounded so that, without loss of generality,
lt
i # [0, 1]. Besides this condition, we make no assumptions
about the form of the loss vectors lt
, or about the manner
in which they are generated; indeed, the adversary's choice
for lt
may even depend on the allocator's chosen mixture pt
.
The goal of the algorithm A is to minimize its cumulative
loss relative to the loss suffered by the best strategy. That is,
A attempts to minimize its net loss
LA&min
i
Li
where
LA= :
T
t=1
pt
} lt
is the total cumulative loss suffered by algorithm A on the
first T trials, and
Li= :
T
t=1
lt
i
is strategy i's cumulative loss.

[Next verbatim source excerpt]

is updated using the multiplicative rule
wt+1
i =wt
i } ;li
t
. (2)
More generally, it can be shown that our analysis is
applicable with only minor modification to an alternative
update rule of the form
wt+1
i =wt
i } U;(lt
i)
where U; : [0, 1] [U+0014 control character] [0, 1] is any function, parameterized
by ; # [0, 1] satisfying
;r
[U+001D control character]U;(r)[U+001D control character]1&(1&;) r
for all r # [0, 1].
2.1. Analysis
The analysis of Hedge(;) mimics directly that given by
Littlestone and Warmuth [20]. The main idea is to derive
upper and lower bounds on [U+001D control character]N
i=1 wT+1

i which, together,
imply an upper bound on the loss of the algorithm. We
begin with an upper bound.
Lemma 1. For any sequence of loss vectors l1
, ..., lT
,
ln
\:
N
i=1
wT+1
i
+[U+001D control character]&(1&;) LHedge(;) .
Algorithm Hedge(;)
Parameters: ; # [0, 1]
initial weight vector w1
[0, 1]N
with [U+001D control character]N
i=1 w1
i =1
number of trials T
Do for t=1, 2, ..., T
1. Choose allocation
pt
=
wt
[U+001D control character]N
i=1 wt
i
2. Receive loss vector lt
[0, 1]N
from environment.
3. Suffer loss pt
} lt
.
4. Set the new weights vector to be
wt+1
i =wt
i ;li
t
FIG. 1. The on-line allocation algorithm.
Proof. By a convexity argument, it can be shown that
:r
[U+001D control character]1&(1&:) r (3)
for :[U+001E control character]0 and r # [0, 1]. Combined with Eqs. (1) and (2),
this implies
:
N

Formalized review target

The archival source above is not asserted equivalent to this formalized target.

For a finite nonempty strategy family, normalized initial weights, losses in [0,1], and 0 < beta <= 1, the logarithm of total final Hedge weight is at most ↪ -(1-beta) times Hedge's cumulative mixture loss.

Archival source anchor: cited publication, lines 46-65

Lean declaration type (metadata)

```
field weight:
{Action : Type u_1} → [inst : Fintype Action] → AppliedModelingLib.Learning.Online.HedgeWeightState Action → Action → ℝ

field nonneg:
∀ {Action : Type u_1} [inst : Fintype Action] (self : AppliedModelingLib.Learning.Online.HedgeWeightState Action)
  (action : Action), 0 ≤ self.weight action

field total_pos:
∀ {Action : Type u_1} [inst : Fintype Action] (self : AppliedModelingLib.Learning.Online.HedgeWeightState Action),
  0 < ∑ action : Action, self.weight action
```

Exact Lean library declaration

```
/--
An unnormalized finite Hedge weight vector with the nonnegativity and positive
total-mass invariants used by the source algorithm.
-/
structure HedgeWeightState (Action : Type*) [Fintype Action] where
  weight : Action → ℝ
  nonneg : ∀ action, 0 ≤ weight action
  total_pos : 0 < ∑ action, weight action
```

Recorded source-to-library screening

Verdict: matches_approved_corrected_target
Reason: Compared lemma1_weight_potential's allocation model and Figure 1, under FS97-HEDGE-ENDPOINT-DOMAIN-01, with the displayed weight, nonneg, and total_pos fields and every displayed use. These represent finite nonnegative unnormalized weights with a positive denominator, while permitting individual zero weights. The Lemma 1/ Theorem 2 uses separately require initial total one and [0,1] losses; Theorem 5 requires uniform initial weights. The AdaBoost uses supply normalized original weights or M2's D(i)/(k-1) pair initialization and the displayed admissibility predicates. Positive discounts preserve positive mass in all these executions. Requiring no sum-one invariant on later

states is necessary for the source’s unnormalized multiplicative update. The comparison is to the displayed approved corrected target bound to FS97-HEDGE-ENDPOINT-DOMAIN-01; archival equivalence is not claimed.

Reviewer check: ☐ Matches formalized target

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

AppliedModelingLib.Learning.Online.AdaBoostAdmissible

Source locator: cited publication, lines 1099-1108

Verbatim paper-source connection

Theorem 6. Suppose the weak learning algorithm WeakLearn, when called by
AdaBoost, generates hypotheses with errors $\epsilon_1,\ldots,\epsilon_T$
(as defined in Step 3 of Fig. 2). Then the error
$\epsilon=\Pr_{i\sim D}[h_f(x_i)\neq y_i]$ of the final hypothesis h_f
output by AdaBoost is bounded above by
$$
\epsilon\leq 2^T\prod_{t=1}^T\sqrt{\epsilon_t(1-\epsilon_t)}. \tag{14}
$$

[Next verbatim source excerpt]

D(i)=1[U+0012 control character]N. The algorithm maintains a set of weights wt
over
the training examples. On iteration t a distribution pt
is
computed by normalizing these weights. This distribution is
fed to the weak learner WeakLearn which generates a
hypothesis ht that (we hope) has small error with respect to
the distribution.1
Using the new hypothesis ht , the boosting
125
A DECISION THEORETIC GENERALIZATION
1
Some learning algorithms can be generalized to use a given distribution
directly. For instance, gradient based algorithms can use the probability
associated with each example to scale the update step size which is based
on the example. If the algorithm cannot be generalized in this way, the
training sample can be re-sampled to generate a new set of training exam-
ples that is distributed according to the given distribution. The computa-
tion required to generate each re-sampled example takes O(log N) time.

[form-feed in source extraction]
File: 571J 150408 . By:CV . Date:28:07:01 . Time:05:54 LOP8M. V8.0. Page 01:01
Codes: 6184 Signs: 4760 . Length: 56 pic 0 pts, 236 mm
Algorithm AdaBoost
Input: sequence of N labeled examples ((x1 , y1), ..., (xN , yN))
distribution D over the N examples
weak learning algorithm WeakLearn
integer T specifying number of iterations
Initialize the weight vector: w1
i =D(i) for i=1, ..., N.
Do for t=1, 2, ..., T
1. Set
pt
=
wt
[U+001D control character]N
i=1 wt
i
2. Call WeakLearn, providing it with the distribution pt
; get back a
hypothesis ht : X[U+0014 control character] [0, 1].
3. Calculate the error of ht: =t=[U+001D control character]N
i=1 pt
i |ht(xi)& yi |.
4. Set ;t==t [U+0012 control character](1&=t).
5. Set the new weights vector to be
wt+1
i =wt
i ;1&|ht(xi)& yi |
t
Output the hypothesis
hf (x)=
{
1 if [U+001D control character]T
t=1 (log 1[U+0012 control character];t) ht(x)[U+001E control character]1
2 [U+001D control character]T
t=1 log 1[U+0012 control character];t
0 otherwise.
FIG. 2. The adaptive boosting algorithm.
algorithm generates the next weight vector wt+1
, and the
process repeats. After T such iterations, the final hypothesis
hf is output. The hypothesis hf combines the outputs of the
T weak hypotheses using a weighted majority vote.
We call the algorithm AdaBoost because, unlike previous
algorithms, it adjusts adaptively to the errors of the weak
hypotheses returned by WeakLearn. If WeakLearn is a PAC
weak learning algorithm in the sense defined above, then
=t[U+001D control character]1[U+0012 control character]2&# for all t (assuming the examples have been
generated appropriately with yi=c(xi) for some c # C).
However, such a bound on the error need not be known
ahead of time. Our results hold for any =t # [0, 1], and
depend only on the performance of the weak learner on
those distributions that are actually generated during the
boosting process.
The parameter ;t is chosen as a function of =t and is used
for updating the weight vector. The update rule reduces
the probability assigned to those examples on which the

hypothesis makes a good prediction and increases the probability of the examples on which the prediction is poor.2
Note that AdaBoost, unlike boost-by-majority, combines the weak hypotheses by summing their probabilistic predictions. Drucker, Schapire and Simard [9], in experiments they performed using boosting to improve the performance of a real-valued neural network, observed that summing the outcomes of the networks and then selecting the best prediction performs better than selecting the best prediction of each network and then combining them with a majority rule. It is interesting that the new boosting algorithm's final hypothesis uses the same combination rule that was observed to be better in practice, but which previously lacked theoretical justification.
Since it was first introduced, several successful experiments have been conducted using AdaBoost, including work by the authors [12], Drucker and Cortes [8], Jackson and Craven [16], Quinlan [21], and Breiman [3].
4.2. Analysis
Comparing Figs. 1 and 2, there is an obvious similarity between the algorithms Hedge(;) and AdaBoost. This similarity reflects a surprising ``dual'' relationship between the on-line allocation model and the problem of boosting. Put another way, there is a direct mapping or reduction of the boosting problem to the on-line allocation problem. In

Formalized review target

The archival source above is not asserted equivalent to this formalized target.

For the finite Figure-2 AdaBoost execution with every observed error strictly between zero and one, the final hypothesis's training error is at most 2^T
↪ times the product of sqrt(epsilon_t(1-epsilon_t)); errors may lie on either side of one half.

Archival source anchor: cited publication, lines 178-186

Lean-expanded library semantic target

```
type:
{Training : Type u_1} →
  [inst : Fintype Training] →
    [Nonempty Training] →
      AppliedModelingLib.Learning.Online.HedgeWeightState Training → (Training → ℝ) → List (Training → ℝ) → Prop

value:
fun {Training : Type u_1} [Fintype Training] [Nonempty Training]
    (state : AppliedModelingLib.Learning.Online.HedgeWeightState Training) (label : Training → ℝ)
    (a : List (Training → ℝ)) =>
  List.brecOn (motive := fun (x : List (Training → ℝ)) =>
    AppliedModelingLib.Learning.Online.HedgeWeightState Training → Prop) a
    (AppliedModelingLib.Learning.Online.AdaBoostAdmissible._f label) state
```

Exact Lean library declaration

```
/--
The recursively generated errors of a finite AdaBoost run are all in the
interior domain of Figure 2's displayed adaptive update.
-/
def AdaBoostAdmissible
    {Training : Type*} [Fintype Training] [Nonempty Training]
    (state : HedgeWeightState Training) (label : Training → ℝ) :
    List (Training → ℝ) → Prop
  | [] => True
  | hypothesis :: remaining =>
      ∃ herror : 0 < adaBoostError state label hypothesis ∧
          adaBoostError state label hypothesis < 1,
        AdaBoostAdmissible (adaBoostStep state label hypothesis herror) label remaining
```

Direct retained dependencies

- AppliedModelingLib.Learning.Online.HedgeWeightState
- AppliedModelingLib.Learning.Online.adaBoostError
- AppliedModelingLib.Learning.Online.adaBoostStep

Recorded source-to-library screening

Verdict: matches_approved_corrected_target
Reason: Compared theorem6_adaboost_error's Figure 2 Steps 3-5 and the complete FS97-ADABOOST-ENDPOINT-DOMAIN-03 record with the displayed recursive predicate. Each nonempty round requires exactly 0 < epsilon < 1 for epsilon equal to the current normalized mean absolute error, advances by weights times (epsilon/(1-epsilon))^(1-|h-y|), and repeats; the empty run is admissible. The displayed Theorem 6 and Equations 21-23 uses supply normalized initial weights, binary labels, and hypotheses in [0,1]. M1/M2 uses apply the displayed binary reductions, with M1's upper-half restriction supplied separately. Theorem 9 retains this execution and explicitly restricts its additional normalization. No weak-learning edge is imposed by this prerequisite. The comparison is to the displayed approved corrected target bound to FS97-ADABOOST-ENDPOINT-DOMAIN-03; archival equivalence is not claimed.
Reviewer check: ☐ Matches formalized target

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

AppliedModelingLib.Learning.Online.AdaBoostRDensityState

Source locator: cited publication, lines 2418-2422

Verbatim paper-source connection

Theorem 12. Suppose WeakLearn, when called by AdaBoost.R, generates
hypotheses with errors $\epsilon_1,\ldots,\epsilon_T$, where ϵ_t is
as defined in Fig. 5. Then the mean squared error
$\epsilon=\mathbb E_{i\sim D}[(h_f(x_i)-y_i)^2]$ is bounded above by
$$
\epsilon\leq2^T\prod_{t=1}^T\sqrt{\epsilon_t(1-\epsilon_t)}. \tag{26}
$$

[Next verbatim source excerpt]

5.3. Boosting Regression Algorithms
In this section we show how boosting can be used for a
regression problem. In this setting, the label space is
Y=[0, 1]. As before, the learner receives examples (x, y)
chosen at random according to some distribution P, and its
goal is to find a hypothesis h: X [U+0014 control character] Y which, given some x
value, predicts approximately the value y that is likely to be
seen. More precisely, the learner attempts to find an h with
small mean squared error (MSE):
E(x, y)t P[(h(x)& y)2
]. (25)
Our methods can be applied to any reasonable bounded
error measure, but, for the sake of concreteness, we concen-
trate here on the squared error measure.
Following our approach for classification problems, we
assume that the leaner has been provided with a training set
(x1 , y1), ..., (xN , yN) of examples distributed according to
P, and we focus only on the minimization of the empirical
MSE:
1
N
:
N
i=1
(h(xi)& yi)2
.
Using techniques similar to those outlined in Section 4.3,
the true MSE given in Eq. (25) can be related to the empiri-
cal MSE.
To derive a boosting algorithm in this context, we reduce
the given regression problem to a binary classification
problem, and then apply AdaBoost. As was done for the
reductions used in the proofs of Theorems 10 and 11, we
mark with tildes all variables in the reduced (AdaBoost)
space. For each example (xi , yi) in the training set, we
define a continuum of examples indexed by pairs (i, y) for
all y # [0, 1]: the associated instance is x
~ i, y=(xi , y), and
the label is y
~ i, y=[U+001A control character]y[U+001E control character] yi[U+0019 control character]. (Recall that [U+001A control character]?[U+0019 control
↪ character] is 1 if predicate
? holds and 0 otherwise.) Although it is obviously infeasible
to explicitly maintain an infinitely large training set, we will
see later how this method can be implemented efficiently.
Also, although the results of Section 4 only dealt with
finitely large training sets, the extension to infinite training
sets is straightforward.
Thus, informally, each instance (xi , yi) is mapped to an
infinite set of binary questions, one for each y # Y, and each
of the form: ``Is the correct label yi bigger or smaller than y?''
In a similar manner, each hypothesis h: X [U+0014 control character] Y is reduced
to a binary-valued hypothesis h
[U+0019 control character] : X_Y[U+0014 control character] [0, 1] defined by
the rule
h
[U+0019 control character] (x, y)=[U+001A control character]y[U+001E control character]h(x)[U+0019 control character].
Thus, h
[U+0019 control character] attempts to answer these binary questions in a
natural way using the estimated value h(x).
Finally, as was done for classification problems, we
assume we are given a distribution D over the training set;
ordinarily, this will be uniform so that D(i)=1[U+0012 control character]N. In our
reduction, this distribution is mapped to a density D
[U+0019 control character] over
pairs (i, y) in such a way that minimization of classification
error in the reduced space is equivalent to minimization of
MSE for the original problem. To do this, we define
D
[U+0019 control character] (i, y)=
D(i) | y& yi |
Z
where Z is a normalization constant:
Z= :
N
i=1
D(i) |
1
0
| y& yi | dy.

It is straightforward to show that 1[U+0012 control character]4[U+001D control character]Z[U+001D control character]1[U+0012 control character]2.
135
A DECISION THEORETIC GENERALIZATION

[form-feed in source extraction]
File: 571J 150418 . By:CV . Date:28:07:01 . Time:05:54 LOP8M. V8.0. Page 01:01
Codes: 5416 Signs: 3774 . Length: 56 pic 0 pts, 236 mm
If we calculate the binary error of h
[U+0019 control character] with respect to the
density D
[U+0019 control character] , we find that, as desired, it is directly proportional
to the mean squared error:
:
N
i=1
|
1
0
| y
~ i, y&h
[U+0019 control character] (x
~ i, y)| D
[U+0019 control character] (i, y) dy
=
1
Z
:
N
i=1
D(i)
}|
h(xi)
yi
| y& yi | dy
}
=
1
2Z
:
N
i=1
D(i)(h(xi)& yi)2
.
The constant of proportionality is 1[U+0012 control character](2Z) # [1, 2].
Unravelling this reduction, we obtain the regression
boosting procedure AdaBoost.R shown in Fig. 5. As
prescribed by the reduction, AdaBoost.R maintains a weight
wt
i, y for each instance i and label y # Y. The initial weight
function w1
is exactly the density D
[U+0019 control character] defined above. By nor-
malizing the weights wt
, a density pt
is defined at Step 1 and
provided to the weak learner at Step 2. The goal of the weak
learner is to find a hypothesis ht : X[U+0014 control character] Y that minimizes the
loss =t defined in Step 3. Finally, at Step 5, the weights are
updated as prescribed by the reduction.
The definition of =t at Step 3 follows directly from the
reduction above; it is exactly the classification error of h
[U+0019 control character] f in
the reduced space. Note that, similar to AdaBoost.M2,
AdaBoost.R not only varies the distribution over the
examples (xi , yi), but also modifies from round to round the
definition of the loss suffered by a hypothesis on each
example. Thus, although our ultimate goal is minimization
of the squared error, the weak learner must be able to
handle loss functions that are more complicated than MSE.
The final hypothesis hf also is consistent with the reduc-
tion. Each reduced weak hypothesis h
[U+0019 control character] f (x, y) is non-
decreasing as a function of y. Thus, the final hypothesis h
[U+0019 control character] f
generated by AdaBoost in the reduced space, being the
threshold of a weighted sum of these hypotheses, also is
non-decreasing as a function of y. As the output of h
[U+0019 control character] f is
binary, this implies that for every x there is one value of y
for which h
[U+0019 control character] f (x, y$)=0 for all y$< y and h
[U+0019 control character] f (x, y$)=1 for all
y$> y. This is exactly the value of y given by hf (x) as defined
in the figure. Note that hf is actually computing a weighted
median of the weak hypotheses.
At first, it might seem impossible to maintain weights wt
i, y
over an uncountable set of points. However, on closer
inspection, it can be seen that, when viewed as a function of
y, wt
i, y is a piece-wise linear function. For t=1, w1
i, y has two
linear pieces, and each update at Step 5 potentially breaks
one of the pieces in two at the point ht(xi). Initializing,
storing and updating such piece-wise linear functions are
all straightforward operations. Also, the integrals which
appear in the figure can be evaluated explicitly since these

only involve integration of piece-wise linear functions.

[Next verbatim source excerpt]

Algorithm AdaBoost.R
Input: sequence of N examples ((x1 , y1)..., (xN , yN)) with labels
yi # Y=[0, 1]
distribution D over the examples
weak learning algorithm WeakLearn
integer T specifying number of iterations
Initialize the weight vector:
w1
i, y=
D(i) | y& yi |
Z
for i=1, ..., N, y # Y, where
Z= :
N
i=1
D(i)
|
1
0
|y& yi | dy.
Do for t=1, 2, ..., T
1. Set
pt
=
wt
[U+001D control character]N
i=1 [U+001E control character]1
0 wt
i, y dy
.
2. Call WeakLearn, providing it with the density pt
; get back a
hypothesis ht : X_Y.
3. Calculate the loss of ht :
=t= :
N
i=1
}|
ht(xi)
yi
pt
i, y dy
}.
If =t>1[U+0012 control character]2, then set T=t&1 and abort the loop.
4. Set ;t==t [U+0012 control character](1&=t).
5. Set the new weights vector to be
wt+1
i, y =
{
wt
i, y
wt
i, y ;t
if yi[U+001D control character] y[U+001D control character]ht(xi) or ht(xi)[U+001D control character] y[U+001D control character] yi
otherwise.
for i=1, ..., N, y # Y.
Output the hypothesis
hf (x)=inf
{y # Y: :
t: ht(x)[U+001D control character] y
log(1[U+0012 control character];t)[U+001E control character]1
2 :
t
log(1[U+0012 control character];t)
=.
FIG. 5. An extension of AdaBoost to regression problems.
The following theorem describes our performance
guarantee for AdaBoost.R. The proof follows from the
reduction described above coupled with a direct application
of Theorem 6.

Formalized review target

The archival source above is not asserted equivalent to this formalized target.

For the finite Figure-5 AdaBoost.R execution with every observed reduced error positive and at most one half, the final weighted-median regressor has mean squared error at most 2^T times the product of sqrt(epsilon_t(1-epsilon_t)).

Archival source anchor: cited publication, lines 315-363

Lean declaration type (metadata)

```
field weight:
{Training : Type u_1} →
  [inst : Fintype Training] → AppliedModelingLib.Learning.Online.AdaBoostRDensityState Training → Training → ℝ → ℝ

field integrable:
∀ {Training : Type u_1} [inst : Fintype Training]
  (self : AppliedModelingLib.Learning.Online.AdaBoostRDensityState Training) (sample : Training),
  MeasureTheory.Integrable (self.weight sample) AppliedModelingLib.Learning.Online.adaBoostRUnitMeasure
```

```
field nonneg:
∀ {Training : Type u_1} [inst : Fintype Training]
  (self : AppliedModelingLib.Learning.Online.AdaBoostRDensityState Training) (sample : Training) (y : ℝ),
  0 ≤ self.weight sample y

field total_pos:
∀ {Training : Type u_1} [inst : Fintype Training]
  (self : AppliedModelingLib.Learning.Online.AdaBoostRDensityState Training),
  0 < AppliedModelingLib.Learning.Online.adaBoostRDensityTotal self.weight
```

Exact Lean library declaration

```
/-- A nonnegative, integrable, positive-mass density over the reduced space. -/
structure AdaBoostRDensityState (Training : Type*) [Fintype Training] where
  weight : Training → ℝ → ℝ
  integrable : ∀ sample, Integrable (weight sample) adaBoostRUnitMeasure
  nonneg : ∀ sample y, 0 ≤ weight sample y
  total_pos : 0 < adaBoostRDensityTotal weight
```

Direct retained dependencies

- AppliedModelingLib.Learning.Online.adaBoostRDensityTotal
- AppliedModelingLib.Learning.Online.adaBoostRUnitMeasure

Recorded source-to-library screening

Verdict: matches_approved_corrected_target
Reason: Compared theorem12_adaboost_r_error's reduced continuum of pairs (i,y), Figure 5 normalization, and its pinned positive-error correction with the four displayed state fields and all density/update uses. The state has a real weight for each finite sample and scalar threshold, nonnegativity, integrability under Lebesgue measure restricted to [0,1], and positive total mass. It permits zero density at particular points and examples. Theorem 12 uses the displayed base-state constructor under nonnegative normalized example weights and [0,1] labels, then the displayed multiplicative updates. Allowing more general integrable states or values outside [0,1] does not restrict these source executions or change any relevant integral. The comparison is to the displayed approved corrected target bound to FS97-ADABOOST-VARIANTS-ENDPOINT-DOMAIN-06; archival equivalence is not claimed.
Reviewer check: ☐ Matches formalized target
If left unchecked, explain the discrepancy or uncertainty below.
Reviewer annotation

AppliedModelingLib.Learning.Online.AdaBoostRAdmissible

Source locator: cited publication, lines 2418-2422

Verbatim paper-source connection

Theorem 12. Suppose WeakLearn, when called by AdaBoost.R, generates
hypotheses with errors $\epsilon_1,\ldots,\epsilon_T$, where ϵ_t is
as defined in Fig. 5. Then the mean squared error
$\epsilon=\mathbb E_{i\sim D}[(h_f(x_i)-y_i)^2]$ is bounded above by
$$
\epsilon\leq2^T\prod_{t=1}^T\sqrt{\epsilon_t(1-\epsilon_t)}. \tag{26}
$$

[Next verbatim source excerpt]

5.3. Boosting Regression Algorithms
In this section we show how boosting can be used for a
regression problem. In this setting, the label space is
Y=[0, 1]. As before, the learner receives examples (x, y)
chosen at random according to some distribution P, and its
goal is to find a hypothesis h: X [U+0014 control character] Y which, given some x
value, predicts approximately the value y that is likely to be
seen. More precisely, the learner attempts to find an h with
small mean squared error (MES):
E(x, y)t P[(h(x)& y)2
]. (25)
Our methods can be applied to any reasonable bounded
error measure, but, for the sake of concreteness, we concen-
trate here on the squared error measure.
Following our approach for classification problems, we
assume that the leaner has been provided with a training set
(x1 , y1), ..., (xN , yN) of examples distributed according to
P, and we focus only on the minimization of the empirical
MSE:
1
N
:
N
i=1
(h(xi)& yi)2
.
Using techniques similar to those outlined in Section 4.3,
the true MSE given in Eq. (25) can be related to the empiri-
cal MSE.
To derive a boosting algorithm in this context, we reduce
the given regression problem to a binary classification
problem, and then apply AdaBoost. As was done for the
reductions used in the proofs of Theorems 10 and 11, we
mark with tildes all variables in the reduced (AdaBoost)
space. For each example (xi , yi) in the training set, we
define a continuum of examples indexed by pairs (i, y) for
all y # [0, 1]: the associated instance is x
~ i, y=(xi , y), and
the label is y
~ i, y=[U+001A control character]y[U+001E control character] yi[U+0019 control character]. (Recall that [U+001A control character]?[U+0019 control
↪ character] is 1 if predicate
? holds and 0 otherwise.) Although it is obviously infeasible
to explicitly maintain an infinitely large training set, we will
see later how this method can be implemented efficiently.
Also, although the results of Section 4 only dealt with
finitely large training sets, the extension to infinite training
sets is straightforward.
Thus, informally, each instance (xi , yi) is mapped to an
infinite set of binary questions, one for each y # Y, and each
of the form: ``Is the correct label yi bigger or smaller than y?''
In a similar manner, each hypothesis h: X [U+0014 control character] Y is reduced
to a binary-valued hypothesis h
[U+0019 control character] : X_Y[U+0014 control character] [0, 1] defined by
the rule
h
[U+0019 control character] (x, y)=[U+001A control character]y[U+001E control character]h(x)[U+0019 control character].
Thus, h
[U+0019 control character] attempts to answer these binary questions in a
natural way using the estimated value h(x).
Finally, as was done for classification problems, we
assume we are given a distribution D over the training set;
ordinarily, this will be uniform so that D(i)=1[U+0012 control character]N. In our
reduction, this distribution is mapped to a density D
[U+0019 control character] over
pairs (i, y) in such a way that minimization of classification
error in the reduced space is equivalent to minimization of
MSE for the original problem. To do this, we define
D
[U+0019 control character] (i, y)=
D(i) | y& yi |
Z
where Z is a normalization constant:
Z= :
N
i=1
D(i) |
1
0
| y& yi | dy.

It is straightforward to show that 1[U+0012 control character]4[U+001D control character]Z[U+001D control character]1[U+0012 control character]2.
135
A DECISION THEORETIC GENERALIZATION

[form-feed in source extraction]
File: 571J 150418 . By:CV . Date:28:07:01 . Time:05:54 LOP8M. V8.0. Page 01:01
Codes: 5416 Signs: 3774 . Length: 56 pic 0 pts, 236 mm
If we calculate the binary error of h
[U+0019 control character] with respect to the
density D
[U+0019 control character] , we find that, as desired, it is directly proportional
to the mean squared error:
:
N
i=1
|
1
0
| y
~ i, y&h
[U+0019 control character] (x
~ i, y)| D
[U+0019 control character] (i, y) dy
=
1
Z
:
N
i=1
D(i)
}|
h(xi)
yi
| y& yi | dy
}
=
1
2Z
:
N
i=1
D(i)(h(xi)& yi)2
.
The constant of proportionality is 1[U+0012 control character](2Z) # [1, 2].
Unravelling this reduction, we obtain the regression
boosting procedure AdaBoost.R shown in Fig. 5. As
prescribed by the reduction, AdaBoost.R maintains a weight
wt
i, y for each instance i and label y # Y. The initial weight
function w1
is exactly the density D
[U+0019 control character] defined above. By nor-
malizing the weights wt
, a density pt
is defined at Step 1 and
provided to the weak learner at Step 2. The goal of the weak
learner is to find a hypothesis ht : X[U+0014 control character] Y that minimizes the
loss =t defined in Step 3. Finally, at Step 5, the weights are
updated as prescribed by the reduction.
The definition of =t at Step 3 follows directly from the
reduction above; it is exactly the classification error of h
[U+0019 control character] f in
the reduced space. Note that, similar to AdaBoost.M2,
AdaBoost.R not only varies the distribution over the
examples (xi , yi), but also modifies from round to round the
definition of the loss suffered by a hypothesis on each
example. Thus, although our ultimate goal is minimization
of the squared error, the weak learner must be able to
handle loss functions that are more complicated than MSE.
The final hypothesis hf also is consistent with the reduc-
tion. Each reduced weak hypothesis h
[U+0019 control character] f (x, y) is non-
decreasing as a function of y. Thus, the final hypothesis h
[U+0019 control character] f
generated by AdaBoost in the reduced space, being the
threshold of a weighted sum of these hypotheses, also is
non-decreasing as a function of y. As the output of h
[U+0019 control character] f is
binary, this implies that for every x there is one value of y
for which h
[U+0019 control character] f (x, y$)=0 for all y$< y and h
[U+0019 control character] f (x, y$)=1 for all
y$> y. This is exactly the value of y given by hf (x) as defined
in the figure. Note that hf is actually computing a weighted
median of the weak hypotheses.
At first, it might seem impossible to maintain weights wt
i, y
over an uncountable set of points. However, on closer
inspection, it can be seen that, when viewed as a function of
y, wt
i, y is a piece-wise linear function. For t=1, w1
i, y has two
linear pieces, and each update at Step 5 potentially breaks
one of the pieces in two at the point ht(xi). Initializing,
storing and updating such piece-wise linear functions are
all straightforward operations. Also, the integrals which
appear in the figure can be evaluated explicitly since these

only involve integration of piece-wise linear functions.

[Next verbatim source excerpt]

Algorithm AdaBoost.R
Input: sequence of N examples ((x1 , y1)..., (xN , yN)) with labels
yi # Y=[0, 1]
distribution D over the examples
weak learning algorithm WeakLearn
integer T specifying number of iterations
Initialize the weight vector:
w1
i, y=
D(i) | y& yi |
Z
for i=1, ..., N, y # Y, where
Z= :
N
i=1
D(i)
|
1
0
|y& yi | dy.
Do for t=1, 2, ..., T
1. Set
pt
=
wt
[U+001D control character]N
i=1 [U+001E control character]1
0 wt
i, y dy
.
2. Call WeakLearn, providing it with the density pt
; get back a
hypothesis ht : X_Y.
3. Calculate the loss of ht :
=t= :
N
i=1
}|
ht(xi)
yi
pt
i, y dy
}.
If =t>1[U+0012 control character]2, then set T=t&1 and abort the loop.
4. Set ;t==t [U+0012 control character](1&=t).
5. Set the new weights vector to be
wt+1
i, y =
{
wt
i, y
wt
i, y ;t
if yi[U+001D control character] y[U+001D control character]ht(xi) or ht(xi)[U+001D control character] y[U+001D control character] yi
otherwise.
for i=1, ..., N, y # Y.
Output the hypothesis
hf (x)=inf
{y # Y: :
t: ht(x)[U+001D control character] y
log(1[U+0012 control character];t)[U+001E control character]1
2 :
t
log(1[U+0012 control character];t)
=.
FIG. 5. An extension of AdaBoost to regression problems.
The following theorem describes our performance
guarantee for AdaBoost.R. The proof follows from the
reduction described above coupled with a direct application
of Theorem 6.

Formalized review target

The archival source above is not asserted equivalent to this formalized target.

For the finite Figure-5 AdaBoost.R execution with every observed reduced error positive and at most one half, the final weighted-median regressor has
↪ mean squared error at most 2^T times the product of sqrt(epsilon_t(1-epsilon_t)).

Archival source anchor: cited publication, lines 315-363

Lean-expanded library semantic target

type:
{Training : Type u_1} →
  [inst : Fintype Training] →
    AppliedModelingLib.Learning.Online.AdaBoostRDensityState Training → (Training → ℝ) → List (Training → ℝ) → Prop

value:
fun {Training : Type u_1} [Fintype Training] (state : AppliedModelingLib.Learning.Online.AdaBoostRDensityState Training)
    (label : Training → ℝ) (a : List (Training → ℝ)) =>
  List.brecOn (motive := fun (x : List (Training → ℝ)) =>

```
AppliedModelingLib.Learning.Online.AdaBoostRDensityState Training → Prop) a
(AppliedModelingLib.Learning.Online.AdaBoostRAdmissible._f label) state
```

Exact Lean library declaration

```
/-- The recursively recorded domain conditions for a literal Figure-5 run. -/
def AdaBoostRAdmissible
    {Training : Type*} [Fintype Training]
    (state : AdaBoostRDensityState Training) (label : Training → ℝ) :
    List (Training → ℝ) → Prop
  | [] => True
  | hypothesis :: remaining =>
      ∃ herror : 0 < adaBoostRReducedError state label hypothesis ∧
          adaBoostRReducedError state label hypothesis ≤ 1 / 2,
        AdaBoostRAdmissible (adaBoostRStep state label hypothesis herror) label remaining
```

Direct retained dependencies

- AppliedModelingLib.Learning.Online.AdaBoostRDensityState
- AppliedModelingLib.Learning.Online.adaBoostRReducedError
- AppliedModelingLib.Learning.Online.adaBoostRStep

Recorded source-to-library screening **Verdict:** matches_approved_corrected_target
Reason: Compared theorem12_adaboost_r_error's Figure 5 Steps 1-5 and FS97-ADABOOST-VARIANTS-ENDPOINT-DOMAIN-06 with the predicate and its displayed Theorem 12 use. Every executed round has 0 < epsilon <= 1/2, where epsilon is the current density-normalized integral over the interval between truth and prediction; the next density keeps interval weights and multiplies the complement by epsilon/(1-epsilon). The recursion permits an empty retained run and represents precisely the executed prefix before the printed stopping rule. The paper use starts from the displayed base density with normalized nonnegative example weights and labels/hypotheses in [0,1]. The comparison is to the displayed approved corrected target bound to FS97-ADABOOST-VARIANTS-ENDPOINT-DOMAIN-06; archival equivalence is not claimed.
Reviewer check: ☐ Matches formalized target
If left unchecked, explain the discrepancy or uncertainty below.
Reviewer annotation

AppliedModelingLib.Learning.Online.GeneralDecisionHedgeGame

Source locator: cited publication, lines 625-645

Verbatim paper-source connection

Theorem 5. For any loss function λ, for any set of experts, and for
any sequence of outcomes, the expected loss of $\operatorname{Hedge}(\beta)$
if used as described above is at most
$$
\sum_{t=1}^T\Lambda(\mathcal D^t,\omega^t)
\leq \min_i\sum_{t=1}^T\Lambda(\mathcal E_i^t,\omega^t)
+\sqrt{2\widetilde L\ln N}+\ln N,
$$
where $\widetilde L\leq T$ is an assumed bound on the expected loss of the
best expert, and $\beta=g(\widetilde L/\ln N)$.

[Next verbatim source excerpt]

O(- (ln N)[U+0012 control character]T). However, if L
[U+0019 control character] is close to zero, then the rate
of convergence will be much faster, roughly, O((ln N)[U+0012 control character]T).
Lemma 4 can also be applied to the other bounds given in
Theorem 2 to obtain analogous results.
The bound given in Eq. (11) can be improved in special
cases in which the loss is a function of a prediction and an
outcome and this function is of a special form (see Example 4
below). However, for the general case, one cannot improve
the square-root term - 2L
[U+0019 control character] ln N, by more than a constant
factor. This is a corollary of the lower bound given by Cesa-
Bianchi et al. ([4], Theorem 7) who analyze an on-line
prediction problem that can be seen as a special case of the
on-line allocation model.
3. APPLICATIONS
The framework described up to this point is quite general
and can be applied in a wide variety of learning problems.
Consider the following set-up used by Chung [5]. We are
given a decision space 2, a space of outcomes 0, and a
bounded loss function *: 2_0 [U+0014 control character] [0, 1]. (Actually, our
results require only that * be bounded, but, by rescaling, we
can assume that its range is [0, 1].) At every time step t, the
learning algorithm selects a decision $t
2, receives an out-
come |t
0, and suffers loss *($t
, |t
). More generally, we
may allow the learner to select a distribution Dt
over the
space of decisions, in which case it suffers the expected loss
of a decision randomly selected according to Dt
; that is, its
expected loss is 4(Dt
, |t
) where
4(D, |)=E$t D[*($, |)].
To decide on distribution Dt
, we assume that the learner
has access to a set of N experts. At every time step t, expert
i produces its own distribution Et
i on 2, and suffers loss
4(Et
i , |t
).
The goal of the learner is to combine the distributions
produced by the experts so as to suffer expected loss ``not
much worse'' than that of the best expert.
The results of Section 2 provide a method for solving this
problem. Specifically, we run algorithm Hedge(;), treating
each expert as a strategy. At every time step, Hedge(;)
produces a distribution pt
on the set of experts which is used
to construct the mixture distribution
Dt
= :
N
i=1
pt
i Et
i .
For any outcome |t
, the loss suffered by Hedge(;) will
then be
4(Dt
, |t
)= :
N
i=1
pt
i 4(Et
i , |t
).
Thus, if we define lt
i =4(Et

i , |t
) then the loss suffered by the
learner is pt
} lt
, i.e., exactly the mixture loss that was
analyzed in Section 2.
Hence, the bounds of Section 2 can be applied to our
current framework. For instance, applying Eq. (11), we
obtain the following:

Formalized review target The archival source above is not asserted equivalent to this formalized target.

For a finite expert family of size N >= 2, a positive best-expert loss bound L-tilde no larger than the horizon, and the general decision-prediction mixture model, Hedge with beta = g(L-tilde/log N) has cumulative expected loss at most the best expert loss plus sqrt(2 L-tilde log N) + log N.

Archival source anchor: cited publication, lines 143-152

Lean declaration type (metadata)

```
field Distribution:
{Expert : Type u} →
  {Decision : Type v} →
    {Outcome : Type w} →
      [inst : Fintype Expert] →
        [inst_1 : DecidableEq Expert] →
          [inst_2 : MeasurableSpace Decision] →
            AppliedModelingLib.Learning.Online.GeneralDecisionHedgeGame Expert Decision Outcome → Type v

field toMeasure:
{Expert : Type u} →
  {Decision : Type v} →
    {Outcome : Type w} →
      [inst : Fintype Expert] →
        [inst_1 : DecidableEq Expert] →
          [inst_2 : MeasurableSpace Decision] →
            (self : AppliedModelingLib.Learning.Online.GeneralDecisionHedgeGame Expert Decision Outcome) →
              self.Distribution → MeasureTheory.Measure Decision

field toMeasure_injective:
∀ {Expert : Type u} {Decision : Type v} {Outcome : Type w} [inst : Fintype Expert] [inst_1 : DecidableEq Expert]
  [inst_2 : MeasurableSpace Decision]
  (self : AppliedModelingLib.Learning.Online.GeneralDecisionHedgeGame Expert Decision Outcome),
  Function.Injective self.toMeasure

field toMeasure_isProbability:
∀ {Expert : Type u} {Decision : Type v} {Outcome : Type w} [inst : Fintype Expert] [inst_1 : DecidableEq Expert]
  [inst_2 : MeasurableSpace Decision]
  (self : AppliedModelingLib.Learning.Online.GeneralDecisionHedgeGame Expert Decision Outcome)
  (distribution : self.Distribution), MeasureTheory.IsProbabilityMeasure (self.toMeasure distribution)

field loss:
{Expert : Type u} →
  {Decision : Type v} →
    {Outcome : Type w} →
      [inst : Fintype Expert] →
        [inst_1 : DecidableEq Expert] →
          [inst_2 : MeasurableSpace Decision] →
            AppliedModelingLib.Learning.Online.GeneralDecisionHedgeGame Expert Decision Outcome → Decision → Outcome → ℝ

field loss_measurable:
∀ {Expert : Type u} {Decision : Type v} {Outcome : Type w} [inst : Fintype Expert] [inst_1 : DecidableEq Expert]
  [inst_2 : MeasurableSpace Decision]
  (self : AppliedModelingLib.Learning.Online.GeneralDecisionHedgeGame Expert Decision Outcome) (outcome : Outcome),
  Measurable fun (decision : Decision) => self.loss decision outcome

field loss_mem_Icc:
∀ {Expert : Type u} {Decision : Type v} {Outcome : Type w} [inst : Fintype Expert] [inst_1 : DecidableEq Expert]
  [inst_2 : MeasurableSpace Decision]
  (self : AppliedModelingLib.Learning.Online.GeneralDecisionHedgeGame Expert Decision Outcome) (decision : Decision)
  (outcome : Outcome), 0 ≤ self.loss decision outcome ∧ self.loss decision outcome ≤ 1

field expectedLoss:
{Expert : Type u} →
  {Decision : Type v} →
    {Outcome : Type w} →
      [inst : Fintype Expert] →
        [inst_1 : DecidableEq Expert] →
          [inst_2 : MeasurableSpace Decision] →
            (self : AppliedModelingLib.Learning.Online.GeneralDecisionHedgeGame Expert Decision Outcome) →
              self.Distribution → Outcome → ℝ

field expectedLoss_eq_integral:
∀ {Expert : Type u} {Decision : Type v} {Outcome : Type w} [inst : Fintype Expert] [inst_1 : DecidableEq Expert]
  [inst_2 : MeasurableSpace Decision]
  (self : AppliedModelingLib.Learning.Online.GeneralDecisionHedgeGame Expert Decision Outcome)
  (distribution : self.Distribution) (outcome : Outcome),
  self.expectedLoss distribution outcome =
    ∫ (decision : Decision), self.loss decision outcome ∂self.toMeasure distribution

field expectedLoss_mem_Icc:
∀ {Expert : Type u} {Decision : Type v} {Outcome : Type w} [inst : Fintype Expert] [inst_1 : DecidableEq Expert]
  [inst_2 : MeasurableSpace Decision]
```

```
  (self : AppliedModelingLib.Learning.Online.GeneralDecisionHedgeGame Expert Decision Outcome)
  (distribution : self.Distribution) (outcome : Outcome),
  0 ≤ self.expectedLoss distribution outcome ∧ self.expectedLoss distribution outcome ≤ 1

field mixture:
{Expert : Type u} →
  {Decision : Type v} →
    {Outcome : Type w} →
      [inst : Fintype Expert] →
        [inst_1 : DecidableEq Expert] →
          [inst_2 : MeasurableSpace Decision] →
            (self : AppliedModelingLib.Learning.Online.GeneralDecisionHedgeGame Expert Decision Outcome) →
              PMF Expert → (Expert → self.Distribution) → self.Distribution

field mixture_toMeasure:
∀ {Expert : Type u} {Decision : Type v} {Outcome : Type w} [inst : Fintype Expert] [inst_1 : DecidableEq Expert]
  [inst_2 : MeasurableSpace Decision]
  (self : AppliedModelingLib.Learning.Online.GeneralDecisionHedgeGame Expert Decision Outcome) (allocation : PMF Expert)
  (expertDecision : Expert → self.Distribution),
  self.toMeasure (self.mixture allocation expertDecision) =
    ∑ expert : Expert, allocation expert • self.toMeasure (expertDecision expert)

field mixture_expectedLoss:
∀ {Expert : Type u} {Decision : Type v} {Outcome : Type w} [inst : Fintype Expert] [inst_1 : DecidableEq Expert]
  [inst_2 : MeasurableSpace Decision]
  (self : AppliedModelingLib.Learning.Online.GeneralDecisionHedgeGame Expert Decision Outcome) (allocation : PMF Expert)
  (expertDecision : Expert → self.Distribution) (outcome : Outcome),
  self.expectedLoss (self.mixture allocation expertDecision) outcome =
    AppliedModelingLib.pmfExp allocation fun (expert : Expert) => self.expectedLoss (expertDecision expert) outcome
```

Exact Lean library declaration

```
/--
A randomized decision game for a fixed finite expert set. Each value of
`Distribution` is injectively represented by an actual probability measure on
the source decision space. Expected loss is the integral of the source's
bounded measurable loss, and finite expert mixing is represented by the
corresponding weighted sum of measures.

The explicit measure representation keeps the reusable theorem independent of
a particular distribution data structure without weakening the paper's model
to an unrelated abstract carrier.
-/
structure GeneralDecisionHedgeGame (Expert : Type u) (Decision : Type v) (Outcome : Type w)
    [Fintype Expert] [DecidableEq Expert] [MeasurableSpace Decision] where
  /-- A representation of randomized decisions on the ambient decision space. -/
  Distribution : Type v
  /-- The probability measure represented by a randomized decision. -/
  toMeasure : Distribution → Measure Decision
  /-- The representation has no semantically distinct duplicate distributions. -/
  toMeasure_injective : Function.Injective toMeasure
  /-- Every represented randomized decision is a probability measure. -/
  toMeasure_isProbability : ∀ distribution, IsProbabilityMeasure (toMeasure distribution)
  /-- The source loss function on realized decisions and outcomes. -/
  loss : Decision → Outcome → ℝ
  /-- The source loss is measurable for every realized outcome. -/
  loss_measurable : ∀ outcome, Measurable (fun decision => loss decision outcome)
  /-- Source normalization of the realized loss to `[0,1]`. -/
  loss_mem_Icc : ∀ decision outcome,
    0 ≤ loss decision outcome ∧ loss decision outcome ≤ 1
  /-- Expected source loss of a randomized decision against an outcome. -/
  expectedLoss : Distribution → Outcome → ℝ
  /-- Expected loss is the integral of the source loss under the represented law. -/
  expectedLoss_eq_integral : ∀ distribution outcome,
    expectedLoss distribution outcome =
      ∫ decision, loss decision outcome ∂toMeasure distribution
  /-- Source normalization of expected loss to `[0,1]`. -/
  expectedLoss_mem_Icc : ∀ distribution outcome,
    0 ≤ expectedLoss distribution outcome ∧ expectedLoss distribution outcome ≤ 1
  /-- Finite convex mixing of the experts' randomized decisions. -/
  mixture : PMF Expert → (Expert → Distribution) → Distribution
  /-- The represented mixture is the source's weighted sum of expert laws. -/
  mixture_toMeasure : ∀ (allocation : PMF Expert) (expertDecision : Expert → Distribution),
    toMeasure (mixture allocation expertDecision) =
      ∑ expert, (allocation expert) • toMeasure (expertDecision expert)
  /-- Linearity of expected loss under the finite mixture. -/
  mixture_expectedLoss : ∀ (allocation : PMF Expert) (expertDecision : Expert → Distribution)
    (outcome : Outcome),
      expectedLoss (mixture allocation expertDecision) outcome =
        pmfExp allocation (fun expert => expectedLoss (expertDecision expert) outcome)
```

Direct retained dependencies

- AppliedModelingLib.pmfExp

Recorded source-to-library screening **Verdict:** matches_approved_corrected_target

Reason: Compared theorem5_decision_prediction_general's Section 3 setup, Lambda(D,omega)=E_D[lambda], and D=sum_i p_i E_i directly with all displayed fields. Distributions represent actual probability measures on the unrestricted measurable decision space; realized losses lie in [0,1], expectedLoss is their integral, and mixture_toMeasure is precisely the weighted sum of expert laws. The displayed finite-expectation dependency gives the same

mixture-loss identity. Probability, measurability, and integral bounds express the source's defined expected losses, while injectivity permits canonical distribution representations. Theorem 5 supplies a finite expert family, uniform initial weights, and the positive loss-bound/N>=2 domain fixed by FS97-THEOREM5-TUNING-DOMAIN-02, without requiring finite decision or outcome spaces. The comparison is to the displayed approved corrected target bound to FS97-THEOREM5-TUNING-DOMAIN-02; archival equivalence is not claimed.

Reviewer check: ☐ Matches formalized target

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

AppliedModelingLib.Learning.Online.GeneralDecisionHedgeRound

Source locator: cited publication, lines 625-645

Verbatim paper-source connection

Theorem 5. For any loss function λ, for any set of experts, and for
any sequence of outcomes, the expected loss of $\operatorname{Hedge}(\beta)$
if used as described above is at most
$$
\sum_{t=1}^T\Lambda(\mathcal D^t,\omega^t)
\leq \min_i\sum_{t=1}^T\Lambda(\mathcal E_i^t,\omega^t)
+\sqrt{2\widetilde L\ln N}+\ln N,
$$
where $\widetilde L\leq T$ is an assumed bound on the expected loss of the
best expert, and $\beta=g(\widetilde L/\ln N)$.

[Next verbatim source excerpt]

O(- (ln N)[U+0012 control character]T). However, if L
[U+0019 control character] is close to zero, then the rate
of convergence will be much faster, roughly, O((ln N)[U+0012 control character]T).
Lemma 4 can also be applied to the other bounds given in
Theorem 2 to obtain analogous results.
The bound given in Eq. (11) can be improved in special
cases in which the loss is a function of a prediction and an
outcome and this function is of a special form (see Example 4
below). However, for the general case, one cannot improve
the square-root term - 2L
[U+0019 control character] ln N, by more than a constant
factor. This is a corollary of the lower bound given by Cesa-
Bianchi et al. ([4], Theorem 7) who analyze an on-line
prediction problem that can be seen as a special case of the
on-line allocation model.
3. APPLICATIONS
The framework described up to this point is quite general
and can be applied in a wide variety of learning problems.
Consider the following set-up used by Chung [5]. We are
given a decision space 2, a space of outcomes 0, and a
bounded loss function *: 2_0 [U+0014 control character] [0, 1]. (Actually, our
results require only that * be bounded, but, by rescaling, we
can assume that its range is [0, 1].) At every time step t, the
learning algorithm selects a decision $t
2, receives an out-
come |t
0, and suffers loss *($t
, |t
). More generally, we
may allow the learner to select a distribution Dt
over the
space of decisions, in which case it suffers the expected loss
of a decision randomly selected according to Dt
; that is, its
expected loss is 4(Dt
, |t
) where
4(D, |)=E$t D[*($, |)].
To decide on distribution Dt
, we assume that the learner
has access to a set of N experts. At every time step t, expert
i produces its own distribution Et
i on 2, and suffers loss
4(Et
i , |t
).
The goal of the learner is to combine the distributions
produced by the experts so as to suffer expected loss ``not
much worse'' than that of the best expert.
The results of Section 2 provide a method for solving this
problem. Specifically, we run algorithm Hedge(;), treating
each expert as a strategy. At every time step, Hedge(;)
produces a distribution pt
on the set of experts which is used
to construct the mixture distribution
Dt
= :
N
i=1
pt
i Et
i .
For any outcome |t
, the loss suffered by Hedge(;) will
then be
4(Dt
, |t
)= :
N
i=1
pt
i 4(Et
i , |t
).
Thus, if we define lt
i =4(Et

i , |t
) then the loss suffered by the
learner is pt
} lt
, i.e., exactly the mixture loss that was
analyzed in Section 2.
Hence, the bounds of Section 2 can be applied to our
current framework. For instance, applying Eq. (11), we
obtain the following:

Formalized review target The archival source above is not asserted equivalent to this formalized target.

For a finite expert family of size N >= 2, a positive best-expert loss bound L-tilde no larger than the horizon, and the general decision-prediction mixture model, Hedge with beta = g(L-tilde/log N) has cumulative expected loss at most the best expert loss plus sqrt(2 L-tilde log N) + log N.

Archival source anchor: cited publication, lines 143-152

Lean declaration type (metadata)

```
field expertDecision:
{Expert : Type u_1} →
  {Decision : Type u_2} →
    {Outcome : Type u_3} →
      [inst : Fintype Expert] →
        [inst_1 : DecidableEq Expert] →
          [inst_2 : MeasurableSpace Decision] →
            {game : AppliedModelingLib.Learning.Online.GeneralDecisionHedgeGame Expert Decision Outcome} →
              AppliedModelingLib.Learning.Online.GeneralDecisionHedgeRound game → Expert → game.Distribution

field outcome:
{Expert : Type u_1} →
  {Decision : Type u_2} →
    {Outcome : Type u_3} →
      [inst : Fintype Expert] →
        [inst_1 : DecidableEq Expert] →
          [inst_2 : MeasurableSpace Decision] →
            {game : AppliedModelingLib.Learning.Online.GeneralDecisionHedgeGame Expert Decision Outcome} →
              AppliedModelingLib.Learning.Online.GeneralDecisionHedgeRound game → Outcome
```

Exact Lean library declaration

```
/-- One source Theorem-5 round in an abstract randomized decision game. -/
structure GeneralDecisionHedgeRound
    {Expert Decision Outcome : Type*} [Fintype Expert] [DecidableEq Expert]
    [MeasurableSpace Decision]
    (game : GeneralDecisionHedgeGame Expert Decision Outcome) where
  expertDecision : Expert → game.Distribution
  outcome : Outcome
```

Direct retained dependencies

- AppliedModelingLib.Learning.Online.GeneralDecisionHedgeGame

Recorded source-to-library screening **Verdict:** matches_approved_corrected_target
Reason: Compared theorem5_decision_prediction_general's per-time expert distributions E_i^t and outcome omega^t with the displayed expertDecision and outcome fields. Each round contains exactly one law for each expert and one common outcome; inducedLoss evaluates that expert law's expected source loss and mixedDecision forms the source's weighted mixture. The displayed game fields bind both operations to probability measures and the bounded source loss. Theorem 5 quantifies over arbitrary lists of these rounds, so the representation imposes no stationarity, independence, or restriction to a finite decision/outcome space, and permits the source's arbitrary realized sequence. The comparison is to the displayed approved corrected target bound to FS97-THEOREM5-TUNING-DOMAIN-02; archival equivalence is not claimed.
Reviewer check: ☐ Matches formalized target
If left unchecked, explain the discrepancy or uncertainty below.
Reviewer annotation

AppliedModelingLib.Learning.Online.GeneralDecisionHedgeRound.hedgeCumulativeDecisionLoss

Source locator: cited publication, lines 625-645

Verbatim paper-source connection

Theorem 5. For any loss function λ, for any set of experts, and for
any sequence of outcomes, the expected loss of $\operatorname{Hedge}(\beta)$
if used as described above is at most
$$
\sum_{t=1}^T\Lambda(\mathcal D^t,\omega^t)
\leq \min_i\sum_{t=1}^T\Lambda(\mathcal E_i^t,\omega^t)
+\sqrt{2\widetilde L\ln N}+\ln N,
$$
where $\widetilde L\leq T$ is an assumed bound on the expected loss of the
best expert, and $\beta=g(\widetilde L/\ln N)$.

[Next verbatim source excerpt]

O(- (ln N)[U+0012 control character]T). However, if L
[U+0019 control character] is close to zero, then the rate
of convergence will be much faster, roughly, O((ln N)[U+0012 control character]T).
Lemma 4 can also be applied to the other bounds given in
Theorem 2 to obtain analogous results.
The bound given in Eq. (11) can be improved in special
cases in which the loss is a function of a prediction and an
outcome and this function is of a special form (see Example 4
below). However, for the general case, one cannot improve
the square-root term - 2L
[U+0019 control character] ln N, by more than a constant
factor. This is a corollary of the lower bound given by Cesa-
Bianchi et al. ([4], Theorem 7) who analyze an on-line
prediction problem that can be seen as a special case of the
on-line allocation model.
3. APPLICATIONS
The framework described up to this point is quite general
and can be applied in a wide variety of learning problems.
Consider the following set-up used by Chung [5]. We are
given a decision space 2, a space of outcomes 0, and a
bounded loss function *: 2_0 [U+0014 control character] [0, 1]. (Actually, our
results require only that * be bounded, but, by rescaling, we
can assume that its range is [0, 1].) At every time step t, the
learning algorithm selects a decision $t
2, receives an out-
come |t
0, and suffers loss *($t
, |t
). More generally, we
may allow the learner to select a distribution Dt
over the
space of decisions, in which case it suffers the expected loss
of a decision randomly selected according to Dt
; that is, its
expected loss is 4(Dt
, |t
) where
4(D, |)=E$t D[*($, |)].
To decide on distribution Dt
, we assume that the learner
has access to a set of N experts. At every time step t, expert
i produces its own distribution Et
i on 2, and suffers loss
4(Et
i , |t
).
The goal of the learner is to combine the distributions
produced by the experts so as to suffer expected loss ``not
much worse'' than that of the best expert.
The results of Section 2 provide a method for solving this
problem. Specifically, we run algorithm Hedge(;), treating
each expert as a strategy. At every time step, Hedge(;)
produces a distribution pt
on the set of experts which is used
to construct the mixture distribution
Dt
= :
N
i=1
pt
i Et
i .
For any outcome |t
, the loss suffered by Hedge(;) will
then be
4(Dt
, |t
)= :
N
i=1
pt
i 4(Et
i , |t
).
Thus, if we define lt
i =4(Et

i , |t
) then the loss suffered by the
learner is pt
} lt
, i.e., exactly the mixture loss that was
analyzed in Section 2.
Hence, the bounds of Section 2 can be applied to our
current framework. For instance, applying Eq. (11), we
obtain the following:

Formalized review target The archival source above is not asserted equivalent to this formalized target.

For a finite expert family of size N >= 2, a positive best-expert loss bound L-tilde no larger than the horizon, and the general decision-prediction mixture model, Hedge with beta = g(L-tilde/log N) has cumulative expected loss at most the best expert loss plus sqrt(2 L-tilde log N) + log N.

Archival source anchor: cited publication, lines 143-152

Lean-expanded library semantic target

```
type:
{Expert : Type u_1} →
  {Decision : Type u_2} →
    {Outcome : Type u_3} →
      [inst : Fintype Expert] →
        [inst_1 : DecidableEq Expert] →
          [inst_2 : MeasurableSpace Decision] →
            [Nonempty Expert] →
              {game : AppliedModelingLib.Learning.Online.GeneralDecisionHedgeGame Expert Decision Outcome} →
                (discount : ℝ) →
                  0 < discount →
                    AppliedModelingLib.Learning.Online.HedgeWeightState Expert →
                      List (AppliedModelingLib.Learning.Online.GeneralDecisionHedgeRound game) → ℝ

value:
fun {Expert : Type u_1} {Decision : Type u_2} {Outcome : Type u_3} [Fintype Expert] [DecidableEq Expert]
    [MeasurableSpace Decision] [Nonempty Expert]
    {game : AppliedModelingLib.Learning.Online.GeneralDecisionHedgeGame Expert Decision Outcome} (discount : ℝ)
    (hdiscount : 0 < discount) (a : AppliedModelingLib.Learning.Online.HedgeWeightState Expert)
    (a_1 : List (AppliedModelingLib.Learning.Online.GeneralDecisionHedgeRound game)) =>
  List.brecOn (motive := fun (x : List (AppliedModelingLib.Learning.Online.GeneralDecisionHedgeRound game)) =>
    AppliedModelingLib.Learning.Online.HedgeWeightState Expert → ℝ) a_1
    (AppliedModelingLib.Learning.Online.GeneralDecisionHedgeRound.hedgeCumulativeDecisionLoss._f discount hdiscount) a
```

Exact Lean library declaration

```
/--
Execute the source decision-mixture prescription through a finite sequence of
abstract decision rounds.
-/
noncomputable def hedgeCumulativeDecisionLoss
    {Expert Decision Outcome : Type*} [Fintype Expert] [DecidableEq Expert]
    [MeasurableSpace Decision]
    [Nonempty Expert] {game : GeneralDecisionHedgeGame Expert Decision Outcome}
    (discount : ℝ) (hdiscount : 0 < discount) :
    HedgeWeightState Expert → List (GeneralDecisionHedgeRound game) → ℝ
  | _, [] => 0
  | state, round :: rounds =>
      game.expectedLoss (round.mixedDecision state.policy) round.outcome +
        hedgeCumulativeDecisionLoss discount hdiscount
          (hedgeWeightStateStep state round.inducedLoss discount hdiscount) rounds
```

Direct retained dependencies

- AppliedModelingLib.Learning.Online.GeneralDecisionHedgeGame
- AppliedModelingLib.Learning.Online.GeneralDecisionHedgeRound
- AppliedModelingLib.Learning.Online.GeneralDecisionHedgeRound.inducedLoss
- AppliedModelingLib.Learning.Online.GeneralDecisionHedgeRound.mixedDecision
- AppliedModelingLib.Learning.Online.HedgeWeightState
- AppliedModelingLib.Learning.Online.HedgeWeightState.policy
- AppliedModelingLib.Learning.Online.hedgeWeightStateStep

Recorded source-to-library screening **Verdict:** matches_approved_corrected_target
Reason: Compared theorem5_decision_prediction_general's sum_t Lambda(D^t,omega^t) and D^t=sum_i p_i^t E_i^t with the displayed recursion and all of its prerequisites. The empty list contributes zero; each round adds the expected loss of the current normalized-weight mixture, then updates each expert weight by beta raised to that expert's expected loss before processing the remaining rounds. finiteWeightedPMF and pmfExp normalize and average exactly as printed. The displayed Theorem 5 use fixes uniform initial weights and its tuned positive discount on the FS97-THEOREM5-TUNING-DOMAIN-02 domain; the reusable function's broader positive-discount domain does not change that use. The

comparison is to the displayed approved corrected target bound to FS97-THEOREM5-TUNING-DOMAIN-02; archival equivalence is not claimed.

Reviewer check: ☐ Matches formalized target

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

AppliedModelingLib.Learning.Online.adaBoostDiscountPowerFactor

Source locator: cited publication, lines 1486-1510

Verbatim paper-source connection

Theorem 9. Let $\epsilon_1,\ldots,\epsilon_T$ be as in Theorem 6, and let
$r(x_i)$ be as defined above. Let the modified final hypothesis be defined by
$h_f(x)=F(r(x))$, where for $r\in[0,1]$,
$$
F(1-r)=1-F(r),\qquad
F(r)\leq\frac12\left(\prod_{t=1}^T\beta_t\right)^{1/2-r}.
$$
Then the error ϵ of h_f is bounded above by
$$
\epsilon\leq 2^{T-1}\prod_{t=1}^T\sqrt{\epsilon_t(1-\epsilon_t)}.
$$

[Next verbatim source excerpt]

D(i)=1[U+0012 control character]N. The algorithm maintains a set of weights wt
over
the training examples. On iteration t a distribution pt
is
computed by normalizing these weights. This distribution is
fed to the weak learner WeakLearn which generates a
hypothesis ht that (we hope) has small error with respect to
the distribution.1
Using the new hypothesis ht , the boosting
125
A DECISION THEORETIC GENERALIZATION
1
Some learning algorithms can be generalized to use a given distribution
directly. For instance, gradient based algorithms can use the probability
associated with each example to scale the update step size which is based
on the example. If the algorithm cannot be generalized in this way, the
training sample can be re-sampled to generate a new set of training exam-
ples that is distributed according to the given distribution. The computa-
tion required to generate each re-sampled example takes O(log N) time.

[form-feed in source extraction]
File: 571J 150408 . By:CV . Date:28:07:01 . Time:05:54 LOP8M. V8.0. Page 01:01
Codes: 6184 Signs: 4760 . Length: 56 pic 0 pts, 236 mm
Algorithm AdaBoost
Input: sequence of N labeled examples ((x1 , y1), ..., (xN , yN))
distribution D over the N examples
weak learning algorithm WeakLearn
integer T specifying number of iterations
Initialize the weight vector: w1
i =D(i) for i=1, ..., N.
Do for t=1, 2, ..., T
1. Set
pt
=
wt
[U+001D control character]N
i=1 wt
i
2. Call WeakLearn, providing it with the distribution pt
; get back a
hypothesis ht : X[U+0014 control character] [0, 1].
3. Calculate the error of ht: =t=[U+001D control character]N
i=1 pt
i |ht(xi)& yi |.
4. Set ;t==t [U+0012 control character](1&=t).
5. Set the new weights vector to be
wt+1
i =wt
i ;1&|ht(xi)& yi |
t
Output the hypothesis
hf (x)=
{
1 if [U+001D control character]T
t=1 (log 1[U+0012 control character];t) ht(x)[U+001E control character]1
2 [U+001D control character]T
t=1 log 1[U+0012 control character];t
0 otherwise.
FIG. 2. The adaptive boosting algorithm.
algorithm generates the next weight vector wt+1
, and the
process repeats. After T such iterations, the final hypothesis
hf is output. The hypothesis hf combines the outputs of the
T weak hypotheses using a weighted majority vote.
We call the algorithm AdaBoost because, unlike previous
algorithms, it adjusts adaptively to the errors of the weak
hypotheses returned by WeakLearn. If WeakLearn is a PAC
weak learning algorithm in the sense defined above, then
=t[U+001D control character]1[U+0012 control character]2&# for all t (assuming the examples have been
generated appropriately with yi=c(xi) for some c # C).
However, such a bound on the error need not be known
ahead of time. Our results hold for any =t # [0, 1], and
depend only on the performance of the weak learner on
those distributions that are actually generated during the
boosting process.

The parameter ;t is chosen as a function of =t and is used for updating the weight vector. The update rule reduces the probability assigned to those examples on which the hypothesis makes a good prediction and increases the probability of the examples on which the prediction is poor.2
Note that AdaBoost, unlike boost-by-majority, combines the weak hypotheses by summing their probabilistic predictions. Drucker, Schapire and Simard [9], in experiments they performed using boosting to improve the performance of a real-valued neural network, observed that summing the outcomes of the networks and then selecting the best prediction performs better than selecting the best prediction of each network and then combining them with a majority rule. It is interesting that the new boosting algorithm's final hypothesis uses the same combination rule that was observed to be better in practice, but which previously lacked theoretical justification.
Since it was first introduced, several successful experiments have been conducted using AdaBoost, including work by the authors [12], Drucker and Cortes [8], Jackson and Craven [16], Quinlan [21], and Breiman [3].
4.2. Analysis
Comparing Figs. 1 and 2, there is an obvious similarity between the algorithms Hedge(;) and AdaBoost. This similarity reflects a surprising ``dual'' relationship between the on-line allocation model and the problem of boosting. Put another way, there is a direct mapping or reduction of the boosting problem to the on-line allocation problem. In

[Next verbatim source excerpt]

Theorem 6. Suppose the weak learning algorithm WeakLearn, when called by AdaBoost, generates hypotheses with errors =1 , ..., =T (as defined in Step 3 of Fig. 2.) Then the error ==PritD[hf (xi){ yi] of the final hypothesis hf output by AdaBoost is bounded above by
=[U+001D control character]2T
`
T
t=1
- =t(1&=t). (14)

[Next verbatim source excerpt]

optimal decision rule says that we should predict the label with the highest likelihood, given the hypothesis values, i.e., we should predict 1 if
Pr[y=1 | h1(x), ..., hT (x)]>Pr[y=0 | h1(x), ..., hT (x)],
and otherwise we should predict 0.
This rule is especially easy to compute if we assume that the errors of the different hypotheses are independent of one another and of the target concept, that is, if we assume that the event ht(x){ y is conditionally independent of the actual label y and the predictions of all the other hypotheses h1(x), ..., ht&1(x), ht+1(x), ..., hT (x). In this case, by applying Bayes rule, we can rewrite the Bayes optimal decision rule in a particularly simple form in which we predict
1 if
Pr[y=1] `
t: ht(x)=0
=t `
t: ht(x)=1
(1&=t)
>Pr[y=0] `
t: ht(x)=0
(1&=t) `
t: ht(x)=1
=t ,
and 0 otherwise. Here =t=Pr[ht(x){ y]. We add to the set of hypotheses the trivial hypothesis h0 which always predicts the value 1. We can then replace Pr[y=0] by =0 . Taking the logarithm of both sides in this inequality and rearranging the terms, we find that the Bayes optimal decision rule is identical to the combination rule that is generated by AdaBoost.
If the errors of the different hypotheses are dependent, then the Bayes optimal decision rule becomes much more complicated. However, in practice, it is common to use the simple rule described above even when there is no justification for assuming independence. (This is sometimes called ``naive Bayes.'') An interesting and more principled alternative to this practice would be to use the algorithm AdaBoost to find a combination rule which, by Theorem 6, has a guaranteed non-trivial accuracy.
129
A DECISION THEORETIC GENERALIZATION

[form-feed in source extraction]
File: 571J 150412 . By:CV . Date:28:07:01 . Time:05:54 LOP8M. V8.0. Page 01:01
Codes: 5125 Signs: 3769 . Length: 56 pic 0 pts, 236 mm
4.5. Improving the Error Bound
We show in this section how the bound given in Theorem 6 can be improved by a factor of two. The main idea of this improvement is to replace the ``hard'' [0, 1]-valued decision used by hf by a ``soft'' threshold.
To be more precise, let

r(x)=
[U+001D control character]T
t=1 (log 1
;t
) ht(x)
[U+001D control character]T
t=1 log 1
;t
be a weighted average of the weak hypotheses ht . We will
here consider final hypotheses of the form hf (x)=F(r(x))
where F: [0, 1] [U+0014 control character] [0, 1]. For the version of AdaBoost given
in Fig. 2, F(r) is the hard threshold that equals 1 if r[U+001E control character]1[U+0012 control character]2
and 0 otherwise. In this section, we will instead use soft
threshold functions that take values in [0, 1]. As mentioned
above, when hf (x) # [0, 1], we can interpret hf as a
randomized hypothesis and hf (x) as the probability of
predicting 1. Then the error EitD[|hf (xi)& yi |] is simply
the probability of an incorrect prediction.

Lean-expanded library semantic target

```
type:
{Training : Type u_1} →
  [inst : Fintype Training] →
    [inst_1 : Nonempty Training] →
      (state : AppliedModelingLib.Learning.Online.HedgeWeightState Training) →
        (label : Training → ℝ) →
          (hypotheses : List (Training → ℝ)) →
            AppliedModelingLib.Learning.Online.AdaBoostAdmissible state label hypotheses → ℝ → ℝ

value:
fun {Training : Type u_1} [Fintype Training] [Nonempty Training]
    (state : AppliedModelingLib.Learning.Online.HedgeWeightState Training) (label : Training → ℝ)
    (hypotheses : List (Training → ℝ))
    (a : AppliedModelingLib.Learning.Online.AdaBoostAdmissible state label hypotheses) (a_1 : ℝ) =>
  List.brecOn (motive := fun (x : List (Training → ℝ)) =>
    (state : AppliedModelingLib.Learning.Online.HedgeWeightState Training) →
      AppliedModelingLib.Learning.Online.AdaBoostAdmissible state label x → ℝ → ℝ)
    hypotheses (AppliedModelingLib.Learning.Online.adaBoostDiscountPowerFactor._f label) state a a_1
```

Exact Lean library declaration

```
/-- The source product `∏_t β_t^q` for a common real exponent `q`. -/
noncomputable def adaBoostDiscountPowerFactor
    {Training : Type*} [Fintype Training] [Nonempty Training]
    (state : HedgeWeightState Training) (label : Training → ℝ) :
    (hypotheses : List (Training → ℝ)) →
      AdaBoostAdmissible state label hypotheses → ℝ → ℝ
  | [], _, _ => 1
  | hypothesis :: remaining, hvalid, exponent =>
      adaBoostDiscount (adaBoostError state label hypothesis) ^ exponent *
        adaBoostDiscountPowerFactor
          (adaBoostStep state label hypothesis (Classical.choose hvalid)) label remaining
          (Classical.choose_spec hvalid) exponent
```

Direct retained dependencies

- AppliedModelingLib.Learning.Online.AdaBoostAdmissible
- AppliedModelingLib.Learning.Online.HedgeWeightState
- AppliedModelingLib.Learning.Online.adaBoostDiscount
- AppliedModelingLib.Learning.Online.adaBoostError
- AppliedModelingLib.Learning.Online.adaBoostStep

Recorded source-to-library screening **Verdict:** matches

Reason: The recursion multiplies the source factors beta_t raised to a common real exponent. The soft-AdaBoost route applies it at the exact source exponent 1/2-r(x).

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

AppliedModelingLib.Learning.Online.adaBoostErrors

Source locator: cited publication, lines 1486-1510

Verbatim paper-source connection

Theorem 9. Let $\epsilon_1,\ldots,\epsilon_T$ be as in Theorem 6, and let
$r(x_i)$ be as defined above. Let the modified final hypothesis be defined by
$h_f(x)=F(r(x))$, where for $r\in[0,1]$,
$$
F(1-r)=1-F(r),\qquad
F(r)\leq\frac12\left(\prod_{t=1}^T\beta_t\right)^{1/2-r}.
$$
Then the error ϵ of h_f is bounded above by
$$
\epsilon\leq 2^{T-1}\prod_{t=1}^T\sqrt{\epsilon_t(1-\epsilon_t)}.
$$

[Next verbatim source excerpt]

D(i)=1[U+0012 control character]N. The algorithm maintains a set of weights wt
over
the training examples. On iteration t a distribution pt
is
computed by normalizing these weights. This distribution is
fed to the weak learner WeakLearn which generates a
hypothesis ht that (we hope) has small error with respect to
the distribution.1
Using the new hypothesis ht , the boosting
125
A DECISION THEORETIC GENERALIZATION
1
Some learning algorithms can be generalized to use a given distribution
directly. For instance, gradient based algorithms can use the probability
associated with each example to scale the update step size which is based
on the example. If the algorithm cannot be generalized in this way, the
training sample can be re-sampled to generate a new set of training exam-
ples that is distributed according to the given distribution. The computa-
tion required to generate each re-sampled example takes O(log N) time.

[form-feed in source extraction]
File: 571J 150408 . By:CV . Date:28:07:01 . Time:05:54 LOP8M. V8.0. Page 01:01
Codes: 6184 Signs: 4760 . Length: 56 pic 0 pts, 236 mm
Algorithm AdaBoost
Input: sequence of N labeled examples ((x1 , y1), ..., (xN , yN))
distribution D over the N examples
weak learning algorithm WeakLearn
integer T specifying number of iterations
Initialize the weight vector: w1
i =D(i) for i=1, ..., N.
Do for t=1, 2, ..., T
1. Set
pt
=
wt
[U+001D control character]N
i=1 wt
i
2. Call WeakLearn, providing it with the distribution pt
; get back a
hypothesis ht : X[U+0014 control character] [0, 1].
3. Calculate the error of ht: =t=[U+001D control character]N
i=1 pt
i |ht(xi)& yi |.
4. Set ;t==t [U+0012 control character](1&=t).
5. Set the new weights vector to be
wt+1
i =wt
i ;1&|ht(xi)& yi |
t
Output the hypothesis
hf (x)=
{
1 if [U+001D control character]T
t=1 (log 1[U+0012 control character];t) ht(x)[U+001E control character]1
2 [U+001D control character]T
t=1 log 1[U+0012 control character];t
0 otherwise.
FIG. 2. The adaptive boosting algorithm.
algorithm generates the next weight vector wt+1
, and the
process repeats. After T such iterations, the final hypothesis
hf is output. The hypothesis hf combines the outputs of the
T weak hypotheses using a weighted majority vote.
We call the algorithm AdaBoost because, unlike previous
algorithms, it adjusts adaptively to the errors of the weak
hypotheses returned by WeakLearn. If WeakLearn is a PAC
weak learning algorithm in the sense defined above, then
=t[U+001D control character]1[U+0012 control character]2&# for all t (assuming the examples have been
generated appropriately with yi=c(xi) for some c # C).
However, such a bound on the error need not be known
ahead of time. Our results hold for any =t # [0, 1], and
depend only on the performance of the weak learner on
those distributions that are actually generated during the
boosting process.

The parameter ;t is chosen as a function of =t and is used for updating the weight vector. The update rule reduces the probability assigned to those examples on which the hypothesis makes a good prediction and increases the probability of the examples on which the prediction is poor.2 Note that AdaBoost, unlike boost-by-majority, combines the weak hypotheses by summing their probabilistic predictions. Drucker, Schapire and Simard [9], in experiments they performed using boosting to improve the performance of a real-valued neural network, observed that summing the outcomes of the networks and then selecting the best prediction performs better than selecting the best prediction of each network and then combining them with a majority rule. It is interesting that the new boosting algorithm's final hypothesis uses the same combination rule that was observed to be better in practice, but which previously lacked theoretical justification.
Since it was first introduced, several successful experiments have been conducted using AdaBoost, including work by the authors [12], Drucker and Cortes [8], Jackson and Craven [16], Quinlan [21], and Breiman [3].
4.2. Analysis
Comparing Figs. 1 and 2, there is an obvious similarity between the algorithms Hedge(;) and AdaBoost. This similarity reflects a surprising ``dual'' relationship between the on-line allocation model and the problem of boosting. Put another way, there is a direct mapping or reduction of the boosting problem to the on-line allocation problem. In

[Next verbatim source excerpt]

Theorem 6. Suppose the weak learning algorithm WeakLearn, when called by AdaBoost, generates hypotheses with errors =1 , ..., =T (as defined in Step 3 of Fig. 2.) Then the error ==PritD[hf (xi){ yi] of the final hypothesis hf output by AdaBoost is bounded above by
=[U+001D control character]2T
`
T
t=1
- =t(1&=t). (14)

[Next verbatim source excerpt]

optimal decision rule says that we should predict the label with the highest likelihood, given the hypothesis values, i.e., we should predict 1 if
Pr[y=1 | h1(x), ..., hT (x)]>Pr[y=0 | h1(x), ..., hT (x)],
and otherwise we should predict 0.
This rule is especially easy to compute if we assume that the errors of the different hypotheses are independent of one another and of the target concept, that is, if we assume that the event ht(x){ y is conditionally independent of the actual label y and the predictions of all the other hypotheses h1(x), ..., ht&1(x), ht+1(x), ..., hT (x). In this case, by applying Bayes rule, we can rewrite the Bayes optimal decision rule in a particularly simple form in which we predict
1 if
Pr[y=1] `
t: ht(x)=0
=t `
t: ht(x)=1
(1&=t)
>Pr[y=0] `
t: ht(x)=0
(1&=t) `
t: ht(x)=1
=t ,
and 0 otherwise. Here =t=Pr[ht(x){ y]. We add to the set of hypotheses the trivial hypothesis h0 which always predicts the value 1. We can then replace Pr[y=0] by =0 . Taking the logarithm of both sides in this inequality and rearranging the terms, we find that the Bayes optimal decision rule is identical to the combination rule that is generated by AdaBoost.
If the errors of the different hypotheses are dependent, then the Bayes optimal decision rule becomes much more complicated. However, in practice, it is common to use the simple rule described above even when there is no justification for assuming independence. (This is sometimes called ``naive Bayes.'') An interesting and more principled alternative to this practice would be to use the algorithm AdaBoost to find a combination rule which, by Theorem 6, has a guaranteed non-trivial accuracy.
129
A DECISION THEORETIC GENERALIZATION

[form-feed in source extraction]
File: 571J 150412 . By:CV . Date:28:07:01 . Time:05:54 LOP8M. V8.0. Page 01:01
Codes: 5125 Signs: 3769 . Length: 56 pic 0 pts, 236 mm
4.5. Improving the Error Bound
We show in this section how the bound given in Theorem 6 can be improved by a factor of two. The main idea of this improvement is to replace the ``hard'' [0, 1]-valued decision used by hf by a ``soft'' threshold.
To be more precise, let

r(x)=
[U+001D control character]T
t=1 (log 1
;t
) ht(x)
[U+001D control character]T
t=1 log 1
;t
be a weighted average of the weak hypotheses ht . We will
here consider final hypotheses of the form hf (x)=F(r(x))
where F: [0, 1] [U+0014 control character] [0, 1]. For the version of AdaBoost given
in Fig. 2, F(r) is the hard threshold that equals 1 if r[U+001E control character]1[U+0012 control character]2
and 0 otherwise. In this section, we will instead use soft
threshold functions that take values in [0, 1]. As mentioned
above, when hf (x) # [0, 1], we can interpret hf as a
randomized hypothesis and hf (x) as the probability of
predicting 1. Then the error EitD[|hf (xi)& yi |] is simply
the probability of an incorrect prediction.

Lean-expanded library semantic target

```
type:
{Training : Type u_1} →
  [inst : Fintype Training] →
    [inst_1 : Nonempty Training] →
      (state : AppliedModelingLib.Learning.Online.HedgeWeightState Training) →
        (label : Training → ℝ) →
          (hypotheses : List (Training → ℝ)) →
            AppliedModelingLib.Learning.Online.AdaBoostAdmissible state label hypotheses → List ℝ

value:
fun {Training : Type u_1} [Fintype Training] [Nonempty Training]
    (state : AppliedModelingLib.Learning.Online.HedgeWeightState Training) (label : Training → ℝ)
    (hypotheses : List (Training → ℝ))
    (a : AppliedModelingLib.Learning.Online.AdaBoostAdmissible state label hypotheses) =>
  List.brecOn (motive := fun (x : List (Training → ℝ)) =>
    (state : AppliedModelingLib.Learning.Online.HedgeWeightState Training) →
      AppliedModelingLib.Learning.Online.AdaBoostAdmissible state label x → List ℝ)
    hypotheses (AppliedModelingLib.Learning.Online.adaBoostErrors._f label) state a
```

Exact Lean library declaration

```
/-- Extract the Step-3 errors encountered along a finite certified AdaBoost run. -/
noncomputable def adaBoostErrors
    {Training : Type*} [Fintype Training] [Nonempty Training]
    (state : HedgeWeightState Training) (label : Training → ℝ) :
    (hypotheses : List (Training → ℝ)) →
      AdaBoostAdmissible state label hypotheses → List ℝ
  | [], _ => []
  | hypothesis :: remaining, hvalid =>
      adaBoostError state label hypothesis ::
        adaBoostErrors
          (adaBoostStep state label hypothesis (Classical.choose hvalid)) label remaining
          (Classical.choose_spec hvalid)
```

Direct retained dependencies

- AppliedModelingLib.Learning.Online.AdaBoostAdmissible
- AppliedModelingLib.Learning.Online.HedgeWeightState
- AppliedModelingLib.Learning.Online.adaBoostError
- AppliedModelingLib.Learning.Online.adaBoostStep

Recorded source-to-library screening **Verdict:** matches

Reason: The list records at each recursive state the normalized weighted absolute loss of the selected hypothesis, which is exactly the epsilon_t sequence used in the source AdaBoost and soft-AdaBoost bounds.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

AppliedModelingLib.Learning.Online.adaBoostM1LabelScore

Source locator: cited publication, lines 1735-1745

Verbatim paper-source connection

Theorem 10. Suppose WeakLearn, when called by AdaBoost.M1, generates
hypotheses with errors $\epsilon_1,\ldots,\epsilon_T$, where ϵ_t is
as defined in Fig. 3. Assume each $\epsilon_t\leq1/2$. Then the error
$\epsilon=\Pr_{i\sim D}[h_f(x_i)\neq y_i]$ is bounded above by
$$
\epsilon\leq2^T\prod_{t=1}^T\sqrt{\epsilon_t(1-\epsilon_t)}.
$$

[Next verbatim source excerpt]

a given instance x, now outputs the label y that maximizes
the sum of the weights of the weak hypotheses predicting
that label.
In the case of binary classification (k=2), a weak
hypothesis h with error significantly larger than 1[U+0012 control character]2 is of
equal value to one with error significantly less than 1[U+0012 control character]2 since
h can be replaced by 1&h. However, for k>2, a hypothesis
ht with error =t[U+001E control character]1[U+0012 control character]2 is useless to the boosting algorithm. If
Algorithm AdaBoost.M1
Input: sequence of N examples ((x1 , y1)..., (xN , yN)) with labels
yi # Y=[1, ..., k]
distribution D over the N examples
weak learning algorithm WeakLearn
integer T specifying number of iterations
Initialize the weight vector: w1
i =D(i) for i=1, ..., N.
Do for t=1, 2, ..., T
1. Set
pt
=
wt
[U+001D control character]N
i=1 wt
i
2. Call WeakLearn, providing it with the distribution pt
; get back a
hypothesis ht : X [U+0014 control character] Y.
3. Calculate the error of ht: =t=[U+001D control character]N
i=1 pt
i [U+001A control character]ht(xi){ yi[U+0019 control character].
If =t>1[U+0012 control character]2, then set T=t&1 and abort loop.
4. Set ;t==t [U+0012 control character](1&=t).
5. Set the new weights vector to be
wt+1
i =wt
i ;1&[U+001A control character]ht(xi){y
% i[U+0019 control character]
t
Output the hypothesis
hf (x)=arg max
y # Y
:
T
t=1
\log
1
;t+[U+001A control character]ht(x)= y[U+0019 control character].
FIG. 3. A first multi-class extension of AdaBoost.
131
A DECISION THEORETIC GENERALIZATION

[form-feed in source extraction]
File: 571J 150414 . By:CV . Date:28:07:01 . Time:05:54 LOP8M. V8.0. Page 01:01
Codes: 5566 Signs: 4204 . Length: 56 pic 0 pts, 236 mm
such a weak hypothesis is returned by the weak learner, our
algorithm simply halts, using only the weak hypotheses that
were already computed.

Formalized review target

The archival source above is not asserted equivalent to this formalized target.

For the finite Figure-3 AdaBoost.M1 execution with every observed mistake error positive and at most one half, the final argmax-vote classifier has training error at most 2^T times the product of sqrt(epsilon_t(1-epsilon_t)).

Archival source anchor: cited publication, lines 242-261

Lean-expanded library semantic target

```
type:
{Training : Type u_1} →
  {Label : Type u_2} →
    [inst : Fintype Training] →
      [inst_1 : Nonempty Training] →
        [inst_2 : DecidableEq Label] →
          (state : AppliedModelingLib.Learning.Online.HedgeWeightState Training) →
```

```
      (label : Training → Label) →
        (hypotheses : List (Training → Label)) →
          AppliedModelingLib.Learning.Online.AdaBoostM1Admissible state label hypotheses → Training → Label → ℝ

value:
fun {Training : Type u_1} {Label : Type u_2} [Fintype Training] [Nonempty Training] [DecidableEq Label]
    (state : AppliedModelingLib.Learning.Online.HedgeWeightState Training) (label : Training → Label)
    (hypotheses : List (Training → Label))
    (a : AppliedModelingLib.Learning.Online.AdaBoostM1Admissible state label hypotheses) (a_1 : Training)
    (a_2 : Label) =>
  List.brecOn (motive := fun (x : List (Training → Label)) =>
    (state : AppliedModelingLib.Learning.Online.HedgeWeightState Training) →
      AppliedModelingLib.Learning.Online.AdaBoostM1Admissible state label x → Training → Label → ℝ)
    hypotheses (AppliedModelingLib.Learning.Online.adaBoostM1LabelScore._f label) state a a_1 a_2
```

Exact Lean library declaration

```
/-- The per-label final score in Figure 3 of Freund--Schapire (1997). -/
noncomputable def adaBoostM1LabelScore
    {Training Label : Type*} [Fintype Training] [Nonempty Training] [DecidableEq Label]
    (state : HedgeWeightState Training) (label : Training → Label) :
    (hypotheses : List (Training → Label)) →
      AdaBoostM1Admissible state label hypotheses → Training → Label → ℝ
  | [], _, _, _ => 0
  | hypothesis :: remaining, hvalid, sample, candidate =>
      adaBoostVoteWeight
          (adaBoostError state (fun _ => 0) (adaBoostM1BinaryHypothesis label hypothesis)) *
        (if hypothesis sample = candidate then 1 else 0) +
      adaBoostM1LabelScore
        (adaBoostStep state (fun _ => 0) (adaBoostM1BinaryHypothesis label hypothesis)
          (Classical.choose hvalid)) label remaining (Classical.choose_spec hvalid) sample candidate
```

Direct retained dependencies

- AppliedModelingLib.Learning.Online.AdaBoostAdmissible
- AppliedModelingLib.Learning.Online.AdaBoostM1Admissible
- AppliedModelingLib.Learning.Online.HedgeWeightState
- AppliedModelingLib.Learning.Online.adaBoostError
- AppliedModelingLib.Learning.Online.adaBoostM1BinaryHypothesis
- AppliedModelingLib.Learning.Online.adaBoostStep
- AppliedModelingLib.Learning.Online.adaBoostVoteWeight

Recorded source-to-library screening

Verdict: matches_approved_corrected_target
Reason: Compared theorem10_adaboost_m1_error's Figure 3 output sum_t log(1/beta_t) [h_t(x)=y] with the full displayed score recursion and binary-reduction dependencies. The current error is the normalized mistake indicator, beta=epsilon/(1-epsilon), each score increment uses the same-round log(1/beta) times the candidate-label indicator, and the recursive state uses the printed correct-example discount. The empty score is zero. The displayed Theorem 10 use has finite nonempty labels, normalized initial weights, positive interior errors, and an explicit error<=1/2 condition, exactly the execution domain in FS97-ADABOOST-VARIANTS-ENDPOINT-DOMAIN-06. Its final-hypothesis dependency selects an argmax of these scores, including ties. The comparison is to the displayed approved corrected target bound to FS97-ADABOOST-VARIANTS-ENDPOINT-DOMAIN-06; archival equivalence is not claimed.
Reviewer check: ☐ Matches formalized target
If left unchecked, explain the discrepancy or uncertainty below.
Reviewer annotation

AppliedModelingLib.Learning.Online.adaBoostM2LabelScore

Source locator: cited publication, lines 2028-2037

Verbatim paper-source connection

Theorem 11. Suppose WeakLearn, when called by AdaBoost.M2, generates
hypotheses with pseudo-losses $\epsilon_1,\ldots,\epsilon_T$, where
ϵ_t is as defined in Fig. 4. Then the error
$\epsilon=\Pr_{i\sim D}[h_f(x_i)\neq y_i]$ is bounded above by
$$
\epsilon\leq(k-1)2^T\prod_{t=1}^T\sqrt{\epsilon_t(1-\epsilon_t)}.
$$

[Next verbatim source excerpt]

PritD[hf (xi){ yi][U+001D control character]PritD[h
[U+0019 control character] f (i)=1].
Since each AdaBoost instance has a 0-label, PritD[h
[U+0019 control character] f (i)
=1] is exactly the error of h
[U+0019 control character] f . Applying Theorem 6, we can
obtain a bound on this error, completing the proof. K
It is possible, for this version of the boosting algorithm, to
allow hypotheses which generate for each x, not only a
predicted class label h(x) # Y, but also a ``confidence''
}(x) # [0, 1]. The learner then suffers loss 1[U+0012 control character]2&}(x)[U+0012 control character]2 if its
prediction is correct and 1[U+0012 control character]2+}(x)[U+0012 control character]2 otherwise. (Details
omitted.)
5.2. Second Multi-class Extension
In this section we describe a second alternative extension
of AdaBoost to the case where the label space Y is finite. This
extension requires more elaborate communication between
the boosting algorithm and the weak learning algorithm.
The advantage of doing this is that it gives the weak learner
more flexibility in making its predictions. In particular, it
sometimes enables the weak learner to make useful con-
tributions to the accuracy of the final hypothesis even when
the weak hypothesis does not predict the correct label with
probability greater than 1[U+0012 control character]2.
As described above, the weak learner generates
hypotheses which have the form h: X_Y [U+0014 control character] [0, 1]. Roughly
speaking, h(x, y) measures the degree to which it is believed
that y is the correct label associated with instance x. If, for
a given x, h(x, y) attains the same value for all y then we say
that the hypothesis is uninformative on instance x. On the
other hand, any deviation from strict equality is potentially
informative, because it predicts some labels to be more
plausible than others. As will be seen, any such information
is potentially useful for the boosting algorithm.
Below, we formalize the goal of the weak learner by
defining a pseudo-loss which measures the goodness of the
weak hypotheses. To motivate our definition, we first
consider the following setup. For a fixed training example
(xi , yi), we use a given hypothesis h to answer k&1 binary
questions. For each of the incorrect labels y{ yi we ask the
question:
``Which is the label of xi : yi or y?''
In other words, we ask that the correct label yi be
discriminated from the incorrect label y.
Assume momentarily that h only takes values in [0, 1].
Then if h(xi , y)=0 and h(xi , yi)=1, we interpret h's
answer to the question above to be yi (since h deems yi to
be a plausible label for xi , but y is considered implausible).
Likewise, if h(xi , y)=1 and h(xi , yi)=0 then the answer is
y. If h(xi , y)=h(xi , yi), then one of the two answers is
chosen uniformly at random.
132 FREUND AND SCHAPIRE

[form-feed in source extraction]
File: 571J 150415 . By:CV . Date:28:07:01 . Time:05:54 LOP8M. V8.0. Page 01:01
Codes: 5466 Signs: 4208 . Length: 56 pic 0 pts, 236 mm
In the more general case that h takes values in [0, 1], we
interpret h(x, y) as a randomized decision for the procedure
above. That is, we first choose a random bit b(x, y) which
is 1 with probability h(x, y) and 0 otherwise. We then apply
the above procedure to the stochastically chosen binary
function b. The probability of choosing the incorrect answer
y to the question above is
Pr[b(xi , yi)=07b(xi , y)=1]+1
2 Pr[b(xi , yi)=b(xi , y)]
=1
2 (1&h(xi , yi)+h(xi , y)).
If the answers to all k&1 questions are considered
equally important, then it is natural to define the loss of the
hypothesis to be the average, over all k&1 questions, of the
probability of an incorrect answer:
1
k&1
:
y{ yi
1
2
(1&h(xi , yi)+h(xi , y))
=

1
2 \1&h(xi , yi)+
1
k&1
:
y{ yi
h(xi , y)
+. (24)
However, as was discussed in the introduction to Section 5, different discrimination questions are likely to have different importance in different situations. For example, considering the OCR problem described earlier, it might be that at some point during the boosting process, some example of the digit ``7'' has been recognized as being either a ``7'' or a ``9''. At this stage the question that discriminates between ``7'' (the correct label) and ``9'' is clearly much more important than the other eight questions that discriminate ``7'' from the other digits.

A natural way of attaching different degrees of importance to the different questions is to assign a weight to each question. So, for each instance xi and incorrect label y{ yi , we assign a weight q(i, y) which we associate with the question that discriminates label y from the correct label yi . We then replace the average used in Eq. (24) with an average weighted according to q(i, y); the resulting formula is called the pseudo-loss of h on training instance i with respect to q:

plossq(h, i).1
2
\1&h(xi , yi)+ :
y{ yi
q(i, y) h(xi , y)
+.

The function q=[1, ..., N]_Y [U+0014 control character] [0, 1], called the label weighting function, assigns to each example i in the training set a probability distribution over the k&1 discrimination problems defined above. So, for all i,

:
y{ yi
q(i, y)=1.

The weak learner's goal is to minimize the expected pseudo-loss for given distribution D and weighting function q:

plossD, q(h) :=EitD [plossq(h, i)].

As we have seen, by manipulating both the distribution on instances, and the label weighting function q, our boosting algorithm effectively forces the weak learner to focus not only on the hard instances, but also on the incorrect class labels that are hardest to eliminate. Conversely, this pseudo-loss measure may make it easier for the weak learner to get a weak advantage. For instance, if the weak learner can simply determine that a particular instance does not belong to a certain class (even if it has no idea which of the remaining classes is the correct one), then, depending on q, this may be enough to gain a weak advantage.

Theorem 11, the main result of this section, shows that a weak learner can be boosted if it can consistently produce weak hypotheses with pseudo-losses smaller than 1[U+0012 control character]2. Note that pseudo-loss 1[U+0012 control character]2 can be achieved trivially by any uninformative hypothesis. Furthermore, a weak hypothesis h with pseudo-loss =>1[U+0012 control character]2 is also beneficial to boosting since it can be replaced by the hypothesis 1&h whose pseudo-loss is 1&=<1[U+0012 control character]2.

Example 5. As a simple example illustrating the use of pseudo-loss, suppose we seek an oblivious weak hypothesis, i.e., a weak hypothesis whose value depends only on the class label y so that h(x, y)=h(y) for all x. Although oblivious hypotheses per se are generally too weak to be of interest, it may often be appropriate to find the best oblivious hypothesis on a part of the instance space (such as the set of instances covered by a leaf of a decision tree).

Let D be the target distribution, and q the label weighting function. For notational convenience, let us define q(i, yi) =&1 for all i so that

plossq(h, i)=1
2
\1+ :
y # Y
q(i, y) h(xi , y)
+.

Setting $(y)=[U+001D control character]i D(i) q(i, y), it can be verified that for an oblivious hypothesis h,

plossD, q(h)=1
2
\1+ :
y # Y
h(y) $(y)
+,

which is clearly minimized by the choice

h(y)=
{
1 if $(y)<0
0 otherwise.

Suppose now that q(i, y)=1[U+0012 control character](k&1) for y{ yi , and let d(y)=PritD[yi= y] be the proportion of examples with

133
A DECISION THEORETIC GENERALIZATION

[form-feed in source extraction]
File: 571J 150416 . By:CV . Date:28:07:01 . Time:05:54 LOP8M. V8.0. Page 01:01
Codes: 5640 Signs: 4166 . Length: 56 pic 0 pts, 236 mm
label y. Then it can be verified that h will always have
pseudo-loss strictly smaller than 1[U+0012 control character]2 except in the case of a
uniform distribution of labels (d(y)=1[U+0012 control character]k for all y). In
contrast, when the weak learner's goal is minimization of
prediction error (as in Section 5.1), it can be shown that an
oblivious hypothesis with prediction error strictly less than
1[U+0012 control character]2 can only be found when one label y covers more than 1[U+0012 control character]2
the distribution (d(y)>1[U+0012 control character]2). So in this case, it is much
easier to find a hypothesis with small pseudo-loss rather
than small prediction error.
On the other hand, if q(i, y)=0 for some values of y, then
the quality of prediction on these labels is of no conse-
quence. In particular, if q(i, y)=0 for all but one incorrect
label for each instance i, then in order to make the pseudo-
loss smaller than 1[U+0012 control character]2 the hypothesis has to predict the
correct label with probability larger than 1[U+0012 control character]2, which means
that in this case the pseudo-loss criterion is as stringent as
the usual prediction error. However, as discussed above,
this case is unavoidable because a hard binary classification
problem can always be embedded in a multi-class problem.
This example suggests that it may often be significantly
easier to find weak hypotheses with small pseudo-loss rather
than hypotheses whose prediction error is small. On the
other hand, our theoretical bound for boosting using the
prediction error (Theorem 10) is stronger than the bound
for ploss (Theorem 11). Empirical tests [12] have shown
that pseudo-loss is generally more successful when the weak
learners use very restricted hypotheses. However, for more
powerful weak learners, such as decision-tree learning algo-
rithms, there is little difference between using pseudo-loss
and prediction error.
Our algorithm called AdaBoost.M2, is shown in Fig. 4.
Here, we maintain weights wt
i, y for each instance i and each
label y # Y&[yi]. The weak learner must be provided both
with a distribution Dt and a label weight function qt . Both
of these are computed using the weight vector wt
as shown
in Step 1. The weak learner's goal then is to minimize the
pseudo-loss =t , as defined in Step 3. The weights are updated
as shown in Step 5. The final hypothesis hf outputs, for a
given instance x, the label y that maximizes a weighted
average of the weak hypothesis values ht(x, y).

[Next verbatim source excerpt]

mark AdaBoost variables with a tilde.
Algorithm AdaBoost.M2
Input: sequence of N examples ((x1 , y1)..., (xN , yN)) with labels
yi # Y=[1, ..., k]
distribution D over the N examples
weak learning algorithm WeakLearn
integer T specifying number of iterations
Initialize the weight vector: w1
i, y=D(i)[U+0012 control character](k&1) for i=1, ..., N, y # Y&[yi].
Do for t=1, 2, ..., T
1. Set Wt
i =[U+001D control character]y{ yi
wt
i, y ;
qt(i, y)=
wt
i, y
Wt
i
for y{ yi ; and set
Dt(i)=
Wt
i
[U+001D control character]N
i=1 Wt
i
.
2. Call WeakLearn, providing it with the distribution Dt and label
weighting function qt ; get back a hypothesis ht: X_Y [U+0014 control character] [0, 1].
3. Calculate the pseudo-loss of ht :
=t=
1
2
:
N
i=1
Dt(i)
\1&ht(xi , yi)+ :
y{ yi
qt(i, y) ht(xi , y)
+.
4. Set ;t==t [U+0012 control character](1&=t).
5. Set the new weights vector to be
wt+1
i, y =wt
i, y ;(1[U+0012 control character]2)(1+ht(xi , yi)&ht(xi , y))
t
for i=1, ..., N, y # Y&[yi].

Output the hypothesis
hf (x)=arg max
y # Y
:
T
t=1
\log
1
;t+ht(x, y).
FIG. 4. A second multi-class extension of AdaBoost.
For each training instance (xi , yi) and for each incorrect
label y # Y&[yi], we define one AdaBoost instance x
~ i, y=
(i, y) with associated label y
~ i, y=0. Thus, there are N
[U+0019 control character] =
N(k&1) AdaBoost instances, each indexed by a pair (i, y).
The distribution over these instances is defined to be D
[U+0019 control character] (i, y)
=D(i)[U+0012 control character](k&1). The tth hypothesis h
[U+0019 control character] t provided to AdaBoost
for this reduction is defined by the rule
h

Formalized review target

The archival source above is not asserted equivalent to this formalized target.

For the finite Figure-4 AdaBoost.M2 execution with every observed pseudo-loss strictly between zero and one, the final argmax-vote classifier has training error at most (k-1) 2^T times the product of sqrt(epsilon_t(1-epsilon_t)).

Archival source anchor: cited publication, lines 265-311

Lean-expanded library semantic target

```
type:
{Training : Type u_1} →
  {Label : Type u_2} →
    [inst : Fintype Training] →
      [inst_1 : Fintype Label] →
        [inst_2 : DecidableEq Label] →
          (label : Training → Label) →
            [inst_3 : Nonempty (AppliedModelingLib.Learning.Online.AdaBoostM2Instance Training Label label)] →
              (state :
                  AppliedModelingLib.Learning.Online.HedgeWeightState
                    (AppliedModelingLib.Learning.Online.AdaBoostM2Instance Training Label label)) →
                (hypotheses : List (Training → Label → ℝ)) →
                  AppliedModelingLib.Learning.Online.AdaBoostM2Admissible label state hypotheses → Training → Label → ℝ

value:
fun {Training : Type u_1} {Label : Type u_2} [Fintype Training] [Fintype Label] [DecidableEq Label]
    (label : Training → Label) [Nonempty (AppliedModelingLib.Learning.Online.AdaBoostM2Instance Training Label label)]
    (state :
      AppliedModelingLib.Learning.Online.HedgeWeightState
        (AppliedModelingLib.Learning.Online.AdaBoostM2Instance Training Label label))
    (hypotheses : List (Training → Label → ℝ))
    (a : AppliedModelingLib.Learning.Online.AdaBoostM2Admissible label state hypotheses) (a_1 : Training)
    (a_2 : Label) =>
  List.brecOn (motive := fun (x : List (Training → Label → ℝ)) =>
    (state :
        AppliedModelingLib.Learning.Online.HedgeWeightState
          (AppliedModelingLib.Learning.Online.AdaBoostM2Instance Training Label label)) →
      AppliedModelingLib.Learning.Online.AdaBoostM2Admissible label state x → Training → Label → ℝ)
    hypotheses (AppliedModelingLib.Learning.Online.adaBoostM2LabelScore._f label) state a a_1 a_2
```

Exact Lean library declaration

```
/-- The Figure-4 score of a candidate label at an original training example. -/
noncomputable def adaBoostM2LabelScore
    {Training Label : Type*} [Fintype Training] [Fintype Label] [DecidableEq Label]
    (label : Training → Label)
    [Nonempty (AdaBoostM2Instance Training Label label)]
    (state : HedgeWeightState (AdaBoostM2Instance Training Label label)) :
    (hypotheses : List (Training → Label → ℝ)) →
      AdaBoostM2Admissible label state hypotheses → Training → Label → ℝ
  | [], _, _, _ => 0
  | hypothesis :: remaining, hvalid, sample, candidate =>
      adaBoostVoteWeight
          (adaBoostError state (fun _ => 0) (adaBoostM2BinaryHypothesis label hypothesis)) *
        hypothesis sample candidate +
      adaBoostM2LabelScore label
        (adaBoostStep state (fun _ => 0) (adaBoostM2BinaryHypothesis label hypothesis)
          (Classical.choose hvalid)) remaining (Classical.choose_spec hvalid) sample candidate
```

Direct retained dependencies

- AppliedModelingLib.Learning.Online.AdaBoostAdmissible
- AppliedModelingLib.Learning.Online.AdaBoostM2Admissible

- AppliedModelingLib.Learning.Online.AdaBoostM2Instance
- AppliedModelingLib.Learning.Online.HedgeWeightState
- AppliedModelingLib.Learning.Online.adaBoostError
- AppliedModelingLib.Learning.Online.adaBoostM2BinaryHypothesis
- AppliedModelingLib.Learning.Online.adaBoostStep
- AppliedModelingLib.Learning.Online.adaBoostVoteWeight

Recorded source-to-library screening **Verdict:** matches_approved_corrected_target
Reason: Compared theorem11_adaboost_m2_error's Figure 4 pseudo-loss, update, and output with the displayed score and reduction. On incorrect-label pairs the reduced hypothesis is (1-h(x,y_i)+h(x,y))/2, so its normalized error is the printed pseudo-loss and its reversed-loss update exponent is (1+h(x,y_i)-h(x,y))/2. The score adds log((1-epsilon)/epsilon)*h(x,candidate) at each actual recursive state, with zero for an empty run. Theorem 11 supplies D(i)/(k-1) initial pair weights, k>1, and hypotheses in [0,1], while the pinned FS97-ADABOOST-VARIANTS-ENDPOINT-DOMAIN-06 record supplies 0<epsilon<1. The displayed final classifier maximizes these exact scores without imposing an upper-half error bound. The comparison is to the displayed approved corrected target bound to FS97-ADABOOST-VARIANTS-ENDPOINT-DOMAIN-06; archival equivalence is not claimed.
Reviewer check: ☐ Matches formalized target
If left unchecked, explain the discrepancy or uncertainty below.
Reviewer annotation

AppliedModelingLib.Learning.Online.adaBoostRErrors

Source locator: cited publication, lines 2418-2422

Verbatim paper-source connection

Theorem 12. Suppose WeakLearn, when called by AdaBoost.R, generates
hypotheses with errors $\epsilon_1,\ldots,\epsilon_T$, where ϵ_t is
as defined in Fig. 5. Then the mean squared error
$\epsilon=\mathbb E_{i\sim D}[(h_f(x_i)-y_i)^2]$ is bounded above by
$$
\epsilon\leq2^T\prod_{t=1}^T\sqrt{\epsilon_t(1-\epsilon_t)}. \tag{26}
$$

[Next verbatim source excerpt]

5.3. Boosting Regression Algorithms
In this section we show how boosting can be used for a
regression problem. In this setting, the label space is
Y=[0, 1]. As before, the learner receives examples (x, y)
chosen at random according to some distribution P, and its
goal is to find a hypothesis h: X [U+0014 control character] Y which, given some x
value, predicts approximately the value y that is likely to be
seen. More precisely, the learner attempts to find an h with
small mean squared error (MES):
E(x, y)t P[(h(x)& y)2
]. (25)
Our methods can be applied to any reasonable bounded
error measure, but, for the sake of concreteness, we concen-
trate here on the squared error measure.
Following our approach for classification problems, we
assume that the leaner has been provided with a training set
(x1 , y1), ..., (xN , yN) of examples distributed according to
P, and we focus only on the minimization of the empirical
MSE:
1
N
:
N
i=1
(h(xi)& yi)2
.
Using techniques similar to those outlined in Section 4.3,
the true MSE given in Eq. (25) can be related to the empiri-
cal MSE.
To derive a boosting algorithm in this context, we reduce
the given regression problem to a binary classification
problem, and then apply AdaBoost. As was done for the
reductions used in the proofs of Theorems 10 and 11, we
mark with tildes all variables in the reduced (AdaBoost)
space. For each example (xi , yi) in the training set, we
define a continuum of examples indexed by pairs (i, y) for
all y # [0, 1]: the associated instance is x
~ i, y=(xi , y), and
the label is y
~ i, y=[U+001A control character]y[U+001E control character] yi[U+0019 control character]. (Recall that [U+001A control character]?[U+0019 control
↪ character] is 1 if predicate
? holds and 0 otherwise.) Although it is obviously infeasible
to explicitly maintain an infinitely large training set, we will
see later how this method can be implemented efficiently.
Also, although the results of Section 4 only dealt with
finitely large training sets, the extension to infinite training
sets is straightforward.
Thus, informally, each instance (xi , yi) is mapped to an
infinite set of binary questions, one for each y # Y, and each
of the form: ``Is the correct label yi bigger or smaller than y?''
In a similar manner, each hypothesis h: X [U+0014 control character] Y is reduced
to a binary-valued hypothesis h
[U+0019 control character] : X_Y[U+0014 control character] [0, 1] defined by
the rule
h
[U+0019 control character] (x, y)=[U+001A control character]y[U+001E control character]h(x)[U+0019 control character].
Thus, h
[U+0019 control character] attempts to answer these binary questions in a
natural way using the estimated value h(x).
Finally, as was done for classification problems, we
assume we are given a distribution D over the training set;
ordinarily, this will be uniform so that D(i)=1[U+0012 control character]N. In our
reduction, this distribution is mapped to a density D
[U+0019 control character] over
pairs (i, y) in such a way that minimization of classification
error in the reduced space is equivalent to minimization of
MSE for the original problem. To do this, we define
D
[U+0019 control character] (i, y)=
D(i) | y& yi |
Z
where Z is a normalization constant:
Z= :
N
i=1
D(i) |
1
0
| y& yi | dy.

It is straightforward to show that 1[U+0012 control character]4[U+001D control character]Z[U+001D control character]1[U+0012 control character]2.
135
A DECISION THEORETIC GENERALIZATION

[form-feed in source extraction]
File: 571J 150418 . By:CV . Date:28:07:01 . Time:05:54 LOP8M. V8.0. Page 01:01
Codes: 5416 Signs: 3774 . Length: 56 pic 0 pts, 236 mm
If we calculate the binary error of h
[U+0019 control character] with respect to the
density D
[U+0019 control character] , we find that, as desired, it is directly proportional
to the mean squared error:
:
N
i=1
|
1
0
| y
~ i, y&h
[U+0019 control character] (x
~ i, y)| D
[U+0019 control character] (i, y) dy
=
1
Z
:
N
i=1
D(i)
}|
h(xi)
yi
| y& yi | dy
}
=
1
2Z
:
N
i=1
D(i)(h(xi)& yi)2
.
The constant of proportionality is 1[U+0012 control character](2Z) # [1, 2].
Unravelling this reduction, we obtain the regression
boosting procedure AdaBoost.R shown in Fig. 5. As
prescribed by the reduction, AdaBoost.R maintains a weight
wt
i, y for each instance i and label y # Y. The initial weight
function w1
is exactly the density D
[U+0019 control character] defined above. By nor-
malizing the weights wt
, a density pt
is defined at Step 1 and
provided to the weak learner at Step 2. The goal of the weak
learner is to find a hypothesis ht : X[U+0014 control character] Y that minimizes the
loss =t defined in Step 3. Finally, at Step 5, the weights are
updated as prescribed by the reduction.
The definition of =t at Step 3 follows directly from the
reduction above; it is exactly the classification error of h
[U+0019 control character] f in
the reduced space. Note that, similar to AdaBoost.M2,
AdaBoost.R not only varies the distribution over the
examples (xi , yi), but also modifies from round to round the
definition of the loss suffered by a hypothesis on each
example. Thus, although our ultimate goal is minimization
of the squared error, the weak learner must be able to
handle loss functions that are more complicated than MSE.
The final hypothesis hf also is consistent with the reduc-
tion. Each reduced weak hypothesis h
[U+0019 control character] f (x, y) is non-
decreasing as a function of y. Thus, the final hypothesis h
[U+0019 control character] f
generated by AdaBoost in the reduced space, being the
threshold of a weighted sum of these hypotheses, also is
non-decreasing as a function of y. As the output of h
[U+0019 control character] f is
binary, this implies that for every x there is one value of y
for which h
[U+0019 control character] f (x, y$)=0 for all y$< y and h
[U+0019 control character] f (x, y$)=1 for all
y$> y. This is exactly the value of y given by hf (x) as defined
in the figure. Note that hf is actually computing a weighted
median of the weak hypotheses.
At first, it might seem impossible to maintain weights wt
i, y
over an uncountable set of points. However, on closer
inspection, it can be seen that, when viewed as a function of
y, wt
i, y is a piece-wise linear function. For t=1, w1
i, y has two
linear pieces, and each update at Step 5 potentially breaks
one of the pieces in two at the point ht(xi). Initializing,
storing and updating such piece-wise linear functions are
all straightforward operations. Also, the integrals which
appear in the figure can be evaluated explicitly since these

only involve integration of piece-wise linear functions.

[Next verbatim source excerpt]

Algorithm AdaBoost.R
Input: sequence of N examples ((x1 , y1)..., (xN , yN)) with labels
yi # Y=[0, 1]
distribution D over the examples
weak learning algorithm WeakLearn
integer T specifying number of iterations
Initialize the weight vector:
w1
i, y=
D(i) | y& yi |
Z
for i=1, ..., N, y # Y, where
Z= :
N
i=1
D(i)
|
1
0
|y& yi | dy.
Do for t=1, 2, ..., T
1. Set
pt
=
wt
[U+001D control character]N
i=1 [U+001E control character]1
0 wt
i, y dy
.
2. Call WeakLearn, providing it with the density pt
; get back a
hypothesis ht : X_Y.
3. Calculate the loss of ht :
=t= :
N
i=1
}|
ht(xi)
yi
pt
i, y dy
}.
If =t>1[U+0012 control character]2, then set T=t&1 and abort the loop.
4. Set ;t==t [U+0012 control character](1&=t).
5. Set the new weights vector to be
wt+1
i, y =
{
wt
i, y
wt
i, y ;t
if yi[U+001D control character] y[U+001D control character]ht(xi) or ht(xi)[U+001D control character] y[U+001D control character] yi
otherwise.
for i=1, ..., N, y # Y.
Output the hypothesis
hf (x)=inf
{y # Y: :
t: ht(x)[U+001D control character] y
log(1[U+0012 control character];t)[U+001E control character]1
2 :
t
log(1[U+0012 control character];t)
=.
FIG. 5. An extension of AdaBoost to regression problems.
The following theorem describes our performance
guarantee for AdaBoost.R. The proof follows from the
reduction described above coupled with a direct application
of Theorem 6.

Formalized review target

The archival source above is not asserted equivalent to this formalized target.

For the finite Figure-5 AdaBoost.R execution with every observed reduced error positive and at most one half, the final weighted-median regressor has mean squared error at most 2^T times the product of sqrt(epsilon_t(1-epsilon_t)).

Archival source anchor: cited publication, lines 315-363

Lean-expanded library semantic target

type:
```
{Training : Type u_1} →
  [inst : Fintype Training] →
    (state : AppliedModelingLib.Learning.Online.AdaBoostRDensityState Training) →
      (label : Training → ℝ) →
        (hypotheses : List (Training → ℝ)) →
          AppliedModelingLib.Learning.Online.AdaBoostRAdmissible state label hypotheses → List ℝ
```

value:

```
fun {Training : Type u_1} [Fintype Training] (state : AppliedModelingLib.Learning.Online.AdaBoostRDensityState Training)
    (label : Training → ℝ) (hypotheses : List (Training → ℝ))
    (a : AppliedModelingLib.Learning.Online.AdaBoostRAdmissible state label hypotheses) =>
  List.brecOn (motive := fun (x : List (Training → ℝ)) =>
    (state : AppliedModelingLib.Learning.Online.AdaBoostRDensityState Training) →
      AppliedModelingLib.Learning.Online.AdaBoostRAdmissible state label x → List ℝ)
    hypotheses (AppliedModelingLib.Learning.Online.adaBoostRErrors._f label) state a
```

Exact Lean library declaration

```
/-- Extract the actual normalized Step-3 errors of a certified Figure-5 run. -/
noncomputable def adaBoostRErrors
    {Training : Type*} [Fintype Training]
    (state : AdaBoostRDensityState Training) (label : Training → ℝ) :
    (hypotheses : List (Training → ℝ)) →
      AdaBoostRAdmissible state label hypotheses → List ℝ
  | [], _ => []
  | hypothesis :: remaining, hvalid =>
      adaBoostRReducedError state label hypothesis ::
        adaBoostRErrors
          (adaBoostRStep state label hypothesis (Classical.choose hvalid)) label remaining
          (Classical.choose_spec hvalid)
```

Direct retained dependencies

- AppliedModelingLib.Learning.Online.AdaBoostRAdmissible
- AppliedModelingLib.Learning.Online.AdaBoostRDensityState
- AppliedModelingLib.Learning.Online.adaBoostRReducedError
- AppliedModelingLib.Learning.Online.adaBoostRStep

Recorded source-to-library screening

Verdict: matches_approved_corrected_target

Reason: Compared theorem12_adaboost_r_error's Figure 5 Step 3 error sequence with the displayed recursion, reduced-error integral, density normalization, and step update. The function emits the normalized integral over the closed unordered interval between the true and predicted values, then computes the next error only after applying that round's density update; the empty run emits no errors. Under the displayed [0,1] label/hypothesis paper use this integral equals the source's absolute oriented integral, with endpoint inclusion immaterial for Lebesgue integration. The retained errors have 0<epsilon<=1/2 as required by the stopping rule and pinned endpoint correction, and the Theorem 12 factor consumes exactly this list. The comparison is to the displayed approved corrected target bound to FS97-ADABOOST-VARIANTS-ENDPOINT-DOMAIN-06; archival equivalence is not claimed.

Reviewer check: ☐ Matches formalized target

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

AppliedModelingLib.Learning.Online.adaBoostRVote

Source locator: cited publication, lines 2418-2422

Verbatim paper-source connection

Theorem 12. Suppose WeakLearn, when called by AdaBoost.R, generates
hypotheses with errors $\epsilon_1,\ldots,\epsilon_T$, where ϵ_t is
as defined in Fig. 5. Then the mean squared error
$\epsilon=\mathbb E_{i\sim D}[(h_f(x_i)-y_i)^2]$ is bounded above by
$$
\epsilon\leq2^T\prod_{t=1}^T\sqrt{\epsilon_t(1-\epsilon_t)}. \tag{26}
$$

[Next verbatim source excerpt]

5.3. Boosting Regression Algorithms
In this section we show how boosting can be used for a
regression problem. In this setting, the label space is
Y=[0, 1]. As before, the learner receives examples (x, y)
chosen at random according to some distribution P, and its
goal is to find a hypothesis h: X [U+0014 control character] Y which, given some x
value, predicts approximately the value y that is likely to be
seen. More precisely, the learner attempts to find an h with
small mean squared error (MES):
E(x, y)t P[(h(x)& y)2
]. (25)
Our methods can be applied to any reasonable bounded
error measure, but, for the sake of concreteness, we concen-
trate here on the squared error measure.
Following our approach for classification problems, we
assume that the leaner has been provided with a training set
(x1 , y1), ..., (xN , yN) of examples distributed according to
P, and we focus only on the minimization of the empirical
MSE:
1
N
:
N
i=1
(h(xi)& yi)2
.
Using techniques similar to those outlined in Section 4.3,
the true MSE given in Eq. (25) can be related to the empiri-
cal MSE.
To derive a boosting algorithm in this context, we reduce
the given regression problem to a binary classification
problem, and then apply AdaBoost. As was done for the
reductions used in the proofs of Theorems 10 and 11, we
mark with tildes all variables in the reduced (AdaBoost)
space. For each example (xi , yi) in the training set, we
define a continuum of examples indexed by pairs (i, y) for
all y # [0, 1]: the associated instance is x
~ i, y=(xi , y), and
the label is y
~ i, y=[U+001A control character]y[U+001E control character] yi[U+0019 control character]. (Recall that [U+001A control character]?[U+0019 control
↪ character] is 1 if predicate
? holds and 0 otherwise.) Although it is obviously infeasible
to explicitly maintain an infinitely large training set, we will
see later how this method can be implemented efficiently.
Also, although the results of Section 4 only dealt with
finitely large training sets, the extension to infinite training
sets is straightforward.
Thus, informally, each instance (xi , yi) is mapped to an
infinite set of binary questions, one for each y # Y, and each
of the form: ``Is the correct label yi bigger or smaller than y?''
In a similar manner, each hypothesis h: X [U+0014 control character] Y is reduced
to a binary-valued hypothesis h
[U+0019 control character] : X_Y[U+0014 control character] [0, 1] defined by
the rule
h
[U+0019 control character] (x, y)=[U+001A control character]y[U+001E control character]h(x)[U+0019 control character].
Thus, h
[U+0019 control character] attempts to answer these binary questions in a
natural way using the estimated value h(x).
Finally, as was done for classification problems, we
assume we are given a distribution D over the training set;
ordinarily, this will be uniform so that D(i)=1[U+0012 control character]N. In our
reduction, this distribution is mapped to a density D
[U+0019 control character] over
pairs (i, y) in such a way that minimization of classification
error in the reduced space is equivalent to minimization of
MSE for the original problem. To do this, we define
D
[U+0019 control character] (i, y)=
D(i) | y& yi |
Z
where Z is a normalization constant:
Z= :
N
i=1
D(i) |
1
0
| y& yi | dy.

It is straightforward to show that 1[U+0012 control character]4[U+001D control character]Z[U+001D control character]1[U+0012 control character]2.
135
A DECISION THEORETIC GENERALIZATION

[form-feed in source extraction]
File: 571J 150418 . By:CV . Date:28:07:01 . Time:05:54 LOP8M. V8.0. Page 01:01
Codes: 5416 Signs: 3774 . Length: 56 pic 0 pts, 236 mm
If we calculate the binary error of h
[U+0019 control character] with respect to the
density D
[U+0019 control character] , we find that, as desired, it is directly proportional
to the mean squared error:
:
N
i=1
|
1
0
| y
~ i, y&h
[U+0019 control character] (x
~ i, y)| D
[U+0019 control character] (i, y) dy
=
1
Z
:
N
i=1
D(i)
}|
h(xi)
yi
| y& yi | dy
}
=
1
2Z
:
N
i=1
D(i)(h(xi)& yi)2
.
The constant of proportionality is 1[U+0012 control character](2Z) # [1, 2].
Unravelling this reduction, we obtain the regression
boosting procedure AdaBoost.R shown in Fig. 5. As
prescribed by the reduction, AdaBoost.R maintains a weight
wt
i, y for each instance i and label y # Y. The initial weight
function w1
is exactly the density D
[U+0019 control character] defined above. By nor-
malizing the weights wt
, a density pt
is defined at Step 1 and
provided to the weak learner at Step 2. The goal of the weak
learner is to find a hypothesis ht : X[U+0014 control character] Y that minimizes the
loss =t defined in Step 3. Finally, at Step 5, the weights are
updated as prescribed by the reduction.
The definition of =t at Step 3 follows directly from the
reduction above; it is exactly the classification error of h
[U+0019 control character] f in
the reduced space. Note that, similar to AdaBoost.M2,
AdaBoost.R not only varies the distribution over the
examples (xi , yi), but also modifies from round to round the
definition of the loss suffered by a hypothesis on each
example. Thus, although our ultimate goal is minimization
of the squared error, the weak learner must be able to
handle loss functions that are more complicated than MSE.
The final hypothesis hf also is consistent with the reduc-
tion. Each reduced weak hypothesis h
[U+0019 control character] f (x, y) is non-
decreasing as a function of y. Thus, the final hypothesis h
[U+0019 control character] f
generated by AdaBoost in the reduced space, being the
threshold of a weighted sum of these hypotheses, also is
non-decreasing as a function of y. As the output of h
[U+0019 control character] f is
binary, this implies that for every x there is one value of y
for which h
[U+0019 control character] f (x, y$)=0 for all y$< y and h
[U+0019 control character] f (x, y$)=1 for all
y$> y. This is exactly the value of y given by hf (x) as defined
in the figure. Note that hf is actually computing a weighted
median of the weak hypotheses.
At first, it might seem impossible to maintain weights wt
i, y
over an uncountable set of points. However, on closer
inspection, it can be seen that, when viewed as a function of
y, wt
i, y is a piece-wise linear function. For t=1, w1
i, y has two
linear pieces, and each update at Step 5 potentially breaks
one of the pieces in two at the point ht(xi). Initializing,
storing and updating such piece-wise linear functions are
all straightforward operations. Also, the integrals which
appear in the figure can be evaluated explicitly since these

only involve integration of piece-wise linear functions.

[Next verbatim source excerpt]

Algorithm AdaBoost.R
Input: sequence of N examples ((x1 , y1)..., (xN , yN)) with labels
yi # Y=[0, 1]
distribution D over the examples
weak learning algorithm WeakLearn
integer T specifying number of iterations
Initialize the weight vector:
w1
i, y=
D(i) | y& yi |
Z
for i=1, ..., N, y # Y, where
Z= :
N
i=1
D(i)
|
1
0
|y& yi | dy.
Do for t=1, 2, ..., T
1. Set
pt
=
wt
[U+001D control character]N
i=1 [U+001E control character]1
0 wt
i, y dy
.
2. Call WeakLearn, providing it with the density pt
; get back a
hypothesis ht : X_Y.
3. Calculate the loss of ht :
=t= :
N
i=1
}|
ht(xi)
yi
pt
i, y dy
}.
If =t>1[U+0012 control character]2, then set T=t&1 and abort the loop.
4. Set ;t==t [U+0012 control character](1&=t).
5. Set the new weights vector to be
wt+1
i, y =
{
wt
i, y
wt
i, y ;t
if yi[U+001D control character] y[U+001D control character]ht(xi) or ht(xi)[U+001D control character] y[U+001D control character] yi
otherwise.
for i=1, ..., N, y # Y.
Output the hypothesis
hf (x)=inf
{y # Y: :
t: ht(x)[U+001D control character] y
log(1[U+0012 control character];t)[U+001E control character]1
2 :
t
log(1[U+0012 control character];t)
=.
FIG. 5. An extension of AdaBoost to regression problems.
The following theorem describes our performance
guarantee for AdaBoost.R. The proof follows from the
reduction described above coupled with a direct application
of Theorem 6.

Formalized review target

The archival source above is not asserted equivalent to this formalized target.

For the finite Figure-5 AdaBoost.R execution with every observed reduced error positive and at most one half, the final weighted-median regressor has mean squared error at most 2^T times the product of sqrt(epsilon_t(1-epsilon_t)).

Archival source anchor: cited publication, lines 315-363

Lean-expanded library semantic target

type:
{Training : Type u_1} →
  [inst : Fintype Training] →
    (state : AppliedModelingLib.Learning.Online.AdaBoostRDensityState Training) →
      (label : Training → ℝ) →
        (hypotheses : List (Training → ℝ)) →
          AppliedModelingLib.Learning.Online.AdaBoostRAdmissible state label hypotheses → Training → ℝ → ℝ

value:

```
fun {Training : Type u_1} [Fintype Training] (state : AppliedModelingLib.Learning.Online.AdaBoostRDensityState Training)
    (label : Training → ℝ) (hypotheses : List (Training → ℝ))
    (a : AppliedModelingLib.Learning.Online.AdaBoostRAdmissible state label hypotheses) (a_1 : Training) (a_2 : ℝ) =>
  List.brecOn (motive := fun (x : List (Training → ℝ)) =>
    (state : AppliedModelingLib.Learning.Online.AdaBoostRDensityState Training) →
      AppliedModelingLib.Learning.Online.AdaBoostRAdmissible state label x → Training → ℝ → ℝ)
    hypotheses (AppliedModelingLib.Learning.Online.adaBoostRVote._f label) state a a_1 a_2
```

Exact Lean library declaration

```
/-- The weighted binary vote of Figure 5's thresholded weak regressors. -/
noncomputable def adaBoostRVote
    {Training : Type*} [Fintype Training]
    (state : AdaBoostRDensityState Training) (label : Training → ℝ) :
    (hypotheses : List (Training → ℝ)) →
      AdaBoostRAdmissible state label hypotheses → Training → ℝ → ℝ
  | [], _, _, _ => 0
  | hypothesis :: remaining, hvalid, sample, y =>
      adaBoostVoteWeight (adaBoostRReducedError state label hypothesis) *
          (if hypothesis sample ≤ y then 1 else 0) +
        adaBoostRVote
          (adaBoostRStep state label hypothesis (Classical.choose hvalid)) label remaining
          (Classical.choose_spec hvalid) sample y
```

Direct retained dependencies

- AppliedModelingLib.Learning.Online.AdaBoostRAdmissible
- AppliedModelingLib.Learning.Online.AdaBoostRDensityState
- AppliedModelingLib.Learning.Online.adaBoostRReducedError
- AppliedModelingLib.Learning.Online.adaBoostRStep
- AppliedModelingLib.Learning.Online.adaBoostVoteWeight

Recorded source-to-library screening **Verdict:** matches_approved_corrected_target

Reason: Compared theorem12_adaboost_r_error's Figure 5 output sum_{t:h_t(x)<=y} log(1/beta_t) with the displayed vote recursion and all density/error dependencies. Each increment is log((1-epsilon_t)/epsilon_t) times the inclusive indicator h_t(sample)<=y, and the next coefficient is obtained after the correct Figure 5 density update. The empty sum is zero. Theorem 12 restricts predictions to [0,1] and uses the corrected executed-round domain 0<epsilon_t<=1/2, so coefficients are finite and nonnegative, including a zero coefficient at epsilon=1/2. The displayed good-candidate filter uses the source's inclusive half-total threshold and the final output selects its smallest candidate. The comparison is to the displayed approved corrected target bound to FS97-ADABOOST-VARIANTS-ENDPOINT-DOMAIN-06; archival equivalence is not claimed.

Reviewer check: ☐ Matches formalized target

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

AppliedModelingLib.Learning.Online.adaBoostRVoteWeightSum

Source locator: cited publication, lines 2418-2422

Verbatim paper-source connection

Theorem 12. Suppose WeakLearn, when called by AdaBoost.R, generates
hypotheses with errors $\epsilon_1,\ldots,\epsilon_T$, where ϵ_t is
as defined in Fig. 5. Then the mean squared error
$\epsilon=\mathbb E_{i\sim D}[(h_f(x_i)-y_i)^2]$ is bounded above by
$$
\epsilon\leq2^T\prod_{t=1}^T\sqrt{\epsilon_t(1-\epsilon_t)}. \tag{26}
$$

[Next verbatim source excerpt]

5.3. Boosting Regression Algorithms
In this section we show how boosting can be used for a
regression problem. In this setting, the label space is
Y=[0, 1]. As before, the learner receives examples (x, y)
chosen at random according to some distribution P, and its
goal is to find a hypothesis h: X [U+0014 control character] Y which, given some x
value, predicts approximately the value y that is likely to be
seen. More precisely, the learner attempts to find an h with
small mean squared error (MES):
E(x, y)t P[(h(x)& y)2
]. (25)
Our methods can be applied to any reasonable bounded
error measure, but, for the sake of concreteness, we concen-
trate here on the squared error measure.
Following our approach for classification problems, we
assume that the leaner has been provided with a training set
(x1 , y1), ..., (xN , yN) of examples distributed according to
P, and we focus only on the minimization of the empirical
MSE:
1
N
:
N
i=1
(h(xi)& yi)2
.
Using techniques similar to those outlined in Section 4.3,
the true MSE given in Eq. (25) can be related to the empiri-
cal MSE.
To derive a boosting algorithm in this context, we reduce
the given regression problem to a binary classification
problem, and then apply AdaBoost. As was done for the
reductions used in the proofs of Theorems 10 and 11, we
mark with tildes all variables in the reduced (AdaBoost)
space. For each example (xi , yi) in the training set, we
define a continuum of examples indexed by pairs (i, y) for
all y # [0, 1]: the associated instance is x
~ i, y=(xi , y), and
the label is y
~ i, y=[U+001A control character]y[U+001E control character] yi[U+0019 control character]. (Recall that [U+001A control character]?[U+0019 control
↪ character] is 1 if predicate
? holds and 0 otherwise.) Although it is obviously infeasible
to explicitly maintain an infinitely large training set, we will
see later how this method can be implemented efficiently.
Also, although the results of Section 4 only dealt with
finitely large training sets, the extension to infinite training
sets is straightforward.
Thus, informally, each instance (xi , yi) is mapped to an
infinite set of binary questions, one for each y # Y, and each
of the form: ``Is the correct label yi bigger or smaller than y?''
In a similar manner, each hypothesis h: X [U+0014 control character] Y is reduced
to a binary-valued hypothesis h
[U+0019 control character] : X_Y[U+0014 control character] [0, 1] defined by
the rule
h
[U+0019 control character] (x, y)=[U+001A control character]y[U+001E control character]h(x)[U+0019 control character].
Thus, h
[U+0019 control character] attempts to answer these binary questions in a
natural way using the estimated value h(x).
Finally, as was done for classification problems, we
assume we are given a distribution D over the training set;
ordinarily, this will be uniform so that D(i)=1[U+0012 control character]N. In our
reduction, this distribution is mapped to a density D
[U+0019 control character] over
pairs (i, y) in such a way that minimization of classification
error in the reduced space is equivalent to minimization of
MSE for the original problem. To do this, we define
D
[U+0019 control character] (i, y)=
D(i) | y& yi |
Z
where Z is a normalization constant:
Z= :
N
i=1
D(i) |
1
0
| y& yi | dy.

It is straightforward to show that 1[U+0012 control character]4[U+001D control character]Z[U+001D control character]1[U+0012 control character]2.
135
A DECISION THEORETIC GENERALIZATION

[form-feed in source extraction]
File: 571J 150418 . By:CV . Date:28:07:01 . Time:05:54 LOP8M. V8.0. Page 01:01
Codes: 5416 Signs: 3774 . Length: 56 pic 0 pts, 236 mm
If we calculate the binary error of h
[U+0019 control character] with respect to the
density D
[U+0019 control character] , we find that, as desired, it is directly proportional
to the mean squared error:
:
N
i=1
|
1
0
| y
~ i, y&h
[U+0019 control character] (x
~ i, y)| D
[U+0019 control character] (i, y) dy
=
1
Z
:
N
i=1
D(i)
}|
h(xi)
yi
| y& yi | dy
}
=
1
2Z
:
N
i=1
D(i)(h(xi)& yi)2
.
The constant of proportionality is 1[U+0012 control character](2Z) # [1, 2].
Unravelling this reduction, we obtain the regression
boosting procedure AdaBoost.R shown in Fig. 5. As
prescribed by the reduction, AdaBoost.R maintains a weight
wt
i, y for each instance i and label y # Y. The initial weight
function w1
is exactly the density D
[U+0019 control character] defined above. By nor-
malizing the weights wt
, a density pt
is defined at Step 1 and
provided to the weak learner at Step 2. The goal of the weak
learner is to find a hypothesis ht : X[U+0014 control character] Y that minimizes the
loss =t defined in Step 3. Finally, at Step 5, the weights are
updated as prescribed by the reduction.
The definition of =t at Step 3 follows directly from the
reduction above; it is exactly the classification error of h
[U+0019 control character] f in
the reduced space. Note that, similar to AdaBoost.M2,
AdaBoost.R not only varies the distribution over the
examples (xi , yi), but also modifies from round to round the
definition of the loss suffered by a hypothesis on each
example. Thus, although our ultimate goal is minimization
of the squared error, the weak learner must be able to
handle loss functions that are more complicated than MSE.
The final hypothesis hf also is consistent with the reduc-
tion. Each reduced weak hypothesis h
[U+0019 control character] f (x, y) is non-
decreasing as a function of y. Thus, the final hypothesis h
[U+0019 control character] f
generated by AdaBoost in the reduced space, being the
threshold of a weighted sum of these hypotheses, also is
non-decreasing as a function of y. As the output of h
[U+0019 control character] f is
binary, this implies that for every x there is one value of y
for which h
[U+0019 control character] f (x, y$)=0 for all y$< y and h
[U+0019 control character] f (x, y$)=1 for all
y$> y. This is exactly the value of y given by hf (x) as defined
in the figure. Note that hf is actually computing a weighted
median of the weak hypotheses.
At first, it might seem impossible to maintain weights wt
i, y
over an uncountable set of points. However, on closer
inspection, it can be seen that, when viewed as a function of
y, wt
i, y is a piece-wise linear function. For t=1, w1
i, y has two
linear pieces, and each update at Step 5 potentially breaks
one of the pieces in two at the point ht(xi). Initializing,
storing and updating such piece-wise linear functions are
all straightforward operations. Also, the integrals which
appear in the figure can be evaluated explicitly since these

only involve integration of piece-wise linear functions.

[Next verbatim source excerpt]

Algorithm AdaBoost.R
Input: sequence of N examples ((x1 , y1)..., (xN , yN)) with labels
yi # Y=[0, 1]
distribution D over the examples
weak learning algorithm WeakLearn
integer T specifying number of iterations
Initialize the weight vector:
w1
i, y=
D(i) | y& yi |
Z
for i=1, ..., N, y # Y, where
Z= :
N
i=1
D(i)
|
1
0
|y& yi | dy.
Do for t=1, 2, ..., T
1. Set
pt
=
wt
[U+001D control character]N
i=1 [U+001E control character]1
0 wt
i, y dy
.
2. Call WeakLearn, providing it with the density pt
; get back a
hypothesis ht : X_Y.
3. Calculate the loss of ht :
=t= :
N
i=1
}|
ht(xi)
yi
pt
i, y dy
}.
If =t>1[U+0012 control character]2, then set T=t&1 and abort the loop.
4. Set ;t==t [U+0012 control character](1&=t).
5. Set the new weights vector to be
wt+1
i, y =
{
wt
i, y
wt
i, y ;t
if yi[U+001D control character] y[U+001D control character]ht(xi) or ht(xi)[U+001D control character] y[U+001D control character] yi
otherwise.
for i=1, ..., N, y # Y.
Output the hypothesis
hf (x)=inf
{y # Y: :
t: ht(x)[U+001D control character] y
log(1[U+0012 control character];t)[U+001E control character]1
2 :
t
log(1[U+0012 control character];t)
=.
FIG. 5. An extension of AdaBoost to regression problems.
The following theorem describes our performance
guarantee for AdaBoost.R. The proof follows from the
reduction described above coupled with a direct application
of Theorem 6.

Formalized review target

The archival source above is not asserted equivalent to this formalized target.

For the finite Figure-5 AdaBoost.R execution with every observed reduced error positive and at most one half, the final weighted-median regressor has
↪ mean squared error at most 2^T times the product of sqrt(epsilon_t(1-epsilon_t)).

Archival source anchor: cited publication, lines 315-363

Lean-expanded library semantic target

type:
{Training : Type u_1} →
  [inst : Fintype Training] →
    (state : AppliedModelingLib.Learning.Online.AdaBoostRDensityState Training) →
      (label : Training → ℝ) →
        (hypotheses : List (Training → ℝ)) →
          AppliedModelingLib.Learning.Online.AdaBoostRAdmissible state label hypotheses → ℝ

value:

```
fun {Training : Type u_1} [Fintype Training] (state : AppliedModelingLib.Learning.Online.AdaBoostRDensityState Training)
    (label : Training → ℝ) (hypotheses : List (Training → ℝ))
    (a : AppliedModelingLib.Learning.Online.AdaBoostRAdmissible state label hypotheses) =>
  List.brecOn (motive := fun (x : List (Training → ℝ)) =>
    (state : AppliedModelingLib.Learning.Online.AdaBoostRDensityState Training) →
      AppliedModelingLib.Learning.Online.AdaBoostRAdmissible state label x → ℝ)
    hypotheses (AppliedModelingLib.Learning.Online.adaBoostRVoteWeightSum._f label) state a
```

Exact Lean library declaration

```
/-- The total Figure-5 vote weight. -/
noncomputable def adaBoostRVoteWeightSum
    {Training : Type*} [Fintype Training]
    (state : AdaBoostRDensityState Training) (label : Training → ℝ) :
    (hypotheses : List (Training → ℝ)) →
      AdaBoostRAdmissible state label hypotheses → ℝ
  | [], _ => 0
  | hypothesis :: remaining, hvalid =>
      adaBoostVoteWeight (adaBoostRReducedError state label hypothesis) +
        adaBoostRVoteWeightSum
          (adaBoostRStep state label hypothesis (Classical.choose hvalid)) label remaining
          (Classical.choose_spec hvalid)
```

Direct retained dependencies

- AppliedModelingLib.Learning.Online.AdaBoostRAdmissible
- AppliedModelingLib.Learning.Online.AdaBoostRDensityState
- AppliedModelingLib.Learning.Online.adaBoostRReducedError
- AppliedModelingLib.Learning.Online.adaBoostRStep
- AppliedModelingLib.Learning.Online.adaBoostVoteWeight

Recorded source-to-library screening **Verdict:** matches_approved_corrected_target

Reason: Compared theorem12_adaboost_r_error's Figure 5 half-threshold denominator sum_t log(1/beta_t) with the displayed total-vote recursion and dependencies. The function sums log((1-epsilon_t)/epsilon_t) along the same evolving normalized-density execution as the threshold vote, returning zero on the empty list. The displayed Figure 5 admissibility enforces precisely 0<epsilon_t<=1/2 under FS97-ADABOOST-VARIANTS-ENDPOINT-DOMAIN-06. Neither this sum nor its displayed median threshold adds a strict-positive-total-vote premise: zero weights at epsilon=1/2 and zero total vote remain allowed. The good-candidate comparison uses this total divided by two, exactly as in the source output formula. The comparison is to the displayed approved corrected target bound to FS97-ADABOOST-VARIANTS-ENDPOINT-DOMAIN-06; archival equivalence is not claimed.

Reviewer check: ☐ Matches formalized target

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

AppliedModelingLib.Learning.Online.adaBoostTheorem6Factor

Source locator: cited publication, lines 1099-1108

Verbatim paper-source connection

Theorem 6. Suppose the weak learning algorithm WeakLearn, when called by
AdaBoost, generates hypotheses with errors $\epsilon_1,\ldots,\epsilon_T$
(as defined in Step 3 of Fig. 2). Then the error
$\epsilon=\Pr_{i\sim D}[h_f(x_i)\neq y_i]$ of the final hypothesis h_f
output by AdaBoost is bounded above by
$$
\epsilon\leq 2^T\prod_{t=1}^T\sqrt{\epsilon_t(1-\epsilon_t)}. \tag{14}
$$

[Next verbatim source excerpt]

D(i)=1[U+0012 control character]N. The algorithm maintains a set of weights wt
over
the training examples. On iteration t a distribution pt
is
computed by normalizing these weights. This distribution is
fed to the weak learner WeakLearn which generates a
hypothesis ht that (we hope) has small error with respect to
the distribution.1
Using the new hypothesis ht , the boosting
125
A DECISION THEORETIC GENERALIZATION
1
Some learning algorithms can be generalized to use a given distribution
directly. For instance, gradient based algorithms can use the probability
associated with each example to scale the update step size which is based
on the example. If the algorithm cannot be generalized in this way, the
training sample can be re-sampled to generate a new set of training exam-
ples that is distributed according to the given distribution. The computa-
tion required to generate each re-sampled example takes O(log N) time.

[form-feed in source extraction]
File: 571J 150408 . By:CV . Date:28:07:01 . Time:05:54 LOP8M. V8.0. Page 01:01
Codes: 6184 Signs: 4760 . Length: 56 pic 0 pts, 236 mm
Algorithm AdaBoost
Input: sequence of N labeled examples ((x1 , y1), ..., (xN , yN))
distribution D over the N examples
weak learning algorithm WeakLearn
integer T specifying number of iterations
Initialize the weight vector: w1
i =D(i) for i=1, ..., N.
Do for t=1, 2, ..., T
1. Set
pt
=
wt
[U+001D control character]N
i=1 wt
i
2. Call WeakLearn, providing it with the distribution pt
; get back a
hypothesis ht : X[U+0014 control character] [0, 1].
3. Calculate the error of ht: =t=[U+001D control character]N
i=1 pt
i |ht(xi)& yi |.
4. Set ;t==t [U+0012 control character](1&=t).
5. Set the new weights vector to be
wt+1
i =wt
i ;1&|ht(xi)& yi |
t
Output the hypothesis
hf (x)=
{
1 if [U+001D control character]T
t=1 (log 1[U+0012 control character];t) ht(x)[U+001E control character]1
2 [U+001D control character]T
t=1 log 1[U+0012 control character];t
0 otherwise.
FIG. 2. The adaptive boosting algorithm.
algorithm generates the next weight vector wt+1
, and the
process repeats. After T such iterations, the final hypothesis
hf is output. The hypothesis hf combines the outputs of the
T weak hypotheses using a weighted majority vote.
We call the algorithm AdaBoost because, unlike previous
algorithms, it adjusts adaptively to the errors of the weak
hypotheses returned by WeakLearn. If WeakLearn is a PAC
weak learning algorithm in the sense defined above, then
=t[U+001D control character]1[U+0012 control character]2&# for all t (assuming the examples have been
generated appropriately with yi=c(xi) for some c # C).
However, such a bound on the error need not be known
ahead of time. Our results hold for any =t # [0, 1], and
depend only on the performance of the weak learner on
those distributions that are actually generated during the
boosting process.
The parameter ;t is chosen as a function of =t and is used
for updating the weight vector. The update rule reduces
the probability assigned to those examples on which the

hypothesis makes a good prediction and increases the probability of the examples on which the prediction is poor.2
Note that AdaBoost, unlike boost-by-majority, combines the weak hypotheses by summing their probabilistic predictions. Drucker, Schapire and Simard [9], in experiments they performed using boosting to improve the performance of a real-valued neural network, observed that summing the outcomes of the networks and then selecting the best prediction performs better than selecting the best prediction of each network and then combining them with a majority rule. It is interesting that the new boosting algorithm's final hypothesis uses the same combination rule that was observed to be better in practice, but which previously lacked theoretical justification.
Since it was first introduced, several successful experiments have been conducted using AdaBoost, including work by the authors [12], Drucker and Cortes [8], Jackson and Craven [16], Quinlan [21], and Breiman [3].
4.2. Analysis
Comparing Figs. 1 and 2, there is an obvious similarity between the algorithms Hedge(;) and AdaBoost. This similarity reflects a surprising ``dual'' relationship between the on-line allocation model and the problem of boosting. Put another way, there is a direct mapping or reduction of the boosting problem to the on-line allocation problem. In

Formalized review target

The archival source above is not asserted equivalent to this formalized target.

For the finite Figure-2 AdaBoost execution with every observed error strictly between zero and one, the final hypothesis's training error is at most 2^T
↪ times the product of sqrt(epsilon_t(1-epsilon_t)); errors may lie on either side of one half.

Archival source anchor: cited publication, lines 178-186

Lean-expanded library semantic target

```
type:
{Training : Type u_1} →
  [inst : Fintype Training] →
    [inst_1 : Nonempty Training] →
      (state : AppliedModelingLib.Learning.Online.HedgeWeightState Training) →
        (label : Training → ℝ) →
          (hypotheses : List (Training → ℝ)) →
            AppliedModelingLib.Learning.Online.AdaBoostAdmissible state label hypotheses → ℝ

value:
fun {Training : Type u_1} [Fintype Training] [Nonempty Training]
    (state : AppliedModelingLib.Learning.Online.HedgeWeightState Training) (label : Training → ℝ)
    (hypotheses : List (Training → ℝ))
    (a : AppliedModelingLib.Learning.Online.AdaBoostAdmissible state label hypotheses) =>
  List.brecOn (motive := fun (x : List (Training → ℝ)) =>
    (state : AppliedModelingLib.Learning.Online.HedgeWeightState Training) →
      AppliedModelingLib.Learning.Online.AdaBoostAdmissible state label x → ℝ)
    hypotheses (AppliedModelingLib.Learning.Online.adaBoostTheorem6Factor._f label) state a
```

Exact Lean library declaration

```
/-- The product on the right-hand side of Freund--Schapire's Theorem 6. -/
noncomputable def adaBoostTheorem6Factor
    {Training : Type*} [Fintype Training] [Nonempty Training]
    (state : HedgeWeightState Training) (label : Training → ℝ) :
    (hypotheses : List (Training → ℝ)) →
      AdaBoostAdmissible state label hypotheses → ℝ
  | [], _ => 1
  | hypothesis :: remaining, hvalid =>
      2 * Real.sqrt
          (adaBoostError state label hypothesis *
            (1 - adaBoostError state label hypothesis)) *
        adaBoostTheorem6Factor
          (adaBoostStep state label hypothesis (Classical.choose hvalid)) label remaining
          (Classical.choose_spec hvalid)
```

Direct retained dependencies

- AppliedModelingLib.Learning.Online.AdaBoostAdmissible
- AppliedModelingLib.Learning.Online.HedgeWeightState
- AppliedModelingLib.Learning.Online.adaBoostError
- AppliedModelingLib.Learning.Online.adaBoostStep

Recorded source-to-library screening **Verdict:** matches_approved_corrected_target
Reason: Compared theorem6_adaboost_error's formula (14), 2^T product_t sqrt(epsilon_t(1-epsilon_t)), with the displayed recursive value and error/update dependencies. It returns one for zero rounds and multiplies one factor 2*sqrt(epsilon*(1-epsilon)) per executed round, recomputing epsilon after each adaptive update. This is exactly the printed product for T equal to the list length. The Theorem 6/Equation 21 uses supply normalized initial weights and source labels/hypotheses on the FS97-ADABOOST-ENDPOINT-DOMAIN-03 interior-error domain. Displayed M1/M2 uses feed their source binary reductions; the M2 wrapper separately multiplies by k-1, preserving the Theorem 11 constant. The comparison is to the displayed approved corrected target bound to FS97-ADABOOST-ENDPOINT-DOMAIN-03; archival equivalence is not claimed.
Reviewer check: ☐ Matches formalized target
If left unchecked, explain the discrepancy or uncertainty below.
Reviewer annotation

AppliedModelingLib.Learning.Online.adaBoostVote

Source locator: cited publication, lines 1099-1108

Verbatim paper-source connection

Theorem 6. Suppose the weak learning algorithm WeakLearn, when called by
AdaBoost, generates hypotheses with errors $\epsilon_1,\ldots,\epsilon_T$
(as defined in Step 3 of Fig. 2). Then the error
$\epsilon=\Pr_{i\sim D}[h_f(x_i)\neq y_i]$ of the final hypothesis h_f
output by AdaBoost is bounded above by
$$
\epsilon\leq 2^T\prod_{t=1}^T\sqrt{\epsilon_t(1-\epsilon_t)}. \tag{14}
$$

[Next verbatim source excerpt]

D(i)=1[U+0012 control character]N. The algorithm maintains a set of weights wt
over
the training examples. On iteration t a distribution pt
is
computed by normalizing these weights. This distribution is
fed to the weak learner WeakLearn which generates a
hypothesis ht that (we hope) has small error with respect to
the distribution.1
Using the new hypothesis ht , the boosting
125
A DECISION THEORETIC GENERALIZATION
1
Some learning algorithms can be generalized to use a given distribution
directly. For instance, gradient based algorithms can use the probability
associated with each example to scale the update step size which is based
on the example. If the algorithm cannot be generalized in this way, the
training sample can be re-sampled to generate a new set of training exam-
ples that is distributed according to the given distribution. The computa-
tion required to generate each re-sampled example takes O(log N) time.

[form-feed in source extraction]
File: 571J 150408 . By:CV . Date:28:07:01 . Time:05:54 LOP8M. V8.0. Page 01:01
Codes: 6184 Signs: 4760 . Length: 56 pic 0 pts, 236 mm
Algorithm AdaBoost
Input: sequence of N labeled examples ((x1 , y1), ..., (xN , yN))
distribution D over the N examples
weak learning algorithm WeakLearn
integer T specifying number of iterations
Initialize the weight vector: w1
i =D(i) for i=1, ..., N.
Do for t=1, 2, ..., T
1. Set
pt
=
wt
[U+001D control character]N
i=1 wt
i
2. Call WeakLearn, providing it with the distribution pt
; get back a
hypothesis ht : X[U+0014 control character] [0, 1].
3. Calculate the error of ht: =t=[U+001D control character]N
i=1 pt
i |ht(xi)& yi |.
4. Set ;t==t [U+0012 control character](1&=t).
5. Set the new weights vector to be
wt+1
i =wt
i ;1&|ht(xi)& yi |
t
Output the hypothesis
hf (x)=
{
1 if [U+001D control character]T
t=1 (log 1[U+0012 control character];t) ht(x)[U+001E control character]1
2 [U+001D control character]T
t=1 log 1[U+0012 control character];t
0 otherwise.
FIG. 2. The adaptive boosting algorithm.
algorithm generates the next weight vector wt+1
, and the
process repeats. After T such iterations, the final hypothesis
hf is output. The hypothesis hf combines the outputs of the
T weak hypotheses using a weighted majority vote.
We call the algorithm AdaBoost because, unlike previous
algorithms, it adjusts adaptively to the errors of the weak
hypotheses returned by WeakLearn. If WeakLearn is a PAC
weak learning algorithm in the sense defined above, then
=t[U+001D control character]1[U+0012 control character]2&# for all t (assuming the examples have been
generated appropriately with yi=c(xi) for some c # C).
However, such a bound on the error need not be known
ahead of time. Our results hold for any =t # [0, 1], and
depend only on the performance of the weak learner on
those distributions that are actually generated during the
boosting process.
The parameter ;t is chosen as a function of =t and is used
for updating the weight vector. The update rule reduces
the probability assigned to those examples on which the

hypothesis makes a good prediction and increases the probability of the examples on which the prediction is poor.2
Note that AdaBoost, unlike boost-by-majority, combines the weak hypotheses by summing their probabilistic predictions. Drucker, Schapire and Simard [9], in experiments they performed using boosting to improve the performance of a real-valued neural network, observed that summing the outcomes of the networks and then selecting the best prediction performs better than selecting the best prediction of each network and then combining them with a majority rule. It is interesting that the new boosting algorithm's final hypothesis uses the same combination rule that was observed to be better in practice, but which previously lacked theoretical justification.
Since it was first introduced, several successful experiments have been conducted using AdaBoost, including work by the authors [12], Drucker and Cortes [8], Jackson and Craven [16], Quinlan [21], and Breiman [3].
4.2. Analysis
Comparing Figs. 1 and 2, there is an obvious similarity between the algorithms Hedge(;) and AdaBoost. This similarity reflects a surprising ``dual'' relationship between the on-line allocation model and the problem of boosting. Put another way, there is a direct mapping or reduction of the boosting problem to the on-line allocation problem. In

Formalized review target

The archival source above is not asserted equivalent to this formalized target.

For the finite Figure-2 AdaBoost execution with every observed error strictly between zero and one, the final hypothesis's training error is at most 2^T
↪ times the product of sqrt(epsilon_t(1-epsilon_t)); errors may lie on either side of one half.

Archival source anchor: cited publication, lines 178-186

Lean-expanded library semantic target

```
type:
{Training : Type u_1} →
  [inst : Fintype Training] →
    [inst_1 : Nonempty Training] →
      (state : AppliedModelingLib.Learning.Online.HedgeWeightState Training) →
        (label : Training → ℝ) →
          (hypotheses : List (Training → ℝ)) →
            AppliedModelingLib.Learning.Online.AdaBoostAdmissible state label hypotheses → Training → ℝ

value:
fun {Training : Type u_1} [Fintype Training] [Nonempty Training]
    (state : AppliedModelingLib.Learning.Online.HedgeWeightState Training) (label : Training → ℝ)
    (hypotheses : List (Training → ℝ))
    (a : AppliedModelingLib.Learning.Online.AdaBoostAdmissible state label hypotheses) (a_1 : Training) =>
  List.brecOn (motive := fun (x : List (Training → ℝ)) =>
    (state : AppliedModelingLib.Learning.Online.HedgeWeightState Training) →
      AppliedModelingLib.Learning.Online.AdaBoostAdmissible state label x → Training → ℝ)
    hypotheses (AppliedModelingLib.Learning.Online.adaBoostVote._f label) state a a_1
```

Exact Lean library declaration

```
/-- The unthresholded weighted vote of Figure 2. -/
noncomputable def adaBoostVote
    {Training : Type*} [Fintype Training] [Nonempty Training]
    (state : HedgeWeightState Training) (label : Training → ℝ) :
    (hypotheses : List (Training → ℝ)) →
      AdaBoostAdmissible state label hypotheses → Training → ℝ
  | [], _, _ => 0
  | hypothesis :: remaining, hvalid, sample =>
      adaBoostVoteWeight (adaBoostError state label hypothesis) * hypothesis sample +
        adaBoostVote
          (adaBoostStep state label hypothesis (Classical.choose hvalid)) label remaining
          (Classical.choose_spec hvalid) sample
```

Direct retained dependencies

- AppliedModelingLib.Learning.Online.AdaBoostAdmissible
- AppliedModelingLib.Learning.Online.HedgeWeightState
- AppliedModelingLib.Learning.Online.adaBoostError
- AppliedModelingLib.Learning.Online.adaBoostStep
- AppliedModelingLib.Learning.Online.adaBoostVoteWeight

Recorded source-to-library screening **Verdict:** matches_approved_corrected_target
Reason: Compared theorem6_adaboost_error's Figure 2 output sum_t log(1/beta_t) h_t(x) with the displayed vote recursion, adaptive error, and weight update. The function adds exactly log((1-epsilon_t)/epsilon_t)*h_t(sample) at each successive source state and returns zero for an empty sequence. The displayed paper uses supply binary labels and [0,1] weak predictions with normalized initial weights. The pinned FS97-ADABOOST-ENDPOINT-DOMAIN-03 record permits every interior error, including errors above one half and their negative vote coefficients. The Boolean final-hypothesis dependency compares the vote directly with half the coefficient sum and chooses one at equality, precisely Figure 2's rule; Theorem 9 adds its own explicit normalized-vote conditions. The comparison is to the displayed approved corrected target bound to FS97-ADABOOST-ENDPOINT-DOMAIN-03; archival equivalence is not claimed.
Reviewer check: ☐ Matches formalized target
If left unchecked, explain the discrepancy or uncertainty below.
Reviewer annotation

AppliedModelingLib.Learning.Online.adaBoostVoteWeightSum

Source locator: cited publication, lines 1099-1108

Verbatim paper-source connection

Theorem 6. Suppose the weak learning algorithm WeakLearn, when called by
AdaBoost, generates hypotheses with errors $\epsilon_1,\ldots,\epsilon_T$
(as defined in Step 3 of Fig. 2). Then the error
$\epsilon=\Pr_{i\sim D}[h_f(x_i)\neq y_i]$ of the final hypothesis h_f
output by AdaBoost is bounded above by
$$
\epsilon\leq 2^T\prod_{t=1}^T\sqrt{\epsilon_t(1-\epsilon_t)}. \tag{14}
$$

[Next verbatim source excerpt]

D(i)=1[U+0012 control character]N. The algorithm maintains a set of weights wt
over
the training examples. On iteration t a distribution pt
is
computed by normalizing these weights. This distribution is
fed to the weak learner WeakLearn which generates a
hypothesis ht that (we hope) has small error with respect to
the distribution.1
Using the new hypothesis ht , the boosting
125
A DECISION THEORETIC GENERALIZATION
1
Some learning algorithms can be generalized to use a given distribution
directly. For instance, gradient based algorithms can use the probability
associated with each example to scale the update step size which is based
on the example. If the algorithm cannot be generalized in this way, the
training sample can be re-sampled to generate a new set of training exam-
ples that is distributed according to the given distribution. The computa-
tion required to generate each re-sampled example takes O(log N) time.

[form-feed in source extraction]
File: 571J 150408 . By:CV . Date:28:07:01 . Time:05:54 LOP8M. V8.0. Page 01:01
Codes: 6184 Signs: 4760 . Length: 56 pic 0 pts, 236 mm
Algorithm AdaBoost
Input: sequence of N labeled examples ((x1 , y1), ..., (xN , yN))
distribution D over the N examples
weak learning algorithm WeakLearn
integer T specifying number of iterations
Initialize the weight vector: w1
i =D(i) for i=1, ..., N.
Do for t=1, 2, ..., T
1. Set
pt
=
wt
[U+001D control character]N
i=1 wt
i
2. Call WeakLearn, providing it with the distribution pt
; get back a
hypothesis ht : X[U+0014 control character] [0, 1].
3. Calculate the error of ht: =t=[U+001D control character]N
i=1 pt
i |ht(xi)& yi |.
4. Set ;t==t [U+0012 control character](1&=t).
5. Set the new weights vector to be
wt+1
i =wt
i ;1&|ht(xi)& yi |
t
Output the hypothesis
hf (x)=
{
1 if [U+001D control character]T
t=1 (log 1[U+0012 control character];t) ht(x)[U+001E control character]1
2 [U+001D control character]T
t=1 log 1[U+0012 control character];t
0 otherwise.
FIG. 2. The adaptive boosting algorithm.
algorithm generates the next weight vector wt+1
, and the
process repeats. After T such iterations, the final hypothesis
hf is output. The hypothesis hf combines the outputs of the
T weak hypotheses using a weighted majority vote.
We call the algorithm AdaBoost because, unlike previous
algorithms, it adjusts adaptively to the errors of the weak
hypotheses returned by WeakLearn. If WeakLearn is a PAC
weak learning algorithm in the sense defined above, then
=t[U+001D control character]1[U+0012 control character]2&# for all t (assuming the examples have been
generated appropriately with yi=c(xi) for some c # C).
However, such a bound on the error need not be known
ahead of time. Our results hold for any =t # [0, 1], and
depend only on the performance of the weak learner on
those distributions that are actually generated during the
boosting process.
The parameter ;t is chosen as a function of =t and is used
for updating the weight vector. The update rule reduces
the probability assigned to those examples on which the

hypothesis makes a good prediction and increases the probability of the examples on which the prediction is poor.2
Note that AdaBoost, unlike boost-by-majority, combines the weak hypotheses by summing their probabilistic predictions. Drucker, Schapire and Simard [9], in experiments they performed using boosting to improve the performance of a real-valued neural network, observed that summing the outcomes of the networks and then selecting the best prediction performs better than selecting the best prediction of each network and then combining them with a majority rule. It is interesting that the new boosting algorithm's final hypothesis uses the same combination rule that was observed to be better in practice, but which previously lacked theoretical justification.
Since it was first introduced, several successful experiments have been conducted using AdaBoost, including work by the authors [12], Drucker and Cortes [8], Jackson and Craven [16], Quinlan [21], and Breiman [3].
4.2. Analysis
Comparing Figs. 1 and 2, there is an obvious similarity between the algorithms Hedge(;) and AdaBoost. This similarity reflects a surprising ``dual'' relationship between the on-line allocation model and the problem of boosting. Put another way, there is a direct mapping or reduction of the boosting problem to the on-line allocation problem. In

Formalized review target

The archival source above is not asserted equivalent to this formalized target.

For the finite Figure-2 AdaBoost execution with every observed error strictly between zero and one, the final hypothesis's training error is at most 2^T
↪ times the product of sqrt(epsilon_t(1-epsilon_t)); errors may lie on either side of one half.

Archival source anchor: cited publication, lines 178-186

Lean-expanded library semantic target

```
type:
{Training : Type u_1} →
  [inst : Fintype Training] →
    [inst_1 : Nonempty Training] →
      (state : AppliedModelingLib.Learning.Online.HedgeWeightState Training) →
        (label : Training → ℝ) →
          (hypotheses : List (Training → ℝ)) →
            AppliedModelingLib.Learning.Online.AdaBoostAdmissible state label hypotheses → ℝ

value:
fun {Training : Type u_1} [Fintype Training] [Nonempty Training]
    (state : AppliedModelingLib.Learning.Online.HedgeWeightState Training) (label : Training → ℝ)
    (hypotheses : List (Training → ℝ))
    (a : AppliedModelingLib.Learning.Online.AdaBoostAdmissible state label hypotheses) =>
  List.brecOn (motive := fun (x : List (Training → ℝ)) =>
    (state : AppliedModelingLib.Learning.Online.HedgeWeightState Training) →
      AppliedModelingLib.Learning.Online.AdaBoostAdmissible state label x → ℝ)
    hypotheses (AppliedModelingLib.Learning.Online.adaBoostVoteWeightSum._f label) state a
```

Exact Lean library declaration

```
/-- The sum of the final-vote coefficients in Figure 2. -/
noncomputable def adaBoostVoteWeightSum
    {Training : Type*} [Fintype Training] [Nonempty Training]
    (state : HedgeWeightState Training) (label : Training → ℝ) :
    (hypotheses : List (Training → ℝ)) →
      AdaBoostAdmissible state label hypotheses → ℝ
  | [], _ => 0
  | hypothesis :: remaining, hvalid =>
      adaBoostVoteWeight (adaBoostError state label hypothesis) +
        adaBoostVoteWeightSum
          (adaBoostStep state label hypothesis (Classical.choose hvalid)) label remaining
          (Classical.choose_spec hvalid)
```

Direct retained dependencies

- AppliedModelingLib.Learning.Online.AdaBoostAdmissible
- AppliedModelingLib.Learning.Online.HedgeWeightState
- AppliedModelingLib.Learning.Online.adaBoostError
- AppliedModelingLib.Learning.Online.adaBoostStep
- AppliedModelingLib.Learning.Online.adaBoostVoteWeight

Recorded source-to-library screening **Verdict:** matches_approved_corrected_target
Reason: Compared theorem6_adaboost_error's Figure 2 threshold (1/2) sum_t log(1/beta_t) with the displayed sum recursion and current-error/update dependencies. It sums log((1-epsilon_t)/epsilon_t) from the same adaptive trajectory as the weighted vote and has the correct empty-sum value zero. Under FS97-ADABOOST-ENDPOINT-DOMAIN-03 each coefficient is a finite real; the function retains negative coefficients and does not require a positive or nonzero total. The displayed Theorem 6 and Equations 21-23 classifier uses compare with half this sum, allowing cancellation exactly as printed. Only the separately displayed normalized Theorem 9 use requires a nonzero denominator. The comparison is to the displayed approved corrected target bound to FS97-ADABOOST-ENDPOINT-DOMAIN-03; archival equivalence is not claimed.
Reviewer check: ☐ Matches formalized target
If left unchecked, explain the discrepancy or uncertainty below.
Reviewer annotation

AppliedModelingLib.Learning.Online.allocationPolicyCumulativeLossFrom

Source locator: cited publication, lines 462-487

Verbatim paper-source connection

Theorem 3. Let B be an algorithm for the on-line allocation problem with an
arbitrary number of strategies. Suppose that there exist positive real numbers
a and c such that for any number of strategies N and for any sequence
of loss vectors $\boldsymbol\ell^1,\ldots,\boldsymbol\ell^T$,
$$
L_B\leq c\min_i L_i+a\ln N.
$$
Then for all $\beta\in(0,1)$, either
$$
c\geq\frac{\ln(1/\beta)}{1-\beta}
\qquad\text{or}\qquad
a\geq\frac{1}{1-\beta}.
$$

[Next verbatim source excerpt]

We formalize our on-line allocation model as follows. The
allocation agent A has N options or strategies to choose
from; we number these using the integers 1, ..., N. At each
time step t=1, 2, ..., T, the allocator A decides on a distribu-
tion pt
over the strategies; that is pt
i [U+001E control character]0 is the amount
allocated to strategy i, and [U+001D control character]N
i=1 pt
i =1. Each strategy i then
suffers some loss lt
i which is determined by the (possibly
adversarial) ``environment.'' The loss suffered by A is then
[U+001D control character]n
i=1 pt
i lt
i =pt
} lt
, i.e., the average loss of the strategies with
respect to A's chosen allocation rule. We call this loss func-
tion the mixture loss.
In this paper, we always assume that the loss suffered by
any strategy is bounded so that, without loss of generality,
lt
i # [0, 1]. Besides this condition, we make no assumptions
about the form of the loss vectors lt
, or about the manner
in which they are generated; indeed, the adversary's choice
for lt
may even depend on the allocator's chosen mixture pt
.
The goal of the algorithm A is to minimize its cumulative
loss relative to the loss suffered by the best strategy. That is,
A attempts to minimize its net loss
LA&min
i
Li
where
LA= :
T
t=1
pt
} lt
is the total cumulative loss suffered by algorithm A on the
first T trials, and
Li= :
T
t=1
lt
i
is strategy i's cumulative loss.

[Next verbatim source excerpt]

replaced by an integral, and the maximum by a supremum.
The bound given in Eq. (9) can be written as
LHedge(;)[U+001D control character]c min
i
Li+a ln N, (10)
where c=ln(1[U+0012 control character];)[U+0012 control character](1&;) and a=1[U+0012 control character](1&;). Vovk [24]
analyzes prediction algorithms that have performance
bounds of this form, and proves tight upper and lower
bounds for the achievable values of c and a. Using Vovk's
results, we can show that the constants a and c achieved by
Hedge(;) are optimal.

Lean-expanded library semantic target

```
type:
{Expert : Type u_1} →
  [inst : Fintype Expert] →
```

AppliedModelingLib.Learning.Online.FiniteAllocationPolicy Expert → List (Expert → ℝ) → List (Expert → ℝ) → ℝ

value:
```
fun {Expert : Type u_1} [Fintype Expert] (policy : AppliedModelingLib.Learning.Online.FiniteAllocationPolicy Expert)
    (a a_1 : List (Expert → ℝ)) =>
  List.brecOn (motive := fun (x : List (Expert → ℝ)) => List (Expert → ℝ) → ℝ) a_1
    (AppliedModelingLib.Learning.Online.allocationPolicyCumulativeLossFrom._f policy) a
```

Exact Lean library declaration

```
/--
Run an allocation policy on a remaining loss sequence after an already observed
loss history. The current allocation is chosen before the next loss is appended.
-/
noncomputable def allocationPolicyCumulativeLossFrom
    {Expert : Type*} [Fintype Expert] (policy : FiniteAllocationPolicy Expert) :
    List (Expert → ℝ) → List (Expert → ℝ) → ℝ
  | _history, [] => 0
  | history, loss :: losses =>
      allocationMixtureLoss (policy history) loss +
        allocationPolicyCumulativeLossFrom policy (history ++ [loss]) losses
```

Direct retained dependencies

- AppliedModelingLib.Learning.Online.FiniteAllocationPolicy
- AppliedModelingLib.Learning.Online.allocationMixtureLoss

Recorded source-to-library screening **Verdict:** matches

Reason: For each loss vector the policy sees only the preceding history, incurs the source mixture loss sum_i p_i^t ell_i^t, and then appends the current loss. This is the Theorem-3 allocation-policy convention.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

AppliedModelingLib.Learning.Online.hedgeWeightStateCumulativeLoss

Source locator: cited publication, lines 286-304

Verbatim paper-source connection

Lemma 1. For any sequence of loss vectors
$\boldsymbol\ell^1,\ldots,\boldsymbol\ell^T$,
$$
\ln\!\left(\sum_{i=1}^N w_i^{T+1}\right)
\leq -(1-\beta)L_{\operatorname{Hedge}(\beta)}.
$$

[Next verbatim source excerpt]

We formalize our on-line allocation model as follows. The
allocation agent A has N options or strategies to choose
from; we number these using the integers 1, ..., N. At each
time step t=1, 2, ..., T, the allocator A decides on a distribu-
tion pt
over the strategies; that is pt
i [U+001E control character]0 is the amount
allocated to strategy i, and [U+001D control character]N
i=1 pt
i =1. Each strategy i then
suffers some loss lt
i which is determined by the (possibly
adversarial) ``environment.'' The loss suffered by A is then
[U+001D control character]n
i=1 pt
i lt
i =pt
} lt
, i.e., the average loss of the strategies with
respect to A's chosen allocation rule. We call this loss func-
tion the mixture loss.
In this paper, we always assume that the loss suffered by
any strategy is bounded so that, without loss of generality,
lt
i # [0, 1]. Besides this condition, we make no assumptions
about the form of the loss vectors lt
, or about the manner
in which they are generated; indeed, the adversary's choice
for lt
may even depend on the allocator's chosen mixture pt
.
The goal of the algorithm A is to minimize its cumulative
loss relative to the loss suffered by the best strategy. That is,
A attempts to minimize its net loss
LA&min
i
Li
where
LA= :
T
t=1
pt
} lt
is the total cumulative loss suffered by algorithm A on the
first T trials, and
Li= :
T
t=1
lt
i
is strategy i's cumulative loss.

[Next verbatim source excerpt]

is updated using the multiplicative rule
wt+1
i =wt
i } ;li
t
. (2)
More generally, it can be shown that our analysis is
applicable with only minor modification to an alternative
update rule of the form
wt+1
i =wt
i } U;(lt
i)
where U; : [0, 1] [U+0014 control character] [0, 1] is any function, parameterized
by ; # [0, 1] satisfying
;r
[U+001D control character]U;(r)[U+001D control character]1&(1&;) r
for all r # [0, 1].
2.1. Analysis
The analysis of Hedge(;) mimics directly that given by
Littlestone and Warmuth [20]. The main idea is to derive
upper and lower bounds on [U+001D control character]N
i=1 wT+1
i which, together,
imply an upper bound on the loss of the algorithm. We
begin with an upper bound.

Lemma 1. For any sequence of loss vectors l1
, ..., lT
,
ln
\:
N
i=1
wT+1
i
+[U+001D control character]&(1&;) LHedge(;) .
Algorithm Hedge(;)
Parameters: ; # [0, 1]
initial weight vector w1
[0, 1]N
with [U+001D control character]N
i=1 w1
i =1
number of trials T
Do for t=1, 2, ..., T
1. Choose allocation
pt
=
wt
[U+001D control character]N
i=1 wt
i
2. Receive loss vector lt
[0, 1]N
from environment.
3. Suffer loss pt
} lt
.
4. Set the new weights vector to be
wt+1
i =wt
i ;li
t
FIG. 1. The on-line allocation algorithm.
Proof. By a convexity argument, it can be shown that
:r
[U+001D control character]1&(1&:) r (3)
for :[U+001E control character]0 and r # [0, 1]. Combined with Eqs. (1) and (2),
this implies
:
N

Formalized review target

The archival source above is not asserted equivalent to this formalized target.

For a finite nonempty strategy family, normalized initial weights, losses in [0,1], and 0 < beta <= 1, the logarithm of total final Hedge weight is at most -(1-beta) times Hedge's cumulative mixture loss.

Archival source anchor: cited publication, lines 46-65

Lean-expanded library semantic target

```
type:
{Action : Type u_1} →
  [inst : Fintype Action] →
    [Nonempty Action] →
      (discount : ℝ) → 0 < discount → AppliedModelingLib.Learning.Online.HedgeWeightState Action → List (Action → ℝ) → ℝ

value:
fun {Action : Type u_1} [Fintype Action] [Nonempty Action] (discount : ℝ) (hdiscount : 0 < discount)
    (a : AppliedModelingLib.Learning.Online.HedgeWeightState Action) (a_1 : List (Action → ℝ)) =>
  List.brecOn (motive := fun (x : List (Action → ℝ)) => AppliedModelingLib.Learning.Online.HedgeWeightState Action → ℝ)
    a_1 (AppliedModelingLib.Learning.Online.hedgeWeightStateCumulativeLoss._f discount hdiscount) a
```

Exact Lean library declaration

```
/-- Cumulative allocation loss along a finite source Hedge trajectory. -/
noncomputable def hedgeWeightStateCumulativeLoss
    {Action : Type*} [Fintype Action] [Nonempty Action]
    (discount : ℝ) (hdiscount : 0 < discount) :
    HedgeWeightState Action → List (Action → ℝ) → ℝ
  | _, [] => 0
  | state, loss :: remaining =>
      by
        classical
        exact state.expectation loss +
          hedgeWeightStateCumulativeLoss discount hdiscount
            (hedgeWeightStateStep state loss discount hdiscount) remaining
```

Direct retained dependencies

- AppliedModelingLib.Learning.Online.HedgeWeightState
- AppliedModelingLib.Learning.Online.HedgeWeightState.expectation
- AppliedModelingLib.Learning.Online.hedgeWeightStateStep

Recorded source-to-library screening **Verdict:** matches_approved_corrected_target
Reason: Compared lemma1_weight_potential's definition L_Hedge=sum_t p^t dot ell^t and Figure 1 with the displayed recursion, expectation, finite PMF normalization, and multiplicative step. The empty loss is zero; a nonempty trajectory first adds sum_i (w_i/sum_j w_j)*ell_i, then updates w_i to w_i*beta^ell_i before the remaining losses. Thus allocations are taken from the correct pre-update state at every round. The displayed Lemma 1 use restricts beta to 0<beta<=1, total initial weight to one, and losses to [0,1], honoring FS97-HEDGE-ENDPOINT-DOMAIN-01; the displayed Theorem 2 use further requires beta<1. Individual zero initial weights remain permitted. The comparison is to the displayed approved corrected target bound to FS97-HEDGE-ENDPOINT-DOMAIN-01; archival equivalence is not claimed.
Reviewer check: ☐ Matches formalized target
If left unchecked, explain the discrepancy or uncertainty below.
Reviewer annotation

AppliedModelingLib.Learning.Online.hedgeWeightStateRun

Source locator: cited publication, lines 286-304

Verbatim paper-source connection

Lemma 1. For any sequence of loss vectors
$\boldsymbol\ell^1,\ldots,\boldsymbol\ell^T$,
$$
\ln\!\left(\sum_{i=1}^N w_i^{T+1}\right)
\leq -(1-\beta)L_{\operatorname{Hedge}(\beta)}.
$$

[Next verbatim source excerpt]

We formalize our on-line allocation model as follows. The
allocation agent A has N options or strategies to choose
from; we number these using the integers 1, ..., N. At each
time step t=1, 2, ..., T, the allocator A decides on a distribu-
tion pt
over the strategies; that is pt
i [U+001E control character]0 is the amount
allocated to strategy i, and [U+001D control character]N
i=1 pt
i =1. Each strategy i then
suffers some loss lt
i which is determined by the (possibly
adversarial) ``environment.'' The loss suffered by A is then
[U+001D control character]n
i=1 pt
i lt
i =pt
} lt
, i.e., the average loss of the strategies with
respect to A's chosen allocation rule. We call this loss func-
tion the mixture loss.
In this paper, we always assume that the loss suffered by
any strategy is bounded so that, without loss of generality,
lt
i # [0, 1]. Besides this condition, we make no assumptions
about the form of the loss vectors lt
, or about the manner
in which they are generated; indeed, the adversary's choice
for lt
may even depend on the allocator's chosen mixture pt
.
The goal of the algorithm A is to minimize its cumulative
loss relative to the loss suffered by the best strategy. That is,
A attempts to minimize its net loss
LA&min
i
Li
where
LA= :
T
t=1
pt
} lt
is the total cumulative loss suffered by algorithm A on the
first T trials, and
Li= :
T
t=1
lt
i
is strategy i's cumulative loss.

[Next verbatim source excerpt]

is updated using the multiplicative rule
wt+1
i =wt
i } ;li
t
. (2)
More generally, it can be shown that our analysis is
applicable with only minor modification to an alternative
update rule of the form
wt+1
i =wt
i } U;(lt
i)
where U; : [0, 1] [U+0014 control character] [0, 1] is any function, parameterized
by ; # [0, 1] satisfying
;r
[U+001D control character]U;(r)[U+001D control character]1&(1&;) r
for all r # [0, 1].
2.1. Analysis
The analysis of Hedge(;) mimics directly that given by
Littlestone and Warmuth [20]. The main idea is to derive
upper and lower bounds on [U+001D control character]N
i=1 wT+1
i which, together,
imply an upper bound on the loss of the algorithm. We
begin with an upper bound.

Lemma 1. For any sequence of loss vectors l1
, ..., lT
,
ln
\:
N
i=1
wT+1
i
+[U+001D control character]&(1&;) LHedge(;) .
Algorithm Hedge(;)
Parameters: ; # [0, 1]
initial weight vector w1
[0, 1]N
with [U+001D control character]N
i=1 w1
i =1
number of trials T
Do for t=1, 2, ..., T
1. Choose allocation
pt
=
wt
[U+001D control character]N
i=1 wt
i
2. Receive loss vector lt
[0, 1]N
from environment.
3. Suffer loss pt
} lt
.
4. Set the new weights vector to be
wt+1
i =wt
i ;li
t
FIG. 1. The on-line allocation algorithm.
Proof. By a convexity argument, it can be shown that
:r
[U+001D control character]1&(1&:) r (3)
for :[U+001E control character]0 and r # [0, 1]. Combined with Eqs. (1) and (2),
this implies
:
N

Formalized review target

The archival source above is not asserted equivalent to this formalized target.

For a finite nonempty strategy family, normalized initial weights, losses in [0,1], and 0 < beta <= 1, the logarithm of total final Hedge weight is at most -(1-beta) times Hedge's cumulative mixture loss.

Archival source anchor: cited publication, lines 46-65

Lean-expanded library semantic target

```
type:
{Action : Type u_1} →
  [inst : Fintype Action] →
    [Nonempty Action] →
      (discount : ℝ) →
        0 < discount →
          AppliedModelingLib.Learning.Online.HedgeWeightState Action →
            List (Action → ℝ) → AppliedModelingLib.Learning.Online.HedgeWeightState Action

value:
fun {Action : Type u_1} [Fintype Action] [Nonempty Action] (discount : ℝ) (hdiscount : 0 < discount)
    (a : AppliedModelingLib.Learning.Online.HedgeWeightState Action) (a_1 : List (Action → ℝ)) =>
  List.brecOn (motive := fun (x : List (Action → ℝ)) =>
    AppliedModelingLib.Learning.Online.HedgeWeightState Action →
      AppliedModelingLib.Learning.Online.HedgeWeightState Action)
    a_1 (AppliedModelingLib.Learning.Online.hedgeWeightStateRun._f discount hdiscount) a
```

Exact Lean library declaration

```
/-- Execute a finite sequence of source Hedge weight updates. -/
noncomputable def hedgeWeightStateRun
    {Action : Type*} [Fintype Action] [Nonempty Action]
    (discount : ℝ) (hdiscount : 0 < discount) :
    HedgeWeightState Action → List (Action → ℝ) → HedgeWeightState Action
  | state, [] => state
  | state, loss :: remaining =>
      hedgeWeightStateRun discount hdiscount
        (hedgeWeightStateStep state loss discount hdiscount) remaining
```

Direct retained dependencies

- AppliedModelingLib.Learning.Online.HedgeWeightState
- AppliedModelingLib.Learning.Online.hedgeWeightStateStep

Recorded source-to-library screening **Verdict:** matches_approved_corrected_target
Reason: Compared lemma1_weight_potential's final weights w^(T+1) and Figure 1 Step 4 with the displayed state-run recursion and step value. The empty list returns the original state, and each loss vector produces component weights w_i*beta^(ell_i) before processing the remaining vectors, giving exactly T updates for a list of length T. The displayed Lemma 1 use supplies normalized initial nonnegative weights, losses in [0,1], and 0<beta<=1 under FS97-HEDGE-ENDPOINT-DOMAIN-01. Positive total mass is preserved while zero individual weights remain allowed; beta=1 and the zero-round potential are both retained. The library's broader positive-beta input domain does not broaden the paper claim. The comparison is to the displayed approved corrected target bound to FS97-HEDGE-ENDPOINT-DOMAIN-01; archival equivalence is not claimed.
Reviewer check: ☐ Matches formalized target
If left unchecked, explain the discrepancy or uncertainty below.
Reviewer annotation

Source claims

Review row 1

Source locator: cited publication, lines 286-304

Verbatim source input

Lemma 1. For any sequence of loss vectors
$\boldsymbol\ell^1,\ldots,\boldsymbol\ell^T$,
$$
\ln\!\left(\sum_{i=1}^N w_i^{T+1}\right)
\leq -(1-\beta)L_{\operatorname{Hedge}(\beta)}.
$$

[Next verbatim source excerpt]

We formalize our on-line allocation model as follows. The
allocation agent A has N options or strategies to choose
from; we number these using the integers 1, ..., N. At each
time step t=1, 2, ..., T, the allocator A decides on a distribu-
tion pt
over the strategies; that is pt
i [U+001E control character]0 is the amount
allocated to strategy i, and [U+001D control character]N
i=1 pt
i =1. Each strategy i then
suffers some loss lt
i which is determined by the (possibly
adversarial) ``environment.'' The loss suffered by A is then
[U+001D control character]n
i=1 pt
i lt
i =pt
} lt
, i.e., the average loss of the strategies with
respect to A's chosen allocation rule. We call this loss func-
tion the mixture loss.
In this paper, we always assume that the loss suffered by
any strategy is bounded so that, without loss of generality,
lt
i # [0, 1]. Besides this condition, we make no assumptions
about the form of the loss vectors lt
, or about the manner
in which they are generated; indeed, the adversary's choice
for lt
may even depend on the allocator's chosen mixture pt
.
The goal of the algorithm A is to minimize its cumulative
loss relative to the loss suffered by the best strategy. That is,
A attempts to minimize its net loss
LA&min
i
Li
where
LA= :
T
t=1
pt
} lt
is the total cumulative loss suffered by algorithm A on the
first T trials, and
Li= :
T
t=1
lt
i
is strategy i's cumulative loss.

[Next verbatim source excerpt]

is updated using the multiplicative rule
wt+1
i =wt
i } ;li
t
. (2)
More generally, it can be shown that our analysis is
applicable with only minor modification to an alternative
update rule of the form
wt+1
i =wt
i } U;(lt
i)
where U; : [0, 1] [U+0014 control character] [0, 1] is any function, parameterized
by ; # [0, 1] satisfying
;r
[U+001D control character]U;(r)[U+001D control character]1&(1&;) r
for all r # [0, 1].
2.1. Analysis
The analysis of Hedge(;) mimics directly that given by
Littlestone and Warmuth [20]. The main idea is to derive
upper and lower bounds on [U+001D control character]N

i=1 wT+1
i which, together,
imply an upper bound on the loss of the algorithm. We
begin with an upper bound.
Lemma 1. For any sequence of loss vectors l1
, ..., lT
,
ln
\:
N
i=1
wT+1
i
+[U+001D control character]&(1&;) LHedge(;) .
Algorithm Hedge(;)
Parameters: ; # [0, 1]
initial weight vector w1
[0, 1]N
with [U+001D control character]N
i=1 w1
i =1
number of trials T
Do for t=1, 2, ..., T
1. Choose allocation
pt
=
wt
[U+001D control character]N
i=1 wt
i
2. Receive loss vector lt
[0, 1]N
from environment.
3. Suffer loss pt
} lt
.
4. Set the new weights vector to be
wt+1
i =wt
i ;li
t
FIG. 1. The on-line allocation algorithm.
Proof. By a convexity argument, it can be shown that
:r
[U+001D control character]1&(1&:) r (3)
for :[U+001E control character]0 and r # [0, 1]. Combined with Eqs. (1) and (2),
this implies
:
N

Formalized review target

The archival source above is not asserted equivalent to this formalized target.

For a finite nonempty strategy family, normalized initial weights, losses in [0,1], and 0 < beta <= 1, the logarithm of total final Hedge weight is at most -(1-beta) times Hedge's cumulative mixture loss.

Archival source anchor: cited publication, lines 46-65

Expanded PaperInterface specification

```
∀ {Action : Type} [inst : Fintype Action] [inst_1 : Nonempty Action] (discount : ℝ) (hdiscount : 0 < discount),
  discount ≤ 1 →
    ∀ (state : AppliedModelingLib.Learning.Online.HedgeWeightState Action),
      ∑ action : Action, state.weight action = 1 →
        ∀ (losses : List (Action → ℝ)),
          (∀ loss ∈ losses, ∀ (action : Action), 0 ≤ loss action ∧ loss action ≤ 1) →
            Real.log
                (∑ action : Action,
                  (AppliedModelingLib.Learning.Online.hedgeWeightStateRun discount hdiscount state losses).weight
                    action) ≤
              -(1 - discount) *
                AppliedModelingLib.Learning.Online.hedgeWeightStateCumulativeLoss discount hdiscount state losses
```

Retained governing declarations

- AppliedModelingLib.Learning.Online.HedgeWeightState
- AppliedModelingLib.Learning.Online.hedgeWeightStateCumulativeLoss
- AppliedModelingLib.Learning.Online.hedgeWeightStateRun

Lean-elaborated claim roles

1. parameter: Type
2. parameter: Fintype Action
3. assumption: Nonempty Action
4. parameter: ℝ
5. assumption: 0 < discount
6. assumption: discount ≤ 1

7. parameter: AppliedModelingLib.Learning.Online.HedgeWeightState Action
8. assumption: ∑ action : Action, state.weight action = 1
9. parameter: List (Action → ℝ)
10. assumption: ∀ loss ∈ losses, ∀ (action : Action), 0 ≤ loss action ∧ loss action ≤ 1
11. conclusion: Real.log
    (∑ action : Action,
      (AppliedModelingLib.Learning.Online.hedgeWeightStateRun discount hdiscount state losses).weight action) ≤
  -(1 - discount) * AppliedModelingLib.Learning.Online.hedgeWeightStateCumulativeLoss discount hdiscount state losses

Lean proof endpoint (built separately)

FreundSchapire1997Hedge.lemma1_finite_weight_potential

Recorded source-to-Spec screening **Verdict:** matches_approved_corrected_target

Reason: Lemma 1’s potential inequality is preserved exactly for the finite Hedge run. The checked target explicitly uses 0 < beta <= 1 because beta = 0 can leave the printed normalized execution undefined after a positive-loss round; this is the documented finite-real source-definedness correction, not an additional economic assumption.

Reviewer check: ☐ Matches formalized target

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 2

Source locator: cited publication, lines 398-412

Verbatim source input

Theorem 2. For any sequence of loss vectors
$\boldsymbol\ell^1,\ldots,\boldsymbol\ell^T$, and for any
$i\in\{1,\ldots,N\}$, we have
$$
L_{\operatorname{Hedge}(\beta)}
\leq \frac{-\ln(w_i^1)-L_i\ln\beta}{1-\beta}. \tag{7}
$$
More generally, for any nonempty set $S\subseteq\{1,\ldots,N\}$, we have
$$
L_{\operatorname{Hedge}(\beta)}
\leq
\frac{-\ln(\sum_{i\in S}w_i^1)-(\ln\beta)\max_{i\in S}L_i}{1-\beta}. \tag{8}
$$

[Next verbatim source excerpt]

We formalize our on-line allocation model as follows. The
allocation agent A has N options or strategies to choose
from; we number these using the integers 1, ..., N. At each
time step t=1, 2, ..., T, the allocator A decides on a distribu-
tion pt
over the strategies; that is pt
i [U+001E control character]0 is the amount
allocated to strategy i, and [U+001D control character]N
i=1 pt
i =1. Each strategy i then
suffers some loss lt
i which is determined by the (possibly
adversarial) ``environment.'' The loss suffered by A is then
[U+001D control character]n
i=1 pt
i lt
i =pt
} lt
, i.e., the average loss of the strategies with
respect to A's chosen allocation rule. We call this loss func-
tion the mixture loss.
In this paper, we always assume that the loss suffered by
any strategy is bounded so that, without loss of generality,
lt
i # [0, 1]. Besides this condition, we make no assumptions
about the form of the loss vectors lt
, or about the manner
in which they are generated; indeed, the adversary's choice
for lt
may even depend on the allocator's chosen mixture pt
.
The goal of the algorithm A is to minimize its cumulative
loss relative to the loss suffered by the best strategy. That is,
A attempts to minimize its net loss
LA&min
i
Li
where
LA= :
T
t=1
pt
} lt
is the total cumulative loss suffered by algorithm A on the
first T trials, and
Li= :
T
t=1
lt
i
is strategy i's cumulative loss.

[Next verbatim source excerpt]

is updated using the multiplicative rule
wt+1
i =wt
i } ;li
t
. (2)
More generally, it can be shown that our analysis is
applicable with only minor modification to an alternative
update rule of the form
wt+1
i =wt
i } U;(lt
i)
where U; : [0, 1] [U+0014 control character] [0, 1] is any function, parameterized
by ; # [0, 1] satisfying
;r
[U+001D control character]U;(r)[U+001D control character]1&(1&;) r
for all r # [0, 1].

2.1. Analysis
The analysis of Hedge(;) mimics directly that given by
Littlestone and Warmuth [20]. The main idea is to derive
upper and lower bounds on [U+001D control character]N
i=1 wT+1
i which, together,
imply an upper bound on the loss of the algorithm. We
begin with an upper bound.
Lemma 1. For any sequence of loss vectors l1
, ..., lT
,
ln
\:
N
i=1
wT+1
i
+[U+001D control character]&(1&;) LHedge(;) .
Algorithm Hedge(;)
Parameters: ; # [0, 1]
initial weight vector w1
[0, 1]N
with [U+001D control character]N
i=1 w1
i =1
number of trials T
Do for t=1, 2, ..., T
1. Choose allocation
pt
=
wt
[U+001D control character]N
i=1 wt
i
2. Receive loss vector lt
[0, 1]N
from environment.
3. Suffer loss pt
} lt
.
4. Set the new weights vector to be
wt+1
i =wt
i ;li
t
FIG. 1. The on-line allocation algorithm.
Proof. By a convexity argument, it can be shown that
:r
[U+001D control character]1&(1&:) r (3)
for :[U+001E control character]0 and r # [0, 1]. Combined with Eqs. (1) and (2),
this implies
:
N

Formalized review target

The archival source above is not asserted equivalent to this formalized target.

For a finite nonempty strategy family, normalized initial weights, losses in [0,1], and 0 < beta < 1, Hedge's cumulative loss satisfies the printed single-strategy and nonempty-subset bounds, with zero initial comparator mass represented by positive infinity.

Archival source anchor: cited publication, lines 67-80

Expanded PaperInterface specification

```
∀ {Action : Type} [inst : Fintype Action] [inst_1 : Nonempty Action] (discount : ℝ) (hdiscount : 0 < discount),
  discount < 1 →
    ∀ (state : AppliedModelingLib.Learning.Online.HedgeWeightState Action),
      ∑ action : Action, state.weight action = 1 →
        ∀ (losses : List (Action → ℝ)),
          (∀ loss ∈ losses, ∀ (action : Action), 0 ≤ loss action ∧ loss action ≤ 1) →
            (∀ (action : Action),
                ↑(AppliedModelingLib.Learning.Online.hedgeWeightStateCumulativeLoss discount hdiscount state losses) ≤
                  AppliedModelingLib.Learning.Online.hedgeWeightStateSubsetComparatorExtendedBound discount
                    (state.weight action) (List.map (fun (loss : Action → ℝ) => loss action) losses).sum) ∧
              ∀ (selected : Finset Action) (hselected : selected.Nonempty),
                ↑(AppliedModelingLib.Learning.Online.hedgeWeightStateCumulativeLoss discount hdiscount state losses) ≤
                  AppliedModelingLib.Learning.Online.hedgeWeightStateSubsetComparatorExtendedBound discount
                    (∑ action ∈ selected, state.weight action)
                    (selected.sup' hselected fun (action : Action) =>
                      (List.map (fun (loss : Action → ℝ) => loss action) losses).sum)
```

Retained governing declarations

- AppliedModelingLib.Learning.Online.HedgeWeightState
- AppliedModelingLib.Learning.Online.hedgeWeightStateCumulativeLoss
- AppliedModelingLib.Learning.Online.hedgeWeightStateSubsetComparatorExtendedBound

Lean-elaborated claim roles

1. parameter: Type
2. parameter: Fintype Action
3. assumption: Nonempty Action
4. parameter: ℝ
5. assumption: 0 < discount
6. assumption: discount < 1
7. parameter: AppliedModelingLib.Learning.Online.HedgeWeightState Action
8. assumption: ∑ action : Action, state.weight action = 1
9. parameter: List (Action → ℝ)
10. assumption: ∀ loss ∈ losses, ∀ (action : Action), 0 ≤ loss action ∧ loss action ≤ 1
11. conclusion: (∀ (action : Action),
    ↑(AppliedModelingLib.Learning.Online.hedgeWeightStateCumulativeLoss discount hdiscount state losses) ≤
     AppliedModelingLib.Learning.Online.hedgeWeightStateSubsetComparatorExtendedBound discount (state.weight action)
       (List.map (fun (loss : Action → ℝ) => loss action) losses).sum) ∧
  ∀ (selected : Finset Action) (hselected : selected.Nonempty),
    ↑(AppliedModelingLib.Learning.Online.hedgeWeightStateCumulativeLoss discount hdiscount state losses) ≤
     AppliedModelingLib.Learning.Online.hedgeWeightStateSubsetComparatorExtendedBound discount
       (∑ action ∈ selected, state.weight action)
       (selected.sup' hselected fun (action : Action) =>
         (List.map (fun (loss : Action → ℝ) => loss action) losses).sum)

Lean proof endpoint (built separately)

FreundSchapire1997Hedge.theorem2_hedge_comparator_bounds

Recorded source-to-Spec screening **Verdict:** matches_approved_corrected_target

Reason: The Spec contains both printed comparator inequalities, including the arbitrary singleton and nonempty-subset forms. It uses 0 < beta < 1 for the source logarithm and denominator, and represents zero comparator mass by the stated extended-real branch; this is the documented finite-real reading of Eqs. (7)--(8).

Reviewer check: ☐ Matches formalized target

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 3

Source locator: cited publication, lines 462-487

Verbatim source input

Theorem 3. Let B be an algorithm for the on-line allocation problem with an
arbitrary number of strategies. Suppose that there exist positive real numbers
a and c such that for any number of strategies N and for any sequence
of loss vectors $\boldsymbol\ell^1,\ldots,\boldsymbol\ell^T$,
$$
L_B\leq c\min_i L_i+a\ln N.
$$
Then for all $\beta\in(0,1)$, either
$$
c\geq\frac{\ln(1/\beta)}{1-\beta}
\qquad\text{or}\qquad
a\geq\frac{1}{1-\beta}.
$$

[Next verbatim source excerpt]

We formalize our on-line allocation model as follows. The
allocation agent A has N options or strategies to choose
from; we number these using the integers 1, ..., N. At each
time step t=1, 2, ..., T, the allocator A decides on a distribu-
tion pt
over the strategies; that is pt
i [U+001E control character]0 is the amount
allocated to strategy i, and [U+001D control character]N
i=1 pt
i =1. Each strategy i then
suffers some loss lt
i which is determined by the (possibly
adversarial) ``environment.'' The loss suffered by A is then
[U+001D control character]n
i=1 pt
i lt
i =pt
} lt
, i.e., the average loss of the strategies with
respect to A's chosen allocation rule. We call this loss func-
tion the mixture loss.
In this paper, we always assume that the loss suffered by
any strategy is bounded so that, without loss of generality,
lt
i # [0, 1]. Besides this condition, we make no assumptions
about the form of the loss vectors lt
, or about the manner
in which they are generated; indeed, the adversary's choice
for lt
may even depend on the allocator's chosen mixture pt
.
The goal of the algorithm A is to minimize its cumulative
loss relative to the loss suffered by the best strategy. That is,
A attempts to minimize its net loss
LA&min
i
Li
where
LA= :
T
t=1
pt
} lt
is the total cumulative loss suffered by algorithm A on the
first T trials, and
Li= :
T
t=1
lt
i
is strategy i's cumulative loss.

[Next verbatim source excerpt]

replaced by an integral, and the maximum by a supremum.
The bound given in Eq. (9) can be written as
LHedge(;)[U+001D control character]c min
i
Li+a ln N, (10)
where c=ln(1[U+0012 control character];)[U+0012 control character](1&;) and a=1[U+0012 control character](1&;). Vovk [24]
analyzes prediction algorithms that have performance
bounds of this form, and proves tight upper and lower
bounds for the achievable values of c and a. Using Vovk's
results, we can show that the constants a and c achieved by
Hedge(;) are optimal.

Expanded PaperInterface specification

∀ {discount c a : ℝ},
  0 < discount →

```
discount < 1 →
  0 < c →
    0 < a →
      AppliedModelingLib.Learning.Online.globalFiniteAllocationPolicyBounded c a →
        -Real.log discount / (1 - discount) ≤ c ∨ 1 / (1 - discount) ≤ a
```

Retained governing declarations

- AppliedModelingLib.Learning.Online.globalFiniteAllocationPolicyBounded

Lean-elaborated claim roles

1. parameter: ℝ
2. parameter: ℝ
3. parameter: ℝ
4. assumption: 0 < discount
5. assumption: discount < 1
6. assumption: 0 < c
7. assumption: 0 < a
8. assumption: AppliedModelingLib.Learning.Online.globalFiniteAllocationPolicyBounded c a
9. conclusion: -Real.log discount / (1 - discount) ≤ c ∨ 1 / (1 - discount) ≤ a

Lean proof endpoint (built separately)

FreundSchapire1997Hedge.theorem3_global_allocation_lower_tradeoff_checked

Recorded source-to-Spec screening **Verdict:** matches

Reason: The source statement assumes positive a and c and a uniform finite-strategy allocation bound, then concludes at every interior beta that c or a exceeds the stated Vovk threshold. The Spec has exactly that global finite-policy premise and either-or conclusion, with log(1/beta) rendered as -log beta.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 4

Source locator: cited publication, lines 488-498

Verbatim source input

Lemma 4. Suppose $0\leq L\leq\widetilde L$ and
$0<R\leq\widetilde R$. Let
$\beta=g(\widetilde L/\widetilde R)$ where
$g(z)=1/(1+\sqrt{2/z})$. Then
$$
\frac{-L\ln\beta+R}{1-\beta}
\leq L+\sqrt{2\widetilde L\widetilde R}+R.
$$

Formalized review target

The archival source above is not asserted equivalent to this formalized target.

For nonnegative loss L no larger than lossBound and positive reward R no larger than rewardBound, use the continuous extension of beta = g(lossBound/rewardBound) that sets beta to zero when lossBound is zero; then (-L log beta + R)/(1-beta) is at most L + sqrt(2 lossBound rewardBound) + R.

Archival source anchor: cited publication, lines 107-114

Expanded PaperInterface specification

```
∀ (loss lossBound regret regretBound : ℝ),
  0 ≤ loss →
    loss ≤ lossBound →
      0 < regret →
        regret ≤ regretBound →
          (-loss * Real.log (AppliedModelingLib.Learning.Online.hedgeDiscountFromBoundsClosed lossBound regretBound) +
              regret) /
            (1 - AppliedModelingLib.Learning.Online.hedgeDiscountFromBoundsClosed lossBound regretBound) ≤
          loss + √(2 * lossBound * regretBound) + regret
```

Retained governing declarations

- AppliedModelingLib.Learning.Online.hedgeDiscountFromBoundsClosed

Lean-elaborated claim roles

1. parameter: ℝ
2. parameter: ℝ
3. parameter: ℝ
4. parameter: ℝ
5. assumption: 0 ≤ loss
6. assumption: loss ≤ lossBound
7. assumption: 0 < regret
8. assumption: regret ≤ regretBound
9. conclusion: (-loss * Real.log (AppliedModelingLib.Learning.Online.hedgeDiscountFromBoundsClosed lossBound regretBound) + regret) / (1 - AppliedModelingLib.Learning.Online.hedgeDiscountFromBoundsClosed lossBound regretBound) ≤ loss + √(2 * lossBound * regretBound) + regret

Lean proof endpoint (built separately)

FreundSchapire1997Hedge.lemma4_learning_rate_bound

Recorded source-to-Spec screening

Verdict: matches_approved_corrected_target
Reason: The approved corrected target controls this row because the archival formula does not assign a finite-real value at lossBound = 0. The expanded Spec has exactly 0 <= loss <= lossBound and 0 < regret <= regretBound and uses beta = 0 when lossBound = 0, otherwise beta = 1/(1+sqrt(2*regretBound/lossBound)), which is algebraically g(lossBound/regretBound) on the positive branch. At the added zero-bound branch the assumptions force loss = 0 and the expanded inequality reduces to regret <= regret. The logarithmic fraction, square-root term, constants, and inequality direction therefore match the approved closed-endpoint correction FS97-LEMMA4-ZERO-LOSS-BOUND-05 without claiming literal archival identity.
Reviewer check: ☐ Matches formalized target
If left unchecked, explain the discrepancy or uncertainty below.
Reviewer annotation

Review row 5

Source locator: cited publication, lines 625-645

Verbatim source input

Theorem 5. For any loss function λ, for any set of experts, and for
any sequence of outcomes, the expected loss of $\operatorname{Hedge}(\beta)$
if used as described above is at most
$$
\sum_{t=1}^T\Lambda(\mathcal D^t,\omega^t)
\leq \min_i\sum_{t=1}^T\Lambda(\mathcal E_i^t,\omega^t)
+\sqrt{2\widetilde L\ln N}+\ln N,
$$
where $\widetilde L\leq T$ is an assumed bound on the expected loss of the
best expert, and $\beta=g(\widetilde L/\ln N)$.

[Next verbatim source excerpt]

O(- (ln N)[U+0012 control character]T). However, if L
[U+0019 control character] is close to zero, then the rate
of convergence will be much faster, roughly, O((ln N)[U+0012 control character]T).
Lemma 4 can also be applied to the other bounds given in
Theorem 2 to obtain analogous results.
The bound given in Eq. (11) can be improved in special
cases in which the loss is a function of a prediction and an
outcome and this function is of a special form (see Example 4
below). However, for the general case, one cannot improve
the square-root term - 2L
[U+0019 control character] ln N, by more than a constant
factor. This is a corollary of the lower bound given by Cesa-
Bianchi et al. ([4], Theorem 7) who analyze an on-line
prediction problem that can be seen as a special case of the
on-line allocation model.
3. APPLICATIONS
The framework described up to this point is quite general
and can be applied in a wide variety of learning problems.
Consider the following set-up used by Chung [5]. We are
given a decision space 2, a space of outcomes 0, and a
bounded loss function *: 2_0 [U+0014 control character] [0, 1]. (Actually, our
results require only that * be bounded, but, by rescaling, we
can assume that its range is [0, 1].) At every time step t, the
learning algorithm selects a decision $t
2, receives an out-
come |t
0, and suffers loss *($t
, |t
). More generally, we
may allow the learner to select a distribution Dt
over the
space of decisions, in which case it suffers the expected loss
of a decision randomly selected according to Dt
; that is, its
expected loss is 4(Dt
, |t
) where
4(D, |)=E$t D[*($, |)].
To decide on distribution Dt
, we assume that the learner
has access to a set of N experts. At every time step t, expert
i produces its own distribution Et
i on 2, and suffers loss
4(Et
i , |t
).
The goal of the learner is to combine the distributions
produced by the experts so as to suffer expected loss ``not
much worse'' than that of the best expert.
The results of Section 2 provide a method for solving this
problem. Specifically, we run algorithm Hedge(;), treating
each expert as a strategy. At every time step, Hedge(;)
produces a distribution pt
on the set of experts which is used
to construct the mixture distribution
Dt
= :
N
i=1
pt
i Et
i .
For any outcome |t
, the loss suffered by Hedge(;) will
then be
4(Dt
, |t
)= :
N
i=1
pt
i 4(Et
i , |t
).
Thus, if we define lt

i =4(Et
i , |t
) then the loss suffered by the
learner is pt
} lt
, i.e., exactly the mixture loss that was
analyzed in Section 2.
Hence, the bounds of Section 2 can be applied to our
current framework. For instance, applying Eq. (11), we
obtain the following:

Formalized review target

The archival source above is not asserted equivalent to this formalized target.

For a finite expert family of size N >= 2, a positive best-expert loss bound L-tilde no larger than the horizon, and the general decision-prediction mixture model, Hedge with beta = g(L-tilde/log N) has cumulative expected loss at most the best expert loss plus sqrt(2 L-tilde log N) + log N.

Archival source anchor: cited publication, lines 143-152

Expanded PaperInterface specification

```
∀ {Expert : Type u_1} {Decision : Type u_2} {Outcome : Type u_3} [inst : Fintype Expert] [inst_1 : Nonempty Expert]
  [inst_2 : MeasurableSpace Decision]
  (game : AppliedModelingLib.Learning.Online.GeneralDecisionHedgeGame Expert Decision Outcome)
  (state : AppliedModelingLib.Learning.Online.HedgeWeightState Expert),
  (∀ (expert : Expert), state.weight expert = 1 / ↑(Fintype.card Expert)) →
    ∀ (rounds : List (AppliedModelingLib.Learning.Online.GeneralDecisionHedgeRound game)) (lossBound : ℝ),
      (Finset.univ.inf' Finset.univ_nonempty fun (expert : Expert) =>
          (List.map
              (fun (round : AppliedModelingLib.Learning.Online.GeneralDecisionHedgeRound game) =>
                round.inducedLoss expert)
              rounds).sum) ≤
        lossBound →
      lossBound ≤ ↑rounds.length →
        ∀ (hlossBound_pos : 0 < lossBound) (hcard : 1 < Fintype.card Expert),
          AppliedModelingLib.Learning.Online.GeneralDecisionHedgeRound.hedgeCumulativeDecisionLoss
              (AppliedModelingLib.Learning.Online.hedgeDiscountFromBounds lossBound (Real.log ↑(Fintype.card Expert)))
              (FreundSchapire1997Hedge.theorem5_decision_prediction_generalSpec._proof_1 lossBound hlossBound_pos
                hcard)
              state rounds ≤
            (Finset.univ.inf' Finset.univ_nonempty fun (expert : Expert) =>
                (List.map
                    (fun (round : AppliedModelingLib.Learning.Online.GeneralDecisionHedgeRound game) =>
                      round.inducedLoss expert)
                    rounds).sum) +
              √(2 * lossBound * Real.log ↑(Fintype.card Expert)) +
            Real.log ↑(Fintype.card Expert)
```

Retained governing declarations

- AppliedModelingLib.Learning.Online.GeneralDecisionHedgeGame
- AppliedModelingLib.Learning.Online.GeneralDecisionHedgeRound
- AppliedModelingLib.Learning.Online.GeneralDecisionHedgeRound.hedgeCumulativeDecisionLoss
- AppliedModelingLib.Learning.Online.GeneralDecisionHedgeRound.inducedLoss
- AppliedModelingLib.Learning.Online.HedgeWeightState
- AppliedModelingLib.Learning.Online.hedgeDiscountFromBounds

Lean-elaborated claim roles

1. parameter: Type u_1
2. parameter: Type u_2
3. parameter: Type u_3
4. parameter: Fintype Expert
5. assumption: Nonempty Expert
6. parameter: MeasurableSpace Decision
7. parameter: AppliedModelingLib.Learning.Online.GeneralDecisionHedgeGame Expert Decision Outcome
8. parameter: AppliedModelingLib.Learning.Online.HedgeWeightState Expert
9. assumption: ∀ (expert : Expert), state.weight expert = 1 / ↑(Fintype.card Expert)
10. parameter: List (AppliedModelingLib.Learning.Online.GeneralDecisionHedgeRound game)
11. parameter: ℝ
12. assumption: (Finset.univ.inf' Finset.univ_nonempty fun (expert : Expert) =>
    (List.map
        (fun (round : AppliedModelingLib.Learning.Online.GeneralDecisionHedgeRound game) => round.inducedLoss expert)
        rounds).sum) ≤
  lossBound
13. assumption: lossBound ≤ ↑rounds.length
14. assumption: 0 < lossBound
15. assumption: 1 < Fintype.card Expert
16. conclusion: AppliedModelingLib.Learning.Online.GeneralDecisionHedgeRound.hedgeCumulativeDecisionLoss
    (AppliedModelingLib.Learning.Online.hedgeDiscountFromBounds lossBound (Real.log ↑(Fintype.card Expert)))
    (FreundSchapire1997Hedge.theorem5_decision_prediction_generalSpec._proof_1 lossBound hlossBound_pos hcard) state
    rounds ≤
  (Finset.univ.inf' Finset.univ_nonempty fun (expert : Expert) =>

```
    (List.map
       (fun (round : AppliedModelingLib.Learning.Online.GeneralDecisionHedgeRound game) =>
         round.inducedLoss expert)
       rounds).sum) +
  √(2 * lossBound * Real.log ↑(Fintype.card Expert)) +
Real.log ↑(Fintype.card Expert)
```

Lean proof endpoint (built separately)

FreundSchapire1997Hedge.theorem5_decision_prediction_general

Recorded source-to-Spec screening

Verdict: matches_approved_corrected_target
Reason: The Spec preserves the source's general decision-prediction mixture setting, best-expert comparator, and sqrt(2 L-tilde log N)+log N bound. It makes only the displayed tuning formula's finite-real domain explicit (finite expert family with N > 1 and L-tilde > 0), leaving decision and outcome spaces unrestricted as in the source; this is the documented correction.
Reviewer check: ☐ Matches formalized target
If left unchecked, explain the discrepancy or uncertainty below.
Reviewer annotation

Review row 6

Source locator: cited publication, lines 1099-1108

Verbatim source input

Theorem 6. Suppose the weak learning algorithm WeakLearn, when called by
AdaBoost, generates hypotheses with errors $\epsilon_1,\ldots,\epsilon_T$
(as defined in Step 3 of Fig. 2). Then the error
$\epsilon=\Pr_{i\sim D}[h_f(x_i)\neq y_i]$ of the final hypothesis h_f
output by AdaBoost is bounded above by
$$
\epsilon\leq 2^T\prod_{t=1}^T\sqrt{\epsilon_t(1-\epsilon_t)}. \tag{14}
$$

[Next verbatim source excerpt]

D(i)=1[U+0012 control character]N. The algorithm maintains a set of weights wt
over
the training examples. On iteration t a distribution pt
is
computed by normalizing these weights. This distribution is
fed to the weak learner WeakLearn which generates a
hypothesis ht that (we hope) has small error with respect to
the distribution.1
Using the new hypothesis ht , the boosting
125
A DECISION THEORETIC GENERALIZATION
1
Some learning algorithms can be generalized to use a given distribution
directly. For instance, gradient based algorithms can use the probability
associated with each example to scale the update step size which is based
on the example. If the algorithm cannot be generalized in this way, the
training sample can be re-sampled to generate a new set of training exam-
ples that is distributed according to the given distribution. The computa-
tion required to generate each re-sampled example takes O(log N) time.

[form-feed in source extraction]
File: 571J 150408 . By:CV . Date:28:07:01 . Time:05:54 LOP8M. V8.0. Page 01:01
Codes: 6184 Signs: 4760 . Length: 56 pic 0 pts, 236 mm
Algorithm AdaBoost
Input: sequence of N labeled examples ((x1 , y1), ..., (xN , yN))
distribution D over the N examples
weak learning algorithm WeakLearn
integer T specifying number of iterations
Initialize the weight vector: w1
i =D(i) for i=1, ..., N.
Do for t=1, 2, ..., T
1. Set
pt
=
wt
[U+001D control character]N
i=1 wt
i
2. Call WeakLearn, providing it with the distribution pt
; get back a
hypothesis ht : X[U+0014 control character] [0, 1].
3. Calculate the error of ht: =t=[U+001D control character]N
i=1 pt
i |ht(xi)& yi |.
4. Set ;t==t [U+0012 control character](1&=t).
5. Set the new weights vector to be
wt+1
i =wt
i ;1&|ht(xi)& yi |
t
Output the hypothesis
hf (x)=
{
1 if [U+001D control character]T
t=1 (log 1[U+0012 control character];t) ht(x)[U+001E control character]1
2 [U+001D control character]T
t=1 log 1[U+0012 control character];t
0 otherwise.
FIG. 2. The adaptive boosting algorithm.
algorithm generates the next weight vector wt+1
, and the
process repeats. After T such iterations, the final hypothesis
hf is output. The hypothesis hf combines the outputs of the
T weak hypotheses using a weighted majority vote.
We call the algorithm AdaBoost because, unlike previous
algorithms, it adjusts adaptively to the errors of the weak
hypotheses returned by WeakLearn. If WeakLearn is a PAC
weak learning algorithm in the sense defined above, then
=t[U+001D control character]1[U+0012 control character]2&# for all t (assuming the examples have been
generated appropriately with yi=c(xi) for some c # C).
However, such a bound on the error need not be known
ahead of time. Our results hold for any =t # [0, 1], and
depend only on the performance of the weak learner on
those distributions that are actually generated during the
boosting process.
The parameter ;t is chosen as a function of =t and is used
for updating the weight vector. The update rule reduces

the probability assigned to those examples on which the hypothesis makes a good prediction and increases the probability of the examples on which the prediction is poor.2
Note that AdaBoost, unlike boost-by-majority, combines the weak hypotheses by summing their probabilistic predictions. Drucker, Schapire and Simard [9], in experiments they performed using boosting to improve the performance of a real-valued neural network, observed that summing the outcomes of the networks and then selecting the best prediction performs better than selecting the best prediction of each network and then combining them with a majority rule. It is interesting that the new boosting algorithm's final hypothesis uses the same combination rule that was observed to be better in practice, but which previously lacked theoretical justification.
Since it was first introduced, several successful experiments have been conducted using AdaBoost, including work by the authors [12], Drucker and Cortes [8], Jackson and Craven [16], Quinlan [21], and Breiman [3].
4.2. Analysis
Comparing Figs. 1 and 2, there is an obvious similarity between the algorithms Hedge(;) and AdaBoost. This similarity reflects a surprising ``dual'' relationship between the on-line allocation model and the problem of boosting. Put another way, there is a direct mapping or reduction of the boosting problem to the on-line allocation problem. In

Formalized review target

The archival source above is not asserted equivalent to this formalized target.

For the finite Figure-2 AdaBoost execution with every observed error strictly between zero and one, the final hypothesis's training error is at most 2^T
↪ times the product of sqrt(epsilon_t(1-epsilon_t)); errors may lie on either side of one half.

Archival source anchor: cited publication, lines 178-186

Expanded PaperInterface specification

```
∀ {Training : Type} [inst : Fintype Training] [inst_1 : Nonempty Training]
  (state : AppliedModelingLib.Learning.Online.HedgeWeightState Training) (label : Training → ℝ),
  ∑ sample : Training, state.weight sample = 1 →
    (∀ (sample : Training), 0 ≤ label sample ∧ label sample ≤ 1) →
      (∀ (sample : Training), label sample = 0 ∨ label sample = 1) →
        ∀ (hypotheses : List (Training → ℝ)),
          (∀ hypothesis ∈ hypotheses, ∀ (sample : Training), 0 ≤ hypothesis sample ∧ hypothesis sample ≤ 1) →
            ∀ (hvalid : AppliedModelingLib.Learning.Online.AdaBoostAdmissible state label hypotheses),
              AppliedModelingLib.Learning.Online.adaBoostTrainingError state label hypotheses hvalid ≤
                AppliedModelingLib.Learning.Online.adaBoostTheorem6Factor state label hypotheses hvalid
```

Retained governing declarations

- AppliedModelingLib.Learning.Online.AdaBoostAdmissible
- AppliedModelingLib.Learning.Online.HedgeWeightState
- AppliedModelingLib.Learning.Online.adaBoostTheorem6Factor
- AppliedModelingLib.Learning.Online.adaBoostTrainingError

Lean-elaborated claim roles

1. parameter: Type
2. parameter: Fintype Training
3. assumption: Nonempty Training
4. parameter: AppliedModelingLib.Learning.Online.HedgeWeightState Training
5. parameter: Training → ℝ
6. assumption: ∑ sample : Training, state.weight sample = 1
7. assumption: ∀ (sample : Training), 0 ≤ label sample ∧ label sample ≤ 1
8. assumption: ∀ (sample : Training), label sample = 0 ∨ label sample = 1
9. parameter: List (Training → ℝ)
10. assumption: ∀ hypothesis ∈ hypotheses, ∀ (sample : Training), 0 ≤ hypothesis sample ∧ hypothesis sample ≤ 1
11. assumption: AppliedModelingLib.Learning.Online.AdaBoostAdmissible state label hypotheses
12. conclusion: AppliedModelingLib.Learning.Online.adaBoostTrainingError state label hypotheses hvalid ≤
  AppliedModelingLib.Learning.Online.adaBoostTheorem6Factor state label hypotheses hvalid

Lean proof endpoint (built separately)

FreundSchapire1997Hedge.theorem6_adaBoost_finite

Recorded source-to-Spec screening

Verdict: matches_approved_corrected_target
Reason: The Spec is the source Theorem-6 training-error product bound for the finite Figure-2 execution. AdaBoostAdmissible supplies 0 < epsilon_t < 1, excluding only endpoint executions for which the printed beta quotient, logarithmic vote, or subsequent normalization has no stated finite-real convention; errors on either side of one half remain allowed.
Reviewer check: ☐ Matches formalized target

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 7

Source locator: cited publication, lines 1347-1355

Verbatim source input

Theorem 8. Let H be a class of binary functions of VC-dimension $d\geq2$.
Then the VC-dimension of $\Theta_T(H)$ is at most
$2(d+1)(T+1)\log_2(e(T+1))$.

[Next verbatim source excerpt]

1
2#2
ln
1
=|. (23)
Note, however, that when the errors of the hypotheses
generated by WeakLearn are not uniform, Theorem 6
implies that the final error depends on the error of all of the
weak hypotheses. Previous bounds on the errors of boosting
algorithms depended only on the maximal error of the
weakest hypothesis and ignored the advantage that can be
gained from the hypotheses whose errors are smaller. This
advantage seems to be very relevant to practical applica-
tions of boosting, because there one expects the error of the
learning algorithm to increase as the distributions fed to
WeakLearn shift more and more away from the target
distribution.
4.3. Generalization Error
We now come back to discussing the error of the final
hypothesis outside the training set. Theorem 6 guarantees
that the error of hf on the sample is small; however, the
quantity that interests us is the generalization error of hf ,
which is the error of hf over the whole instance space X; that
is, =g=Pr(x, y)t P[hf (x){ y]. In order to make =g close to
the empirical error =^ on the training set, we have to restrict
the choice of hf in some way. One natural way of doing this
in the context of boosting is to restrict the weak learner to
choose its hypotheses from some simple class of functions
and restrict T, the number of weak hypotheses that are com-
bined to make hf . The choice of the class of weak hypotheses
is specific to the learning problem at hand and should reflect
our knowledge about the properties of the unknown con-
cept. As for the choice of T, various general methods can be
devised. One popular method is to use an upper bound
on the VC-dimension of the concept class. This method
is sometimes called ``structural risk minimization.'' See
Vapnik's book [23] for an extensive discussion of the
theory of structural risk minimization. For our purposes, we
quote Vapnik's Theorem 6.7:
Theorem 7 (Vapnik). Let H be a class of binary func-
tions over some domain X. Let d be the VC-dimension of H.
Let P be a distribution over the pairs X_[0, 1]. For h # H,
define the (generalization) error of h with respect to P to be
=g(h).Pr(x, y)t P [h(x){ y].
Let S=[(x1 , y1), ..., (xN , yN)] be a sample (training set) of
N independent random examples drawn from X_[0, 1]
according to P. Define the empirical error of h with respect
to the sample S to be
=^ (h).
|[i: h(xi){ yi]|
N
.
Then, for any $>0 we have that
Pr
__h # H: |=^ (h)&=g(h)|
>2
[U+001C control character]
d(ln 2N[U+0012 control character]d+1)+ln 9[U+0012 control character]$
N &[U+001D control character]$
where the probability is computed with respect to the random
choice of the sample S.
Let %: R [U+0014 control character] [0, 1] be defined by
%(x)=
{
1 if x[U+001E control character]0
0 otherwise
and, for any class H of functions, let 3T (H) be the class of
all functions defined as a linear threshold of T functions
in H:
3T (H)=
{%
\:
T
t=1
at ht&b
+: b, a1 , ..., aT # R;
h1 , ..., hT # H
=.
Clearly, if all hypotheses generated by WeakLearn belong to
some class H, then the final hypothesis of AdaBoost, after T
rounds of boosting, belongs to 3T (H). Thus, the next
theorem provides an upper bound on the VC-dimension of

the class of final hypotheses generated by AdaBoost in terms
of the weak hypothesis class.
128 FREUND AND SCHAPIRE

[form-feed in source extraction]
File: 571J 150411 . By:CV . Date:28:07:01 . Time:05:54 LOP8M. V8.0. Page 01:01
Codes: 6095 Signs: 5147 . Length: 56 pic 0 pts, 236 mm

Expanded PaperInterface specification

```
∀ {X : Type u_1} (width d : ℕ) (concepts : Set (AppliedModelingLib.Statistics.BinaryClassifier X)),
  2 ≤ d →
    (AppliedModelingLib.Statistics.VCDimensionAtMost concepts d ∧
       ∃ (sample : Finset X), AppliedModelingLib.Statistics.binaryTraceVCDimension concepts sample = d) →
     ∀ (sample : Finset X),
       ↑(AppliedModelingLib.Statistics.binaryTraceVCDimension
            (AppliedModelingLib.Statistics.binaryThresholdCombinationClass concepts width) sample) ≤
         AppliedModelingLib.Statistics.sourceThresholdCombinationVCBound width d
```

Retained governing declarations

- AppliedModelingLib.Statistics.BinaryClassifier
- AppliedModelingLib.Statistics.VCDimensionAtMost
- AppliedModelingLib.Statistics.binaryThresholdCombinationClass
- AppliedModelingLib.Statistics.binaryTraceVCDimension
- AppliedModelingLib.Statistics.sourceThresholdCombinationVCBound

Lean-elaborated claim roles

1. parameter: Type u_1
2. parameter: ℕ
3. parameter: ℕ
4. parameter: Set (AppliedModelingLib.Statistics.BinaryClassifier X)
5. assumption: 2 ≤ d
6. assumption: AppliedModelingLib.Statistics.VCDimensionAtMost concepts d ∧
   ∃ (sample : Finset X), AppliedModelingLib.Statistics.binaryTraceVCDimension concepts sample = d
7. parameter: Finset X
8. conclusion: ↑(AppliedModelingLib.Statistics.binaryTraceVCDimension
      (AppliedModelingLib.Statistics.binaryThresholdCombinationClass concepts width) sample) ≤
   AppliedModelingLib.Statistics.sourceThresholdCombinationVCBound width d

Lean proof endpoint (built separately)

FreundSchapire1997Hedge.theorem8_weighted_threshold_vc

Recorded source-to-Spec screening **Verdict:** matches

Reason: The Spec identifies the source's Theta_T(H) with the affine threshold class of T binary weak hypotheses and gives the exact displayed 2(d+1)(T+1) log_2(e(T+1)) bound. The finite-trace VC formulation states that the source class has actual VC dimension d >= 2 and proves the same upper bound for every finite trace.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 8

Source locator: cited publication, lines 1486-1510

Verbatim source input

Theorem 9. Let $\epsilon_1,\ldots,\epsilon_T$ be as in Theorem 6, and let
$r(x_i)$ be as defined above. Let the modified final hypothesis be defined by
$h_f(x)=F(r(x))$, where for $r\in[0,1]$,
$$
F(1-r)=1-F(r),\qquad
F(r)\leq\frac12\left(\prod_{t=1}^T\beta_t\right)^{1/2-r}.
$$
Then the error ϵ of h_f is bounded above by
$$
\epsilon\leq 2^{T-1}\prod_{t=1}^T\sqrt{\epsilon_t(1-\epsilon_t)}.
$$

[Next verbatim source excerpt]

D(i)=1[U+0012 control character]N. The algorithm maintains a set of weights wt
over
the training examples. On iteration t a distribution pt
is
computed by normalizing these weights. This distribution is
fed to the weak learner WeakLearn which generates a
hypothesis ht that (we hope) has small error with respect to
the distribution.1
Using the new hypothesis ht , the boosting
125
A DECISION THEORETIC GENERALIZATION
1
Some learning algorithms can be generalized to use a given distribution
directly. For instance, gradient based algorithms can use the probability
associated with each example to scale the update step size which is based
on the example. If the algorithm cannot be generalized in this way, the
training sample can be re-sampled to generate a new set of training exam-
ples that is distributed according to the given distribution. The computa-
tion required to generate each re-sampled example takes O(log N) time.

[form-feed in source extraction]
File: 571J 150408 . By:CV . Date:28:07:01 . Time:05:54 LOP8M. V8.0. Page 01:01
Codes: 6184 Signs: 4760 . Length: 56 pic 0 pts, 236 mm
Algorithm AdaBoost
Input: sequence of N labeled examples ((x1 , y1), ..., (xN , yN))
distribution D over the N examples
weak learning algorithm WeakLearn
integer T specifying number of iterations
Initialize the weight vector: w1
i =D(i) for i=1, ..., N.
Do for t=1, 2, ..., T
1. Set
pt
=
wt
[U+001D control character]N
i=1 wt
i
2. Call WeakLearn, providing it with the distribution pt
; get back a
hypothesis ht : X[U+0014 control character] [0, 1].
3. Calculate the error of ht: =t=[U+001D control character]N
i=1 pt
i |ht(xi)& yi |.
4. Set ;t==t [U+0012 control character](1&=t).
5. Set the new weights vector to be
wt+1
i =wt
i ;1&|ht(xi)& yi |
t
Output the hypothesis
hf (x)=
{
1 if [U+001D control character]T
t=1 (log 1[U+0012 control character];t) ht(x)[U+001E control character]1
2 [U+001D control character]T
t=1 log 1[U+0012 control character];t
0 otherwise.
FIG. 2. The adaptive boosting algorithm.
algorithm generates the next weight vector wt+1
, and the
process repeats. After T such iterations, the final hypothesis
hf is output. The hypothesis hf combines the outputs of the
T weak hypotheses using a weighted majority vote.
We call the algorithm AdaBoost because, unlike previous
algorithms, it adjusts adaptively to the errors of the weak
hypotheses returned by WeakLearn. If WeakLearn is a PAC
weak learning algorithm in the sense defined above, then
=t[U+001D control character]1[U+0012 control character]2&# for all t (assuming the examples have been
generated appropriately with yi=c(xi) for some c # C).
However, such a bound on the error need not be known
ahead of time. Our results hold for any =t # [0, 1], and
depend only on the performance of the weak learner on
those distributions that are actually generated during the

boosting process.
The parameter ;t is chosen as a function of =t and is used for updating the weight vector. The update rule reduces the probability assigned to those examples on which the hypothesis makes a good prediction and increases the probability of the examples on which the prediction is poor.2
Note that AdaBoost, unlike boost-by-majority, combines the weak hypotheses by summing their probabilistic predictions. Drucker, Schapire and Simard [9], in experiments they performed using boosting to improve the performance of a real-valued neural network, observed that summing the outcomes of the networks and then selecting the best prediction performs better than selecting the best prediction of each network and then combining them with a majority rule. It is interesting that the new boosting algorithm's final hypothesis uses the same combination rule that was observed to be better in practice, but which previously lacked theoretical justification.
Since it was first introduced, several successful experiments have been conducted using AdaBoost, including work by the authors [12], Drucker and Cortes [8], Jackson and Craven [16], Quinlan [21], and Breiman [3].
4.2. Analysis
Comparing Figs. 1 and 2, there is an obvious similarity between the algorithms Hedge(;) and AdaBoost. This similarity reflects a surprising ``dual'' relationship between the on-line allocation model and the problem of boosting. Put another way, there is a direct mapping or reduction of the boosting problem to the on-line allocation problem. In

[Next verbatim source excerpt]

Theorem 6. Suppose the weak learning algorithm WeakLearn, when called by AdaBoost, generates hypotheses with errors =1 , ..., =T (as defined in Step 3 of Fig. 2.) Then the error ==PritD[hf (xi){ yi] of the final hypothesis hf output by AdaBoost is bounded above by
=[U+001D control character]2T
`
T
t=1
- =t(1&=t). (14)

[Next verbatim source excerpt]

optimal decision rule says that we should predict the label with the highest likelihood, given the hypothesis values, i.e., we should predict 1 if
Pr[y=1 | h1(x), ..., hT (x)]>Pr[y=0 | h1(x), ..., hT (x)],
and otherwise we should predict 0.
This rule is especially easy to compute if we assume that the errors of the different hypotheses are independent of one another and of the target concept, that is, if we assume that the event ht(x){ y is conditionally independent of the actual label y and the predictions of all the other hypotheses h1(x), ..., ht&1(x), ht+1(x), ..., hT (x). In this case, by applying Bayes rule, we can rewrite the Bayes optimal decision rule in a particularly simple form in which we predict 1 if
Pr[y=1] `
t: ht(x)=0
=t `
t: ht(x)=1
(1&=t)
>Pr[y=0] `
t: ht(x)=0
(1&=t) `
t: ht(x)=1
=t ,
and 0 otherwise. Here =t=Pr[ht(x){ y]. We add to the set of hypotheses the trivial hypothesis h0 which always predicts the value 1. We can then replace Pr[y=0] by =0 . Taking the logarithm of both sides in this inequality and rearranging the terms, we find that the Bayes optimal decision rule is identical to the combination rule that is generated by AdaBoost.
If the errors of the different hypotheses are dependent, then the Bayes optimal decision rule becomes much more complicated. However, in practice, it is common to use the simple rule described above even when there is no justification for assuming independence. (This is sometimes called ``naive Bayes.'') An interesting and more principled alternative to this practice would be to use the algorithm AdaBoost to find a combination rule which, by Theorem 6, has a guaranteed non-trivial accuracy.
129
A DECISION THEORETIC GENERALIZATION

[form-feed in source extraction]
File: 571J 150412 . By:CV . Date:28:07:01 . Time:05:54 LOP8M. V8.0. Page 01:01
Codes: 5125 Signs: 3769 . Length: 56 pic 0 pts, 236 mm
4.5. Improving the Error Bound
We show in this section how the bound given in Theorem 6 can be improved by a factor of two. The main idea of this improvement is to replace the ``hard'' [0, 1]-valued decision used by hf by a ``soft'' threshold.

To be more precise, let
r(x)=
[U+001D control character]T
t=1 (log 1
;t
) ht(x)
[U+001D control character]T
t=1 log 1
;t
be a weighted average of the weak hypotheses ht . We will
here consider final hypotheses of the form hf (x)=F(r(x))
where F: [0, 1] [U+0014 control character] [0, 1]. For the version of AdaBoost given
in Fig. 2, F(r) is the hard threshold that equals 1 if r[U+001E control character]1[U+0012 control character]2
and 0 otherwise. In this section, we will instead use soft
threshold functions that take values in [0, 1]. As mentioned
above, when hf (x) # [0, 1], we can interpret hf as a
randomized hypothesis and hf (x) as the probability of
predicting 1. Then the error EitD[|hf (xi)& yi |] is simply
the probability of an incorrect prediction.

Expanded PaperInterface specification

```
∀ {Training : Type} [inst : Fintype Training] [inst_1 : Nonempty Training]
  (state : AppliedModelingLib.Learning.Online.HedgeWeightState Training) (label : Training → ℝ),
  ∑ sample : Training, state.weight sample = 1 →
    (∀ (sample : Training), label sample = 0 ∨ label sample = 1) →
      ∀ (hypotheses : List (Training → ℝ)),
        (∀ hypothesis ∈ hypotheses, ∀ (sample : Training), 0 ≤ hypothesis sample ∧ hypothesis sample ≤ 1) →
          ∀ (hvalid : AppliedModelingLib.Learning.Online.AdaBoostAdmissible state label hypotheses),
            AppliedModelingLib.Learning.Online.adaBoostVoteWeightSum state label hypotheses hvalid ≠ 0 →
              (∀ (sample : Training),
                  0 ≤ AppliedModelingLib.Learning.Online.adaBoostNormalizedVote state label hypotheses hvalid sample ∧
                    AppliedModelingLib.Learning.Online.adaBoostNormalizedVote state label hypotheses hvalid sample ≤
                      1) →
                ∀ (threshold : ℝ → ℝ),
                  AppliedModelingLib.Learning.Online.AdaBoostSoftThreshold state label hypotheses hvalid threshold →
                    AppliedModelingLib.Learning.Online.adaBoostSoftTrainingError state label hypotheses hvalid
                        threshold ≤
                      AppliedModelingLib.Learning.Online.adaBoostTheorem9Factor state label hypotheses hvalid
```

Retained governing declarations

- AppliedModelingLib.Learning.Online.AdaBoostAdmissible
- AppliedModelingLib.Learning.Online.AdaBoostSoftThreshold
- AppliedModelingLib.Learning.Online.HedgeWeightState
- AppliedModelingLib.Learning.Online.adaBoostNormalizedVote
- AppliedModelingLib.Learning.Online.adaBoostSoftTrainingError
- AppliedModelingLib.Learning.Online.adaBoostTheorem9Factor
- AppliedModelingLib.Learning.Online.adaBoostVoteWeightSum

Lean-elaborated claim roles

1. parameter: Type
2. parameter: Fintype Training
3. assumption: Nonempty Training
4. parameter: AppliedModelingLib.Learning.Online.HedgeWeightState Training
5. parameter: Training → ℝ
6. assumption: ∑ sample : Training, state.weight sample = 1
7. assumption: ∀ (sample : Training), label sample = 0 ∨ label sample = 1
8. parameter: List (Training → ℝ)
9. assumption: ∀ hypothesis ∈ hypotheses, ∀ (sample : Training), 0 ≤ hypothesis sample ∧ hypothesis sample ≤ 1
10. assumption: AppliedModelingLib.Learning.Online.AdaBoostAdmissible state label hypotheses
11. assumption: AppliedModelingLib.Learning.Online.adaBoostVoteWeightSum state label hypotheses hvalid ≠ 0
12. assumption: ∀ (sample : Training),
  0 ≤ AppliedModelingLib.Learning.Online.adaBoostNormalizedVote state label hypotheses hvalid sample ∧
    AppliedModelingLib.Learning.Online.adaBoostNormalizedVote state label hypotheses hvalid sample ≤ 1
13. parameter: ℝ → ℝ
14. assumption: AppliedModelingLib.Learning.Online.AdaBoostSoftThreshold state label hypotheses hvalid threshold
15. conclusion: AppliedModelingLib.Learning.Online.adaBoostSoftTrainingError state label hypotheses hvalid threshold ≤
  AppliedModelingLib.Learning.Online.adaBoostTheorem9Factor state label hypotheses hvalid

Lean proof endpoint (built separately)

FreundSchapire1997Hedge.theorem9_soft_adaBoost_error

Recorded source-to-Spec screening **Verdict:** matches

Reason: Compared theorem9_soft_adaboost_error's complete verbatim Theorem 9 and Section 4.5 bundle (cited publication:1486-1510), the expanded proposition and all 15 claim roles, and every one of its 21 displayed prerequisites. Normalized nonnegative initial weights represent D; binary labels and [0,1] weak predictions give the source's probability of error. The adaptive errors, beta_t=epsilon_t/(1-epsilon_t), and weights w_i*beta_t^(1-|h_t-y_i|) are computed along exactly the same recursive Figure 2 execution. The conditions 0<epsilon_t<1 and nonzero sum_t log(1/beta_t) make the source's displayed real logarithms and quotient r(x) defined; requiring each evaluated r(x_i) in [0,1] is precisely the stated domain of F:[0,1]->[0,1]. No positive-edge, positive-total-coefficient, or independence premise is introduced: individual vote coefficients may be zero or negative. The displayed threshold predicate imposes range, F(1-r)=1-F(r), and the printed upper bound for every r in [0,1]. Its product_t beta_t^(1/2-r) equals (product_t beta_t)^(1/2-r) because all beta_t are positive; extending F to the rest of R has no effect on any evaluated term. The conclusion is exactly sum_i D(i)|F(r(x_i))-y_i| <= 2^(T-1) product_t sqrt(epsilon_t(1-epsilon_t)). The denominator condition excludes T=0, so the displayed natural-number subtraction in T-1 leaves the source constant unchanged. These are the formula's defined source domain and its full conclusion, so this is an ordinary source match.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 9

Source locator: cited publication, lines 1214-1230

Verbatim source input

Write each observed weak-hypothesis error as
$\epsilon_t=1/2-\gamma_t$. Theorem 6's bound can also be written as
$$
\epsilon\leq
\prod_{t=1}^T\sqrt{1-4\gamma_t^2}
=\exp\left(-\sum_{t=1}^T
  \operatorname{KL}(1/2\,\|\,1/2-\gamma_t)\right)
\leq\exp\left(-2\sum_{t=1}^T\gamma_t^2\right), \tag{21}
$$
where
$\operatorname{KL}(a\,\|\,b)=a\ln(a/b)+(1-a)\ln((1-a)/(1-b))$.

[Next verbatim source excerpt]

Theorem 6. Suppose the weak learning algorithm
WeakLearn, when called by AdaBoost, generates hypotheses
with errors =1 , ..., =T (as defined in Step 3 of Fig. 2.) Then the
error ==PritD[hf (xi){ yi] of the final hypothesis hf output
by AdaBoost is bounded above by
=[U+001D control character]2T
`
T
t=1
- =t(1&=t). (14)

Expanded PaperInterface specification

```
∀ {Training : Type} [inst : Fintype Training] [inst_1 : Nonempty Training]
  (state : AppliedModelingLib.Learning.Online.HedgeWeightState Training) (label : Training → ℝ),
  ∑ sample : Training, state.weight sample = 1 →
    (∀ (sample : Training), 0 ≤ label sample ∧ label sample ≤ 1) →
      (∀ (sample : Training), label sample = 0 ∨ label sample = 1) →
        ∀ (hypotheses : List (Training → ℝ)),
          (∀ hypothesis ∈ hypotheses, ∀ (sample : Training), 0 ≤ hypothesis sample ∧ hypothesis sample ≤ 1) →
            ∀ (hvalid : AppliedModelingLib.Learning.Online.AdaBoostAdmissible state label hypotheses),
              AppliedModelingLib.Learning.Online.adaBoostTrainingError state label hypotheses hvalid ≤
                  AppliedModelingLib.Learning.Online.adaBoostTheorem6Factor state label hypotheses hvalid ∧
                AppliedModelingLib.Learning.Online.adaBoostTheorem6Factor state label hypotheses hvalid =
                  Real.exp
                    (-(List.map (fun (error : ℝ) => AppliedModelingLib.Probability.binaryKLDivergence (1 / 2) error)
                        (AppliedModelingLib.Learning.Online.adaBoostErrors state label hypotheses hvalid)).sum) ∧
                Real.exp
                    (-(List.map (fun (error : ℝ) => AppliedModelingLib.Probability.binaryKLDivergence (1 / 2) error)
                        (AppliedModelingLib.Learning.Online.adaBoostErrors state label hypotheses hvalid)).sum) ≤
                  Real.exp
                    (-2 *
                      (List.map (fun (error : ℝ) => (1 / 2 - error) ^ 2)
                        (AppliedModelingLib.Learning.Online.adaBoostErrors state label hypotheses hvalid)).sum)
```

Retained governing declarations

- AppliedModelingLib.Learning.Online.AdaBoostAdmissible
- AppliedModelingLib.Learning.Online.HedgeWeightState
- AppliedModelingLib.Learning.Online.adaBoostErrors
- AppliedModelingLib.Learning.Online.adaBoostTheorem6Factor
- AppliedModelingLib.Learning.Online.adaBoostTrainingError
- AppliedModelingLib.Probability.binaryKLDivergence

Lean-elaborated claim roles

1. parameter: Type
2. parameter: Fintype Training
3. assumption: Nonempty Training
4. parameter: AppliedModelingLib.Learning.Online.HedgeWeightState Training
5. parameter: Training → ℝ
6. assumption: ∑ sample : Training, state.weight sample = 1
7. assumption: ∀ (sample : Training), 0 ≤ label sample ∧ label sample ≤ 1
8. assumption: ∀ (sample : Training), label sample = 0 ∨ label sample = 1
9. parameter: List (Training → ℝ)
10. assumption: ∀ hypothesis ∈ hypotheses, ∀ (sample : Training), 0 ≤ hypothesis sample ∧ hypothesis sample ≤ 1
11. assumption: AppliedModelingLib.Learning.Online.AdaBoostAdmissible state label hypotheses
12. conclusion: AppliedModelingLib.Learning.Online.adaBoostTrainingError state label hypotheses hvalid ≤
    AppliedModelingLib.Learning.Online.adaBoostTheorem6Factor state label hypotheses hvalid ∧
  AppliedModelingLib.Learning.Online.adaBoostTheorem6Factor state label hypotheses hvalid =
    Real.exp
      (-(List.map (fun (error : ℝ) => AppliedModelingLib.Probability.binaryKLDivergence (1 / 2) error)

```
        (AppliedModelingLib.Learning.Online.adaBoostErrors state label hypotheses hvalid)).sum) ∧
  Real.exp
    (-(List.map (fun (error : ℝ) => AppliedModelingLib.Probability.binaryKLDivergence (1 / 2) error)
        (AppliedModelingLib.Learning.Online.adaBoostErrors state label hypotheses hvalid)).sum) ≤
    Real.exp
      (-2 *
        (List.map (fun (error : ℝ) => (1 / 2 - error) ^ 2)
          (AppliedModelingLib.Learning.Online.adaBoostErrors state label hypotheses hvalid)).sum)
```

Lean proof endpoint (built separately)

FreundSchapire1997Hedge.equation21_adaBoost_binaryKL

Recorded source-to-Spec screening **Verdict:** matches

Reason: The direct Eq. (21) excerpt states the Theorem-6 training-error product bound, its binary-KL exponential identity, and the quadratic exponential upper bound. The Spec states those three conclusions under the finite binary AdaBoost execution that makes each displayed product and divergence defined.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 10

Source locator: cited publication, lines 1238-1243

Verbatim source input

If every weak hypothesis has the same error $1/2-\gamma$, this becomes
$$
\epsilon\leq(1-4\gamma^2)^{T/2}
=\exp\left(-T\operatorname{KL}(1/2\,\|\,1/2-\gamma)\right)
\leq\exp(-2T\gamma^2). \tag{22}
$$

Expanded PaperInterface specification

```
∀ {Training : Type} [inst : Fintype Training] [inst_1 : Nonempty Training]
  (state : AppliedModelingLib.Learning.Online.HedgeWeightState Training) (label : Training → ℝ),
  ∑ sample : Training, state.weight sample = 1 →
    (∀ (sample : Training), 0 ≤ label sample ∧ label sample ≤ 1) →
      (∀ (sample : Training), label sample = 0 ∨ label sample = 1) →
        ∀ (hypotheses : List (Training → ℝ)),
          (∀ hypothesis ∈ hypotheses, ∀ (sample : Training), 0 ≤ hypothesis sample ∧ hypothesis sample ≤ 1) →
            ∀ (hvalid : AppliedModelingLib.Learning.Online.AdaBoostAdmissible state label hypotheses) (gamma : ℝ),
              -(1 / 2) < gamma →
                gamma < 1 / 2 →
                  (∀ observed ∈ AppliedModelingLib.Learning.Online.adaBoostErrors state label hypotheses hvalid,
                      observed = 1 / 2 - gamma) →
                    AppliedModelingLib.Learning.Online.adaBoostTrainingError state label hypotheses hvalid ≤
                        (1 - 4 * gamma ^ 2) ^ (↑hypotheses.length / 2) ∧
                      (1 - 4 * gamma ^ 2) ^ (↑hypotheses.length / 2) =
                        Real.exp
                          (-↑hypotheses.length *
                            AppliedModelingLib.Probability.binaryKLDivergence (1 / 2) (1 / 2 - gamma)) ∧
                      Real.exp
                          (-↑hypotheses.length *
                            AppliedModelingLib.Probability.binaryKLDivergence (1 / 2) (1 / 2 - gamma)) ≤
                        Real.exp (-2 * ↑hypotheses.length * gamma ^ 2)
```

Retained governing declarations

- AppliedModelingLib.Learning.Online.AdaBoostAdmissible
- AppliedModelingLib.Learning.Online.HedgeWeightState
- AppliedModelingLib.Learning.Online.adaBoostErrors
- AppliedModelingLib.Learning.Online.adaBoostTrainingError
- AppliedModelingLib.Probability.binaryKLDivergence

Lean-elaborated claim roles

1. parameter: Type
2. parameter: Fintype Training
3. assumption: Nonempty Training
4. parameter: AppliedModelingLib.Learning.Online.HedgeWeightState Training
5. parameter: Training → ℝ
6. assumption: ∑ sample : Training, state.weight sample = 1
7. assumption: ∀ (sample : Training), 0 ≤ label sample ∧ label sample ≤ 1
8. assumption: ∀ (sample : Training), label sample = 0 ∨ label sample = 1
9. parameter: List (Training → ℝ)
10. assumption: ∀ hypothesis ∈ hypotheses, ∀ (sample : Training), 0 ≤ hypothesis sample ∧ hypothesis sample ≤ 1
11. assumption: AppliedModelingLib.Learning.Online.AdaBoostAdmissible state label hypotheses
12. parameter: ℝ
13. assumption: -(1 / 2) < gamma
14. assumption: gamma < 1 / 2
15. assumption: ∀ observed ∈ AppliedModelingLib.Learning.Online.adaBoostErrors state label hypotheses hvalid, observed = 1 / 2 - gamma
16. conclusion: AppliedModelingLib.Learning.Online.adaBoostTrainingError state label hypotheses hvalid ≤
    (1 - 4 * gamma ^ 2) ^ (↑hypotheses.length / 2) ∧
  (1 - 4 * gamma ^ 2) ^ (↑hypotheses.length / 2) =
    Real.exp (-↑hypotheses.length * AppliedModelingLib.Probability.binaryKLDivergence (1 / 2) (1 / 2 - gamma)) ∧
  Real.exp (-↑hypotheses.length * AppliedModelingLib.Probability.binaryKLDivergence (1 / 2) (1 / 2 - gamma)) ≤
    Real.exp (-2 * ↑hypotheses.length * gamma ^ 2)

Lean proof endpoint (built separately)

FreundSchapire1997Hedge.equation22_adaBoost_uniform_edge

Recorded source-to-Spec screening **Verdict:** matches

Reason: The byte-pinned Eq. (22) states trainingError <= (1-4 gamma^2)^(T/2) = exp(-T KL(1/2 || 1/2-gamma)) <= exp(-2 T gamma^2). The expanded Spec uses T = hypotheses.length, requires every sequential AdaBoost error to equal 1/2-gamma, and uses the exact printed binary KL formula. Its repaired conclusion contains the complete displayed chain, including both the power-to-KL equality and the KL-to-quadratic-exponential inequality, with the correct constants and directions. The strict domain -1/2 < gamma < 1/2 is the error domain supplied by admissibility and makes every displayed term well defined.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 11

Source locator: cited publication, lines 1249-1264

Verbatim source input

Consequently, a sufficient number of boosting iterations for final error at
most ϵ is
$$
T=\left\lceil
  \frac{1}{\operatorname{KL}(1/2\,\|\,1/2-\gamma)}
  \ln\frac1\epsilon
\right\rceil
\leq
\left\lceil\frac{1}{2\gamma^2}\ln\frac1\epsilon\right\rceil. \tag{23}
$$

[Next verbatim source excerpt]

hypotheses are equal to 1[U+0012 control character]2&#, Eq. (21) simplifies to
=[U+001D control character](1&4#2
)T[U+0012 control character]2
=exp(&T } KL(1[U+0012 control character]2 & 1[U+0012 control character]2&#))
[U+001D control character]exp(&2T#2
). (22)

Expanded PaperInterface specification

```
∀ {Training : Type} [inst : Fintype Training] [inst_1 : Nonempty Training]
  (state : AppliedModelingLib.Learning.Online.HedgeWeightState Training) (label : Training → ℝ),
  ∑ sample : Training, state.weight sample = 1 →
    (∀ (sample : Training), 0 ≤ label sample ∧ label sample ≤ 1) →
      (∀ (sample : Training), label sample = 0 ∨ label sample = 1) →
        ∀ (hypotheses : List (Training → ℝ)),
          (∀ hypothesis ∈ hypotheses, ∀ (sample : Training), 0 ≤ hypothesis sample ∧ hypothesis sample ≤ 1) →
            ∀ (hvalid : AppliedModelingLib.Learning.Online.AdaBoostAdmissible state label hypotheses)
              (gamma target : ℝ),
              0 < gamma →
                gamma < 1 / 2 →
                  0 < target →
                    target < 1 →
                      (∀ observed ∈ AppliedModelingLib.Learning.Online.adaBoostErrors state label hypotheses hvalid,
                          observed = 1 / 2 - gamma) →
                        ⌈Real.log (1 / target) /
                              AppliedModelingLib.Probability.binaryKLDivergence (1 / 2) (1 / 2 - gamma)⌉₊ ≤
                          ⌈Real.log (1 / target) / (2 * gamma ^ 2)⌉₊ ∧
                        (⌈Real.log (1 / target) /
                              AppliedModelingLib.Probability.binaryKLDivergence (1 / 2) (1 / 2 - gamma)⌉₊ ≤
                            hypotheses.length →
                          AppliedModelingLib.Learning.Online.adaBoostTrainingError state label hypotheses hvalid ≤
                            target)
```

Retained governing declarations

- AppliedModelingLib.Learning.Online.AdaBoostAdmissible
- AppliedModelingLib.Learning.Online.HedgeWeightState
- AppliedModelingLib.Learning.Online.adaBoostErrors
- AppliedModelingLib.Learning.Online.adaBoostTrainingError
- AppliedModelingLib.Probability.binaryKLDivergence

Lean-elaborated claim roles

1. parameter: Type
2. parameter: Fintype Training
3. assumption: Nonempty Training
4. parameter: AppliedModelingLib.Learning.Online.HedgeWeightState Training
5. parameter: Training → ℝ
6. assumption: ∑ sample : Training, state.weight sample = 1
7. assumption: ∀ (sample : Training), 0 ≤ label sample ∧ label sample ≤ 1
8. assumption: ∀ (sample : Training), label sample = 0 ∨ label sample = 1
9. parameter: List (Training → ℝ)
10. assumption: ∀ hypothesis ∈ hypotheses, ∀ (sample : Training), 0 ≤ hypothesis sample ∧ hypothesis sample ≤ 1
11. assumption: AppliedModelingLib.Learning.Online.AdaBoostAdmissible state label hypotheses
12. parameter: ℝ
13. parameter: ℝ
14. assumption: 0 < gamma
15. assumption: gamma < 1 / 2
16. assumption: 0 < target
17. assumption: target < 1
18. assumption: ∀ observed ∈ AppliedModelingLib.Learning.Online.adaBoostErrors state label hypotheses hvalid, observed = 1 / 2 - gamma
19. conclusion: ⌈Real.log (1 / target) / AppliedModelingLib.Probability.binaryKLDivergence (1 / 2) (1 / 2 - gamma)⌉₊ ≤
    ⌈Real.log (1 / target) / (2 * gamma ^ 2)⌉₊ ∧
  (⌈Real.log (1 / target) / AppliedModelingLib.Probability.binaryKLDivergence (1 / 2) (1 / 2 - gamma)⌉₊ ≤
      hypotheses.length →
    AppliedModelingLib.Learning.Online.adaBoostTrainingError state label hypotheses hvalid ≤ target)

Lean proof endpoint (built separately)

FreundSchapire1997Hedge.equation23_adaBoost_quadratic_iterations

Recorded source-to-Spec screening **Verdict:** matches

Reason: The direct Eq. (23) excerpt gives the exact KL iteration threshold, its quadratic upper threshold, and sufficiency for a target error. The Spec preserves both ceilings and expresses sufficiency conditionally on the run length under the explicit positive-edge and target domains that keep the displayed denominators finite.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 12

Source locator: cited publication, lines 1735-1745

Verbatim source input

Theorem 10. Suppose WeakLearn, when called by AdaBoost.M1, generates
hypotheses with errors $\epsilon_1,\ldots,\epsilon_T$, where ϵ_t is
as defined in Fig. 3. Assume each $\epsilon_t\leq1/2$. Then the error
$\epsilon=\Pr_{i\sim D}[h_f(x_i)\neq y_i]$ is bounded above by
$$
\epsilon\leq2^T\prod_{t=1}^T\sqrt{\epsilon_t(1-\epsilon_t)}.
$$

[Next verbatim source excerpt]

a given instance x, now outputs the label y that maximizes
the sum of the weights of the weak hypotheses predicting
that label.
In the case of binary classification (k=2), a weak
hypothesis h with error significantly larger than 1[U+0012 control character]2 is of
equal value to one with error significantly less than 1[U+0012 control character]2 since
h can be replaced by 1&h. However, for k>2, a hypothesis
ht with error =t[U+001E control character]1[U+0012 control character]2 is useless to the boosting algorithm. If
Algorithm AdaBoost.M1
Input: sequence of N examples ((x1 , y1)..., (xN , yN)) with labels
yi # Y=[1, ..., k]
distribution D over the N examples
weak learning algorithm WeakLearn
integer T specifying number of iterations
Initialize the weight vector: w1
i =D(i) for i=1, ..., N.
Do for t=1, 2, ..., T
1. Set
pt
=
wt
[U+001D control character]N
i=1 wt
i
2. Call WeakLearn, providing it with the distribution pt
; get back a
hypothesis ht : X [U+0014 control character] Y.
3. Calculate the error of ht: =t=[U+001D control character]N
i=1 pt
i [U+001A control character]ht(xi){ yi[U+0019 control character].
If =t>1[U+0012 control character]2, then set T=t&1 and abort loop.
4. Set ;t==t [U+0012 control character](1&=t).
5. Set the new weights vector to be
wt+1
i =wt
i ;1&[U+001A control character]ht(xi){y
% i[U+0019 control character]
t
Output the hypothesis
hf (x)=arg max
y # Y
:
T
t=1
\log
1
;t+[U+001A control character]ht(x)= y[U+0019 control character].
FIG. 3. A first multi-class extension of AdaBoost.
131
A DECISION THEORETIC GENERALIZATION

[form-feed in source extraction]
File: 571J 150414 . By:CV . Date:28:07:01 . Time:05:54 LOP8M. V8.0. Page 01:01
Codes: 5566 Signs: 4204 . Length: 56 pic 0 pts, 236 mm
such a weak hypothesis is returned by the weak learner, our
algorithm simply halts, using only the weak hypotheses that
were already computed.

Formalized review target

The archival source above is not asserted equivalent to this formalized target.

For the finite Figure-3 AdaBoost.M1 execution with every observed mistake error positive and at most one half, the final argmax-vote classifier has training error at most 2^T times the product of sqrt(epsilon_t(1-epsilon_t)).

Archival source anchor: cited publication, lines 242-261

Expanded PaperInterface specification

```
∀ {Training Label : Type} [inst : Fintype Training] [inst_1 : Nonempty Training] [inst_2 : Fintype Label]
  [inst_3 : Nonempty Label] (state : AppliedModelingLib.Learning.Online.HedgeWeightState Training)
  (label : Training → Label),
  ∑ sample : Training, state.weight sample = 1 →
    ∀ (hypotheses : List (Training → Label))
      (hvalid : AppliedModelingLib.Learning.Online.AdaBoostM1Admissible state label hypotheses),
      (∀ error ∈ AppliedModelingLib.Learning.Online.adaBoostM1Errors state label hypotheses hvalid, error ≤ 1 / 2) →
```

```
(∀ (sample : Training) (candidate : Label),
    AppliedModelingLib.Learning.Online.adaBoostM1LabelScore state label hypotheses hvalid sample candidate ≤
      AppliedModelingLib.Learning.Online.adaBoostM1LabelScore state label hypotheses hvalid sample
        (AppliedModelingLib.Learning.Online.adaBoostM1FinalHypothesis state label hypotheses hvalid sample)) ∧
  AppliedModelingLib.Learning.Online.adaBoostM1TrainingError state label hypotheses hvalid ≤
    AppliedModelingLib.Learning.Online.adaBoostM1Theorem10Factor state label hypotheses hvalid
```

Retained governing declarations

- AppliedModelingLib.Learning.Online.AdaBoostM1Admissible
- AppliedModelingLib.Learning.Online.HedgeWeightState
- AppliedModelingLib.Learning.Online.adaBoostM1Errors
- AppliedModelingLib.Learning.Online.adaBoostM1FinalHypothesis
- AppliedModelingLib.Learning.Online.adaBoostM1LabelScore
- AppliedModelingLib.Learning.Online.adaBoostM1Theorem10Factor
- AppliedModelingLib.Learning.Online.adaBoostM1TrainingError

Lean-elaborated claim roles

1. parameter: Type
2. parameter: Type
3. parameter: Fintype Training
4. assumption: Nonempty Training
5. parameter: Fintype Label
6. assumption: Nonempty Label
7. parameter: AppliedModelingLib.Learning.Online.HedgeWeightState Training
8. parameter: Training → Label
9. assumption: ∑ sample : Training, state.weight sample = 1
10. parameter: List (Training → Label)
11. assumption: AppliedModelingLib.Learning.Online.AdaBoostM1Admissible state label hypotheses
12. assumption: ∀ error ∈ AppliedModelingLib.Learning.Online.adaBoostM1Errors state label hypotheses hvalid, error ≤ 1 / 2
13. conclusion: (∀ (sample : Training) (candidate : Label),
    AppliedModelingLib.Learning.Online.adaBoostM1LabelScore state label hypotheses hvalid sample candidate ≤
      AppliedModelingLib.Learning.Online.adaBoostM1LabelScore state label hypotheses hvalid sample
        (AppliedModelingLib.Learning.Online.adaBoostM1FinalHypothesis state label hypotheses hvalid sample)) ∧
  AppliedModelingLib.Learning.Online.adaBoostM1TrainingError state label hypotheses hvalid ≤
    AppliedModelingLib.Learning.Online.adaBoostM1Theorem10Factor state label hypotheses hvalid

Lean proof endpoint (built separately)

FreundSchapire1997Hedge.theorem10_adaBoostM1_error

Recorded source-to-Spec screening **Verdict:** matches_approved_corrected_target

Reason: The Spec retains the Figure-3 multiclass argmax-vote conclusion and the source product factor, together with the printed epsilon_t <= 1/2 regime. Its admissibility predicate excludes a zero-error executed round so the displayed quotient, logarithmic vote, and next normalized state are defined; that finite-execution treatment is the documented correction.

Reviewer check: ☐ Matches formalized target

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 13

Source locator: cited publication, lines 2028-2037

Verbatim source input

Theorem 11. Suppose WeakLearn, when called by AdaBoost.M2, generates
hypotheses with pseudo-losses $\epsilon_1,\ldots,\epsilon_T$, where
ϵ_t is as defined in Fig. 4. Then the error
$\epsilon=\Pr_{i\sim D}[h_f(x_i)\neq y_i]$ is bounded above by
$$
\epsilon\leq(k-1)2^T\prod_{t=1}^T\sqrt{\epsilon_t(1-\epsilon_t)}.
$$

[Next verbatim source excerpt]

PritD[hf (xi){ yi][U+001D control character]PritD[h
[U+0019 control character] f (i)=1].
Since each AdaBoost instance has a 0-label, PritD[h
[U+0019 control character] f (i)
=1] is exactly the error of h
[U+0019 control character] f . Applying Theorem 6, we can
obtain a bound on this error, completing the proof. K
It is possible, for this version of the boosting algorithm, to
allow hypotheses which generate for each x, not only a
predicted class label h(x) # Y, but also a ``confidence''
}(x) # [0, 1]. The learner then suffers loss 1[U+0012 control character]2&}(x)[U+0012 control character]2 if its
prediction is correct and 1[U+0012 control character]2+}(x)[U+0012 control character]2 otherwise. (Details
omitted.)
5.2. Second Multi-class Extension
In this section we describe a second alternative extension
of AdaBoost to the case where the label space Y is finite. This
extension requires more elaborate communication between
the boosting algorithm and the weak learning algorithm.
The advantage of doing this is that it gives the weak learner
more flexibility in making its predictions. In particular, it
sometimes enables the weak learner to make useful con-
tributions to the accuracy of the final hypothesis even when
the weak hypothesis does not predict the correct label with
probability greater than 1[U+0012 control character]2.
As described above, the weak learner generates
hypotheses which have the form h: X_Y [U+0014 control character] [0, 1]. Roughly
speaking, h(x, y) measures the degree to which it is believed
that y is the correct label associated with instance x. If, for
a given x, h(x, y) attains the same value for all y then we say
that the hypothesis is uninformative on instance x. On the
other hand, any deviation from strict equality is potentially
informative, because it predicts some labels to be more
plausible than others. As will be seen, any such information
is potentially useful for the boosting algorithm.
Below, we formalize the goal of the weak learner by
defining a pseudo-loss which measures the goodness of the
weak hypotheses. To motivate our definition, we first
consider the following setup. For a fixed training example
(xi , yi), we use a given hypothesis h to answer k&1 binary
questions. For each of the incorrect labels y{ yi we ask the
question:
``Which is the label of xi : yi or y?''
In other words, we ask that the correct label yi be
discriminated from the incorrect label y.
Assume momentarily that h only takes values in [0, 1].
Then if h(xi , y)=0 and h(xi , yi)=1, we interpret h's
answer to the question above to be yi (since h deems yi to
be a plausible label for xi , but y is considered implausible).
Likewise, if h(xi , y)=1 and h(xi , yi)=0 then the answer is
y. If h(xi , y)=h(xi , yi), then one of the two answers is
chosen uniformly at random.
132 FREUND AND SCHAPIRE

[form-feed in source extraction]
File: 571J 150415 . By:CV . Date:28:07:01 . Time:05:54 LOP8M. V8.0. Page 01:01
Codes: 5466 Signs: 4208 . Length: 56 pic 0 pts, 236 mm
In the more general case that h takes values in [0, 1], we
interpret h(x, y) as a randomized decision for the procedure
above. That is, we first choose a random bit b(x, y) which
is 1 with probability h(x, y) and 0 otherwise. We then apply
the above procedure to the stochastically chosen binary
function b. The probability of choosing the incorrect answer
y to the question above is
Pr[b(xi , yi)=07b(xi , y)=1]+1
2 Pr[b(xi , yi)=b(xi , y)]
=1
2 (1&h(xi , yi)+h(xi , y)).
If the answers to all k&1 questions are considered
equally important, then it is natural to define the loss of the
hypothesis to be the average, over all k&1 questions, of the
probability of an incorrect answer:
1
k&1
:
y{ yi
1
2
(1&h(xi , yi)+h(xi , y))

=
1
2 \1&h(xi , yi)+
1
k&1
:
y{ yi
h(xi , y)
+. (24)
However, as was discussed in the introduction to Section 5, different discrimination questions are likely to have different importance in different situations. For example, considering the OCR problem described earlier, it might be that at some point during the boosting process, some example of the digit ``7'' has been recognized as being either a ``7'' or a ``9''. At this stage the question that discriminates between ``7'' (the correct label) and ``9'' is clearly much more important than the other eight questions that discriminate ``7'' from the other digits.

A natural way of attaching different degrees of importance to the different questions is to assign a weight to each question. So, for each instance xi and incorrect label y{ yi , we assign a weight q(i, y) which we associate with the question that discriminates label y from the correct label yi . We then replace the average used in Eq. (24) with an average weighted according to q(i, y); the resulting formula is called the pseudo-loss of h on training instance i with respect to q:
plossq(h, i).1
2
\1&h(xi , yi)+ :
y{ yi
q(i, y) h(xi , y)
+.
The function q=[1, ..., N]_Y [U+0014 control character] [0, 1], called the label weighting function, assigns to each example i in the training set a probability distribution over the k&1 discrimination problems defined above. So, for all i,
:
y{ yi
q(i, y)=1.
The weak learner's goal is to minimize the expected pseudo-loss for given distribution D and weighting function q:
plossD, q(h) :=EitD [plossq(h, i)].
As we have seen, by manipulating both the distribution on instances, and the label weighting function q, our boosting algorithm effectively forces the weak learner to focus not only on the hard instances, but also on the incorrect class labels that are hardest to eliminate. Conversely, this pseudo-loss measure may make it easier for the weak learner to get a weak advantage. For instance, if the weak learner can simply determine that a particular instance does not belong to a certain class (even if it has no idea which of the remaining classes is the correct one), then, depending on q, this may be enough to gain a weak advantage.

Theorem 11, the main result of this section, shows that a weak learner can be boosted if it can consistently produce weak hypotheses with pseudo-losses smaller than 1[U+0012 control character]2. Note that pseudo-loss 1[U+0012 control character]2 can be achieved trivially by any uninformative hypothesis. Furthermore, a weak hypothesis h with pseudo-loss =>1[U+0012 control character]2 is also beneficial to boosting since it can be replaced by the hypothesis 1&h whose pseudo-loss is 1&=<1[U+0012 control character]2.

Example 5. As a simple example illustrating the use of pseudo-loss, suppose we seek an oblivious weak hypothesis, i.e., a weak hypothesis whose value depends only on the class label y so that h(x, y)=h(y) for all x. Although oblivious hypotheses per se are generally too weak to be of interest, it may often be appropriate to find the best oblivious hypothesis on a part of the instance space (such as the set of instances covered by a leaf of a decision tree).

Let D be the target distribution, and q the label weighting function. For notational convenience, let us define q(i, yi) =&1 for all i so that
plossq(h, i)=1
2
\1+ :
y # Y
q(i, y) h(xi , y)
+.
Setting $(y)=[U+001D control character]i D(i) q(i, y), it can be verified that for an oblivious hypothesis h,
plossD, q(h)=1
2
\1+ :
y # Y
h(y) $(y)
+,
which is clearly minimized by the choice
h(y)=
{
1 if $(y)<0
0 otherwise.
Suppose now that q(i, y)=1[U+0012 control character](k&1) for y{ yi , and let d(y)=PritD[yi= y] be the proportion of examples with

[form-feed in source extraction]
File: 571J 150416 . By:CV . Date:28:07:01 . Time:05:54 LOP8M. V8.0. Page 01:01
Codes: 5640 Signs: 4166 . Length: 56 pic 0 pts, 236 mm
label y. Then it can be verified that h will always have
pseudo-loss strictly smaller than 1[U+0012 control character]2 except in the case of a
uniform distribution of labels (d(y)=1[U+0012 control character]k for all y). In
contrast, when the weak learner's goal is minimization of
prediction error (as in Section 5.1), it can be shown that an
oblivious hypothesis with prediction error strictly less than
1[U+0012 control character]2 can only be found when one label y covers more than 1[U+0012 control character]2
the distribution (d(y)>1[U+0012 control character]2). So in this case, it is much
easier to find a hypothesis with small pseudo-loss rather
than small prediction error.
On the other hand, if q(i, y)=0 for some values of y, then
the quality of prediction on these labels is of no conse-
quence. In particular, if q(i, y)=0 for all but one incorrect
label for each instance i, then in order to make the pseudo-
loss smaller than 1[U+0012 control character]2 the hypothesis has to predict the
correct label with probability larger than 1[U+0012 control character]2, which means
that in this case the pseudo-loss criterion is as stringent as
the usual prediction error. However, as discussed above,
this case is unavoidable because a hard binary classification
problem can always be embedded in a multi-class problem.
This example suggests that it may often be significantly
easier to find weak hypotheses with small pseudo-loss rather
than hypotheses whose prediction error is small. On the
other hand, our theoretical bound for boosting using the
prediction error (Theorem 10) is stronger than the bound
for ploss (Theorem 11). Empirical tests [12] have shown
that pseudo-loss is generally more successful when the weak
learners use very restricted hypotheses. However, for more
powerful weak learners, such as decision-tree learning algo-
rithms, there is little difference between using pseudo-loss
and prediction error.
Our algorithm called AdaBoost.M2, is shown in Fig. 4.
Here, we maintain weights wt
i, y for each instance i and each
label y # Y&[yi]. The weak learner must be provided both
with a distribution Dt and a label weight function qt . Both
of these are computed using the weight vector wt
as shown
in Step 1. The weak learner's goal then is to minimize the
pseudo-loss =t , as defined in Step 3. The weights are updated
as shown in Step 5. The final hypothesis hf outputs, for a
given instance x, the label y that maximizes a weighted
average of the weak hypothesis values ht(x, y).

[Next verbatim source excerpt]

mark AdaBoost variables with a tilde.
Algorithm AdaBoost.M2
Input: sequence of N examples ((x1 , y1)..., (xN , yN)) with labels
yi # Y=[1, ..., k]
distribution D over the N examples
weak learning algorithm WeakLearn
integer T specifying number of iterations
Initialize the weight vector: w1
i, y=D(i)[U+0012 control character](k&1) for i=1, ..., N, y # Y&[yi].
Do for t=1, 2, ..., T
1. Set Wt
i =[U+001D control character]y{ yi
wt
i, y ;
qt(i, y)=
wt
i, y
Wt
i
for y{ yi ; and set
Dt(i)=
Wt
i
[U+001D control character]N
i=1 Wt
i
.
2. Call WeakLearn, providing it with the distribution Dt and label
weighting function qt ; get back a hypothesis ht: X_Y [U+0014 control character] [0, 1].
3. Calculate the pseudo-loss of ht :
=t=
1
2
:
N
i=1
Dt(i)
\1&ht(xi , yi)+ :
y{ yi
qt(i, y) ht(xi , y)
+.
4. Set ;t==t [U+0012 control character](1&=t).
5. Set the new weights vector to be
wt+1
i, y =wt
i, y ;(1[U+0012 control character]2)(1+ht(xi , yi)&ht(xi , y))
t

for i=1, ..., N, y # Y&[yi].
Output the hypothesis
hf (x)=arg max
y # Y
:
T
t=1
\log
1
;t+ht(x, y).
FIG. 4. A second multi-class extension of AdaBoost.
For each training instance (xi , yi) and for each incorrect
label y # Y&[yi], we define one AdaBoost instance x
~ i, y=
(i, y) with associated label y
~ i, y=0. Thus, there are N
[U+0019 control character] =
N(k&1) AdaBoost instances, each indexed by a pair (i, y).
The distribution over these instances is defined to be D
[U+0019 control character] (i, y)
=D(i)[U+0012 control character](k&1). The tth hypothesis h
[U+0019 control character] t provided to AdaBoost
for this reduction is defined by the rule
h

Formalized review target

The archival source above is not asserted equivalent to this formalized target.

For the finite Figure-4 AdaBoost.M2 execution with every observed pseudo-loss strictly between zero and one, the final argmax-vote classifier has training error at most (k-1) 2^T times the product of sqrt(epsilon_t(1-epsilon_t)).

Archival source anchor: cited publication, lines 265-311

Expanded PaperInterface specification

```
∀ {Training Label : Type} [inst : Fintype Training] [inst_1 : Nonempty Training] [inst_2 : Fintype Label]
  [inst_3 : Nonempty Label] (label : Training → Label) (exampleWeight : Training → ℝ)
  (state :
    AppliedModelingLib.Learning.Online.HedgeWeightState
      (AppliedModelingLib.Learning.Online.AdaBoostM2Instance Training Label label)),
  (∀ (sample : Training), 0 ≤ exampleWeight sample) →
    ∑ sample : Training, exampleWeight sample = 1 →
      ∀ (hlabels : 1 < Fintype.card Label),
        (∀ (pair : AppliedModelingLib.Learning.Online.AdaBoostM2Instance Training Label label),
           state.weight pair = exampleWeight (↑pair).1 / ↑(Fintype.card Label - 1)) →
          ∑ pair : AppliedModelingLib.Learning.Online.AdaBoostM2Instance Training Label label, state.weight pair = 1 →
            ∀ (hypotheses : List (Training → Label → ℝ)),
              (∀ hypothesis ∈ hypotheses,
                 ∀ (sample : Training) (candidate : Label),
                   0 ≤ hypothesis sample candidate ∧ hypothesis sample candidate ≤ 1) →
                ∀ (hvalid : AppliedModelingLib.Learning.Online.AdaBoostM2Admissible label state hypotheses),
                  (∀ (sample : Training) (candidate : Label),
                     AppliedModelingLib.Learning.Online.adaBoostM2LabelScore label state hypotheses hvalid sample
                       candidate ≤
                       AppliedModelingLib.Learning.Online.adaBoostM2LabelScore label state hypotheses hvalid sample
                         (AppliedModelingLib.Learning.Online.adaBoostM2FinalHypothesis label state hypotheses hvalid
                           sample)) ∧
                    AppliedModelingLib.Learning.Online.adaBoostM2TrainingError label exampleWeight state hypotheses
                      hvalid ≤
                      AppliedModelingLib.Learning.Online.adaBoostM2Theorem11Factor label state hypotheses hvalid
```

Retained governing declarations

- AppliedModelingLib.Learning.Online.AdaBoostM2Admissible
- AppliedModelingLib.Learning.Online.AdaBoostM2Instance
- AppliedModelingLib.Learning.Online.HedgeWeightState
- AppliedModelingLib.Learning.Online.adaBoostM2FinalHypothesis
- AppliedModelingLib.Learning.Online.adaBoostM2LabelScore
- AppliedModelingLib.Learning.Online.adaBoostM2Theorem11Factor
- AppliedModelingLib.Learning.Online.adaBoostM2TrainingError

Lean-elaborated claim roles

1. parameter: Type
2. parameter: Type
3. parameter: Fintype Training
4. assumption: Nonempty Training
5. parameter: Fintype Label
6. assumption: Nonempty Label
7. parameter: Training → Label
8. parameter: Training → ℝ
9. parameter: AppliedModelingLib.Learning.Online.HedgeWeightState

(AppliedModelingLib.Learning.Online.AdaBoostM2Instance Training Label label)
10. assumption: ∀ (sample : Training), 0 ≤ exampleWeight sample
11. assumption: ∑ sample : Training, exampleWeight sample = 1
12. assumption: 1 < Fintype.card Label
13. assumption: ∀ (pair : AppliedModelingLib.Learning.Online.AdaBoostM2Instance Training Label label),
state.weight pair = exampleWeight (↑pair).1 / ↑(Fintype.card Label - 1)
14. assumption: ∑ pair : AppliedModelingLib.Learning.Online.AdaBoostM2Instance Training Label label, state.weight pair = 1
15. parameter: List (Training → Label → ℝ)
16. assumption: ∀ hypothesis ∈ hypotheses,
∀ (sample : Training) (candidate : Label), 0 ≤ hypothesis sample candidate ∧ hypothesis sample candidate ≤ 1
17. assumption: AppliedModelingLib.Learning.Online.AdaBoostM2Admissible label state hypotheses
18. conclusion: (∀ (sample : Training) (candidate : Label),
AppliedModelingLib.Learning.Online.adaBoostM2LabelScore label state hypotheses hvalid sample candidate ≤
AppliedModelingLib.Learning.Online.adaBoostM2LabelScore label state hypotheses hvalid sample
(AppliedModelingLib.Learning.Online.adaBoostM2FinalHypothesis label state hypotheses hvalid sample)) ∧
AppliedModelingLib.Learning.Online.adaBoostM2TrainingError label exampleWeight state hypotheses hvalid ≤
AppliedModelingLib.Learning.Online.adaBoostM2Theorem11Factor label state hypotheses hvalid

Lean proof endpoint (built separately)

FreundSchapire1997Hedge.theorem11_adaBoostM2_error

Recorded source-to-Spec screening **Verdict:** matches_approved_corrected_target

Reason: The Spec gives the Figure-4 initialization, final-label argmax property, and (k-1) times product training-error bound. It explicitly requires the pseudo-loss execution domain needed by the printed quotient, logarithmic weights, and recursive normalization, which is the documented finite-real correction rather than an unannounced source strengthening.

Reviewer check: ☐ Matches formalized target

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 14

Source locator: cited publication, lines 2418-2422

Verbatim source input

Theorem 12. Suppose WeakLearn, when called by AdaBoost.R, generates
hypotheses with errors $\epsilon_1,\ldots,\epsilon_T$, where ϵ_t is
as defined in Fig. 5. Then the mean squared error
$\epsilon=\mathbb E_{i\sim D}[(h_f(x_i)-y_i)^2]$ is bounded above by
$$
\epsilon\leq2^T\prod_{t=1}^T\sqrt{\epsilon_t(1-\epsilon_t)}. \tag{26}
$$

[Next verbatim source excerpt]

5.3. Boosting Regression Algorithms
In this section we show how boosting can be used for a
regression problem. In this setting, the label space is
Y=[0, 1]. As before, the learner receives examples (x, y)
chosen at random according to some distribution P, and its
goal is to find a hypothesis h: X [U+0014 control character] Y which, given some x
value, predicts approximately the value y that is likely to be
seen. More precisely, the learner attempts to find an h with
small mean squared error (MSE):
E(x, y)t P[(h(x)& y)2
]. (25)
Our methods can be applied to any reasonable bounded
error measure, but, for the sake of concreteness, we concen-
trate here on the squared error measure.
Following our approach for classification problems, we
assume that the leaner has been provided with a training set
(x1 , y1), ..., (xN , yN) of examples distributed according to
P, and we focus only on the minimization of the empirical
MSE:
1
N
:
N
i=1
(h(xi)& yi)2
.
Using techniques similar to those outlined in Section 4.3,
the true MSE given in Eq. (25) can be related to the empiri-
cal MSE.
To derive a boosting algorithm in this context, we reduce
the given regression problem to a binary classification
problem, and then apply AdaBoost. As was done for the
reductions used in the proofs of Theorems 10 and 11, we
mark with tildes all variables in the reduced (AdaBoost)
space. For each example (xi , yi) in the training set, we
define a continuum of examples indexed by pairs (i, y) for
all y # [0, 1]: the associated instance is x
~ i, y=(xi , y), and
the label is y
~ i, y=[U+001A control character]y[U+001E control character] yi[U+0019 control character]. (Recall that [U+001A control character]?[U+0019 control
↪ character] is 1 if predicate
? holds and 0 otherwise.) Although it is obviously infeasible
to explicitly maintain an infinitely large training set, we will
see later how this method can be implemented efficiently.
Also, although the results of Section 4 only dealt with
finitely large training sets, the extension to infinite training
sets is straightforward.
Thus, informally, each instance (xi , yi) is mapped to an
infinite set of binary questions, one for each y # Y, and each
of the form: ``Is the correct label yi bigger or smaller than y?''
In a similar manner, each hypothesis h: X [U+0014 control character] Y is reduced
to a binary-valued hypothesis h
[U+0019 control character] : X_Y[U+0014 control character] [0, 1] defined by
the rule
h
[U+0019 control character] (x, y)=[U+001A control character]y[U+001E control character]h(x)[U+0019 control character].
Thus, h
[U+0019 control character] attempts to answer these binary questions in a
natural way using the estimated value h(x).
Finally, as was done for classification problems, we
assume we are given a distribution D over the training set;
ordinarily, this will be uniform so that D(i)=1[U+0012 control character]N. In our
reduction, this distribution is mapped to a density D
[U+0019 control character] over
pairs (i, y) in such a way that minimization of classification
error in the reduced space is equivalent to minimization of
MSE for the original problem. To do this, we define
D
[U+0019 control character] (i, y)=
D(i) | y& yi |
Z
where Z is a normalization constant:
Z= :
N
i=1
D(i) |
1
0

| y& yi | dy.
It is straightforward to show that 1[U+0012 control character]4[U+001D control character]Z[U+001D control character]1[U+0012 control character]2.
135
A DECISION THEORETIC GENERALIZATION

[form-feed in source extraction]
File: 571J 150418 . By:CV . Date:28:07:01 . Time:05:54 LOP8M. V8.0. Page 01:01
Codes: 5416 Signs: 3774 . Length: 56 pic 0 pts, 236 mm
If we calculate the binary error of h
[U+0019 control character] with respect to the
density D
[U+0019 control character] , we find that, as desired, it is directly proportional
to the mean squared error:
:
N
i=1
|
1
0
| y
~ i, y&h
[U+0019 control character] (x
~ i, y)| D
[U+0019 control character] (i, y) dy
=
1
Z
:
N
i=1
D(i)
}|
h(xi)
yi
| y& yi | dy
}
=
1
2Z
:
N
i=1
D(i)(h(xi)& yi)2
.
The constant of proportionality is 1[U+0012 control character](2Z) # [1, 2].
Unravelling this reduction, we obtain the regression
boosting procedure AdaBoost.R shown in Fig. 5. As
prescribed by the reduction, AdaBoost.R maintains a weight
wt
i, y for each instance i and label y # Y. The initial weight
function w1
is exactly the density D
[U+0019 control character] defined above. By nor-
malizing the weights wt
, a density pt
is defined at Step 1 and
provided to the weak learner at Step 2. The goal of the weak
learner is to find a hypothesis ht : X[U+0014 control character] Y that minimizes the
loss =t defined in Step 3. Finally, at Step 5, the weights are
updated as prescribed by the reduction.
The definition of =t at Step 3 follows directly from the
reduction above; it is exactly the classification error of h
[U+0019 control character] f in
the reduced space. Note that, similar to AdaBoost.M2,
AdaBoost.R not only varies the distribution over the
examples (xi , yi), but also modifies from round to round the
definition of the loss suffered by a hypothesis on each
example. Thus, although our ultimate goal is minimization
of the squared error, the weak learner must be able to
handle loss functions that are more complicated than MSE.
The final hypothesis hf also is consistent with the reduc-
tion. Each reduced weak hypothesis h
[U+0019 control character] f (x, y) is non-
decreasing as a function of y. Thus, the final hypothesis h
[U+0019 control character] f
generated by AdaBoost in the reduced space, being the
threshold of a weighted sum of these hypotheses, also is
non-decreasing as a function of y. As the output of h
[U+0019 control character] f is
binary, this implies that for every x there is one value of y
for which h
[U+0019 control character] f (x, y$)=0 for all y$< y and h
[U+0019 control character] f (x, y$)=1 for all
y$> y. This is exactly the value of y given by hf (x) as defined
in the figure. Note that hf is actually computing a weighted
median of the weak hypotheses.
At first, it might seem impossible to maintain weights wt
i, y
over an uncountable set of points. However, on closer
inspection, it can be seen that, when viewed as a function of
y, wt
i, y is a piece-wise linear function. For t=1, w1
i, y has two
linear pieces, and each update at Step 5 potentially breaks
one of the pieces in two at the point ht(xi). Initializing,
storing and updating such piece-wise linear functions are
all straightforward operations. Also, the integrals which

appear in the figure can be evaluated explicitly since these
only involve integration of piece-wise linear functions.

[Next verbatim source excerpt]

Algorithm AdaBoost.R
Input: sequence of N examples ((x1 , y1)..., (xN , yN)) with labels
yi # Y=[0, 1]
distribution D over the examples
weak learning algorithm WeakLearn
integer T specifying number of iterations
Initialize the weight vector:
w1
i, y=
D(i) | y& yi |
Z
for i=1, ..., N, y # Y, where
Z= :
N
i=1
D(i)
|
1
0
|y& yi | dy.
Do for t=1, 2, ..., T
1. Set
pt
=
wt
[U+001D control character]N
i=1 [U+001E control character]1
0 wt
i, y dy
.
2. Call WeakLearn, providing it with the density pt
; get back a
hypothesis ht : X_Y.
3. Calculate the loss of ht :
=t= :
N
i=1
}|
ht(xi)
yi
pt
i, y dy
}.
If =t>1[U+0012 control character]2, then set T=t&1 and abort the loop.
4. Set ;t==t [U+0012 control character](1&=t).
5. Set the new weights vector to be
wt+1
i, y =
{
wt
i, y
wt
i, y ;t
if yi[U+001D control character] y[U+001D control character]ht(xi) or ht(xi)[U+001D control character] y[U+001D control character] yi
otherwise.
for i=1, ..., N, y # Y.
Output the hypothesis
hf (x)=inf
{y # Y: :
t: ht(x)[U+001D control character] y
log(1[U+0012 control character];t)[U+001E control character]1
2 :
t
log(1[U+0012 control character];t)
=.
FIG. 5. An extension of AdaBoost to regression problems.
The following theorem describes our performance
guarantee for AdaBoost.R. The proof follows from the
reduction described above coupled with a direct application
of Theorem 6.

Formalized review target

The archival source above is not asserted equivalent to this formalized target.

For the finite Figure-5 AdaBoost.R execution with every observed reduced error positive and at most one half, the final weighted-median regressor has mean squared error at most 2^T times the product of sqrt(epsilon_t(1-epsilon_t)).

Archival source anchor: cited publication, lines 315-363

Expanded PaperInterface specification

```
∀ {Training : Type} [inst : Fintype Training] (exampleWeight label : Training → ℝ)
  (hexample_nonneg : ∀ (sample : Training), 0 ≤ exampleWeight sample)
  (hexample_total : ∑ sample : Training, exampleWeight sample = 1)
  (hlabel : ∀ (sample : Training), 0 ≤ label sample ∧ label sample ≤ 1) (hypotheses : List (Training → ℝ))
  (hhypotheses : ∀ hypothesis ∈ hypotheses, ∀ (sample : Training), 0 ≤ hypothesis sample ∧ hypothesis sample ≤ 1)
  (hvalid :
    AppliedModelingLib.Learning.Online.AdaBoostRAdmissible
      (AppliedModelingLib.Learning.Online.adaBoostRBaseState exampleWeight label hexample_nonneg hexample_total hlabel)
```

```
    label hypotheses),
  AppliedModelingLib.Learning.Online.adaBoostRMeanSquaredError exampleWeight label
    (AppliedModelingLib.Learning.Online.adaBoostRFinalHypothesis
      (AppliedModelingLib.Learning.Online.adaBoostRBaseState exampleWeight label hexample_nonneg hexample_total
        hlabel)
      label hypotheses hhypotheses hvalid) ≤
   AppliedModelingLib.Learning.Online.adaBoostRTheorem12Factor
    (AppliedModelingLib.Learning.Online.adaBoostRBaseState exampleWeight label hexample_nonneg hexample_total hlabel)
    label hypotheses hvalid
```

Retained governing declarations

- AppliedModelingLib.Learning.Online.AdaBoostRAdmissible
- AppliedModelingLib.Learning.Online.adaBoostRBaseState
- AppliedModelingLib.Learning.Online.adaBoostRFinalHypothesis
- AppliedModelingLib.Learning.Online.adaBoostRMeanSquaredError
- AppliedModelingLib.Learning.Online.adaBoostRTheorem12Factor

Lean-elaborated claim roles

1. parameter: Type
2. parameter: Fintype Training
3. parameter: Training → ℝ
4. parameter: Training → ℝ
5. assumption: ∀ (sample : Training), 0 ≤ exampleWeight sample
6. assumption: ∑ sample : Training, exampleWeight sample = 1
7. assumption: ∀ (sample : Training), 0 ≤ label sample ∧ label sample ≤ 1
8. parameter: List (Training → ℝ)
9. assumption: ∀ hypothesis ∈ hypotheses, ∀ (sample : Training), 0 ≤ hypothesis sample ∧ hypothesis sample ≤ 1
10. assumption: AppliedModelingLib.Learning.Online.AdaBoostRAdmissible
    (AppliedModelingLib.Learning.Online.adaBoostRBaseState exampleWeight label hexample_nonneg hexample_total hlabel)
    label hypotheses
11. conclusion: AppliedModelingLib.Learning.Online.adaBoostRMeanSquaredError exampleWeight label
      (AppliedModelingLib.Learning.Online.adaBoostRFinalHypothesis
        (AppliedModelingLib.Learning.Online.adaBoostRBaseState exampleWeight label hexample_nonneg hexample_total hlabel)
        label hypotheses hhypotheses hvalid) ≤
    AppliedModelingLib.Learning.Online.adaBoostRTheorem12Factor
      (AppliedModelingLib.Learning.Online.adaBoostRBaseState exampleWeight label hexample_nonneg hexample_total hlabel)
      label hypotheses hvalid

Lean proof endpoint (built separately)

FreundSchapire1997Hedge.theorem12_adaBoostR_error

Recorded source-to-Spec screening

Verdict: matches_approved_corrected_target
Reason: The Spec retains Figure 5's finite initialization, weighted-median final predictor, mean-squared-error conclusion, and Theorem-12 product factor. Its admissibility domain excludes a zero-error executed round only to give the source quotient, logarithmic vote, and later positive-density updates a finite meaning, as recorded in the approved correction note.
Reviewer check: ☐ Matches formalized target
If left unchecked, explain the discrepancy or uncertainty below.
Reviewer annotation